

**INSTITUTO TECNOLÓGICO DE SANTO DOMINGO (INTEC)**

Maestría en Ciencia de Datos

MCD512 · Big Data

# Análisis Exploratorio y Visualización de Datos con SQL y NoSQL

Northwind en SQL Server, HR en Oracle y sensores en MongoDB sobre Docker

**Estudiante:** Dan Carrión

**Matrícula:** 1131022

**Asignatura:** MCD512 · Big Data

**Docente:** Prof. José M. Aquino Cepeda, MSc

**Pruebas realizadas:** 11 de octubre de 2026

**Fecha de entrega:** 15 de octubre de 2026

**Repositorio:** github.com/dcarrioncordones/proyecto-final-db

**Dashboard:** dcarrioncordones.github.io/proyecto-final-db/dashboard

# Índice

1. Introducción y objetivos ..... 2
2. Metodología y herramientas ..... 4
3. Laboratorio en Docker y conexiones ..... 7
4. SQL Server y Northwind ..... 11
5. Oracle y recursos humanos ..... 24
6. MongoDB y sensores ..... 38
7. Extracción integrada, validación y dashboard ..... 52
8. Discusión y recomendaciones ..... 55
9. Conclusiones ..... 56
Referencias ..... 58
Anexo A. Reproducción del laboratorio ..... 59
Anexo B. Código fuente y configuración ..... 60
Anexo C. Capturas del dashboard ..... 61

# 1. Introducción y objetivos

Este trabajo transforma los resultados de tres gestores de bases de datos en visualizaciones que apoyan el análisis exploratorio y la toma de decisiones. Se utilizan la base **Northwind** en **SQL Server 2022**, el esquema **HR** en **Oracle Database Free** y una colección de lecturas de sensores en **MongoDB 8.0**. Los tres gestores se ejecutan como contenedores de Docker Desktop en un Mac con Apple Silicon y se consultan desde un cliente Python con los drivers ODBC de Microsoft y Oracle y con PyMongo.

Las conexiones se verificaron el 11 de octubre de 2026 con consultas reales sobre cada gestor. A partir de esas conexiones se extrajeron 830 pedidos y 2,155 líneas de venta de Northwind, 107 empleados de HR y 12,960 documentos de sensores. Sobre esos datos se construyeron **30 visualizaciones** (diez por gestor), un dashboard interactivo en HTML y un notebook de Jupyter. Cada figura indica qué requisito de la consigna cumple y cómo leerla, y se acompaña de su construcción, el resultado observado, la decisión que apoya, su límite de interpretación y el código utilizado.

El **análisis exploratorio de datos** consiste en examinar un conjunto de datos —cómo se distribuyen sus valores, qué variables se relacionan, qué tendencias muestran y qué valores son atípicos— antes de modelarlo o de tomar decisiones con él. Las visualizaciones son su herramienta principal, porque dejan ver patrones que una tabla de números esconde.

Las tres bases representan dos modelos de datos distintos. **Northwind** y **HR** son bases **relacionales (SQL)**: la información se reparte en tablas con columnas fijas, que se relacionan entre sí mediante claves y se consultan con SQL. La colección de sensores está en **MongoDB**, una base **documental (NoSQL)**: cada lectura es un documento JSON independiente, con sus propios campos, y se consulta con agregaciones. En concreto:

- **Northwind (SQL Server):** las ventas de una distribuidora de alimentos entre 1996 y 1998: pedidos, productos, clientes, empleados y proveedores.
- **HR (Oracle):** los empleados de una empresa: salarios, puestos, departamentos, ubicaciones y cambios de puesto.
- **Sensores (MongoDB):** lecturas horarias de temperatura, humedad y CO2 durante septiembre de 2026 en seis estaciones de Santo Domingo, Santiago y Punta Cana. Son datos simulados, generados para este trabajo.

En Northwind, las ventas netas suman 1,265,793.04 con un máximo mensual en abril de 1998 y una concentración del 45.0% en los diez principales clientes. En HR, el salario mediano es 6,200.00 y Americas concentra el 65.4% de la plantilla. En los sensores, el detector robusto basado en la mediana y la MAD identifica las 103 anomalías inyectadas con una precisión de 75.2%.

### Objetivos

- Establecer conexiones verificables a SQL Server y Oracle mediante ODBC, y a MongoDB mediante su driver para Python.
- Describir las tres bases de datos y extraer sus datos con consultas de sólo lectura, conservando relaciones, unidades y procedencia.

- Construir diez visualizaciones por gestor, seleccionando el tipo de gráfico según el *insight* buscado.
- Explicar cada visualización con su método, resultado, decisión que apoya y límites, junto con el código o fórmula utilizada.
- Integrar las visualizaciones en un dashboard con filtros, interactividad y un monitoreo simulado en tiempo real.
- Validar la precisión de los cálculos con comprobaciones automáticas de sumas, claves y reglas estadísticas.

### Organización del informe

La sección 2 describe la metodología, las herramientas, las definiciones de las métricas y un glosario de términos técnicos. La sección 3 documenta el laboratorio en Docker y la configuración de las conexiones. Las secciones 4, 5 y 6 corresponden a cada gestor: descripción de la base, carga, evidencia de conexión, diez visualizaciones y los *insights* obtenidos. La sección 7 presenta la extracción integrada, las validaciones y el dashboard; la 8 discute los resultados y la 9 muestra dónde se cumple cada requisito de la consigna y resume las conclusiones. Los anexos contienen los pasos de reproducción, el inventario del código fuente, los enlaces al repositorio y al dashboard, y las capturas del dashboard.

### Código fuente

Todo el código, las consultas, los datos extraídos, el dashboard y este informe están en el repositorio https://github.com/dcarrioncordones/proyecto-final-db. El archivo `COMO_CORRER.md` explica paso a paso cómo reproducir el laboratorio y regenerar los resultados.

### Dashboard interactivo

El dashboard está publicado en https://dcarrioncordones.github.io/proyecto-final-db/dashboard/ y se abre en cualquier navegador, sin instalar nada. Permite elegir la base de datos y cada una de sus diez visualizaciones, aplicar filtros, hacer zoom, inspeccionar valores al pasar el cursor y reproducir el monitoreo simulado de S10 (sección 7.2).

# 2. Metodología y herramientas

El trabajo siguió las cuatro fases de la guía del curso. Cada fase produce archivos que la siguiente reutiliza, de modo que las figuras, el dashboard y el informe se calculan sobre los mismos datos extraídos.

| Fase | Actividades realizadas | Resultado |
|---|---|---|
| 1. Conexión y extracción | Contenedores de los tres gestores, carga de Northwind, HR y sensores; conexiones ODBC y PyMongo; extracción de sólo lectura | CSV por tabla y colección, con hashes de procedencia |
| 2. Análisis exploratorio | Conversión de fechas, uniones con claves, estadística descriptiva, correlaciones y detección de atípicos con MAD | Tres tablas analíticas y resultados verificables |
| 3. Visualización | Selección del gráfico según el insight, paleta accesible, etiquetas y unidades; versión estática (PNG) e interactiva (Plotly) | 30 figuras con explicación |
| 4. Dashboarding | Selector de base y figura, filtros, KPI, panel explicativo, reproducción temporal y diseño responsivo; pruebas de uso | dashboard/index.html |

### Cómo se presenta cada visualización

En las secciones 4, 5 y 6, cada visualización indica el ejemplo de la consigna al que responde, por qué se eligió ese tipo de gráfico y cómo leerlo; luego, cómo se construye, el resultado, la decisión que apoya, su límite de interpretación y la consulta y el código utilizados.

## 2.1 Herramientas

| Componente | Versión / detalle | Uso en el trabajo |
|---|---|---|
| Docker Desktop | 4.94.0 (Engine 29.8.2, Compose 5.5.1), Apple Virtualization y Rosetta | Ejecución de los tres gestores y del cliente de análisis |
| SQL Server | 2022 Developer (imagen mssql/server:2022-latest, amd64) | Base Northwind |
| Oracle Database Free | Imagen database/free:23.26.3.0-lite-arm64 | Esquema HR en el servicio FREEPDB1 |
| MongoDB | 8.0 (arm64) | Base bigdata_final, colección sensores_datos |
| Drivers | Microsoft ODBC Driver 18; Oracle Instant Client 23.26 ODBC; unixODBC | Conexión ODBC a SQL Server y Oracle |
| Python | 3.11 con pyodbc 5.3, PyMongo 4.18, pandas 3.0, NumPy, SciPy | Conexión, extracción y análisis |
| Visualización | Matplotlib 3.11 (PNG) y Plotly 7.1 (interactivo) | Figuras del informe y del dashboard |
| Entornos | Jupyter Notebook y scripts Python; Safari para el dashboard | Ejecución reproducible y revisión |

**ODBC** (*Open Database Connectivity*) es un estándar para que un programa se conecte a distintas bases de datos con la misma interfaz. Cada gestor aporta su propio **driver**, el componente que traduce esa interfaz a su protocolo: Microsoft ODBC Driver 18 para SQL Server y Oracle Instant Client ODBC para Oracle. Desde

Python, la librería **pyodbc** usa esos drivers. MongoDB no se consulta por ODBC sino con su driver oficial para Python, **PyMongo**.

## 2.2 Definiciones y criterios de calidad

Las métricas se definieron antes de construir las figuras para evitar ambigüedades. La tabla resume, para cada base de datos, las definiciones aplicadas y las comprobaciones automáticas de `src/validar.py` que validan la precisión de sus visualizaciones.

| Base de datos | Definiciones y reglas | Comprobaciones automáticas |
|---|---|---|
| Northwind (SQL Server) | **Venta neta de línea** = UnitPrice × Quantity × (1 − Discount). Se usa el precio cobrado en cada pedido (*Order Details*), no el precio vigente del catálogo (*Products*), porque los precios cambiaron con el tiempo: por ejemplo, el Queso Cabrales se vendió a 14.00 en el pedido 10248 (julio de 1996) y hoy cuesta 21.00 en el catálogo; 658 de las 2,155 líneas tienen un precio distinto al del catálogo. Venta bruta total: 1,354,458.59; reducción por descuentos: 88,665.55. **Clave de línea**: OrderID + ProductID. | 5: uniones que no multiplican líneas; clave de línea única; fórmula de venta neta; ventas que concilian por mes, cliente, categoría y empleado; descuentos entre 0 y 1. |
| HR (Oracle) | **Antigüedad** = (fecha de referencia 08/10/2026 − hire_date) / 365.25 años. Las uniones izquierdas conservan al empleado sin departamento. | 4: conservación de los 107 empleados; existencia de supervisores; organigrama sin ciclos; salarios positivos. |
| Sensores (MongoDB) | **Fechas**: BSON Date en UTC, agrupadas en la zona America/Santo_Domingo. **Atípico**: \|valor − mediana\| / (1.4826 × MAD) > 3.5, con mediana y MAD por sensor y hora del día [11]. **Clave de lectura**: sensor_id + timestamp. | 6: clave sensor-tiempo única; sin nulos en campos críticos; 720 lecturas por sensor; puntaje MAD finito; regla de alertas reproducible; matriz de confusión que concilia. |
| Generales | Las figuras PNG y las interactivas se calculan sobre las mismas tablas. | 2: treinta visualizaciones generadas; archivos de figura válidos. |

Las 17 comprobaciones resultaron correctas sobre los datos extraídos de los tres gestores.

Cada comprobación protege contra un error concreto. Por ejemplo, si una unión entre tablas multiplicara las líneas de venta, el total de ventas saldría inflado; la comprobación «uniones que no multiplican líneas» confirma que después de unir *Order Details* con productos, clientes y empleados siguen existiendo exactamente 2,155 líneas.

Para la accesibilidad se usan paletas distinguibles para daltonismo, etiquetas y leyendas en todos los gráficos, y la información principal también se expresa en texto, de modo que el color no es la única vía de lectura.

## 2.3 Glosario de términos

| Término | Significado en este trabajo |
|---|---|
| Contenedor / imagen | Una imagen es un paquete con un programa y todo lo que necesita (por ejemplo, SQL Server); un contenedor es esa imagen en ejecución, aislada del resto del sistema. |
| Docker Compose | Herramienta que define en un archivo (`compose.yaml`) varios contenedores, sus puertos, volúmenes y red, y los inicia juntos. |
| Volumen | Espacio de almacenamiento de un contenedor que se conserva aunque el contenedor se detenga o se recree; guarda los datos de cada base. |
| Healthcheck | Prueba periódica que Docker ejecuta dentro del contenedor (por ejemplo, `SELECT 1`) para saber si el servicio responde; si responde, el contenedor aparece como *healthy*. |
| Esquema / PDB | En Oracle, un esquema es el conjunto de tablas de un usuario (HR); FREEPDB1 es la base de datos «conectable» (*pluggable*) dentro de Oracle Free donde vive ese esquema. |
| Documento / colección | En MongoDB, un documento es un registro en formato JSON (BSON internamente) y una colección es un conjunto de documentos, equivalente a una tabla. |
| Upsert | Operación que inserta un documento si no existe y lo actualiza si ya existe; evita duplicados al repetir una carga. |
| Hash SHA-256 | Huella digital de un archivo: un código que cambia por completo si se modifica un solo byte. Sirve para comprobar que un archivo no cambió. |
| MAD | Desviación absoluta mediana: mide la dispersión típica alrededor de la mediana; a diferencia de la desviación estándar, casi no se altera con valores extremos. |
| KDE | Estimación de densidad por núcleos: una curva suave que muestra dónde se concentran los valores, en lugar de barras de un histograma. |
| Precisión / sensibilidad | De las alertas emitidas, qué porcentaje eran anomalías reales (precisión); de las anomalías reales, qué porcentaje se detectó (sensibilidad). |

# 3. Laboratorio en Docker y conexiones

Instalar SQL Server, Oracle y MongoDB directamente en el Mac sería lento y difícil de reproducir. Con Docker, cada gestor corre en su propio contenedor, con la versión exacta indicada en un archivo de configuración; cualquiera puede recrear el mismo laboratorio con unos pocos comandos y borrarlo sin dejar rastros en el sistema.

Los tres gestores se ejecutan en un proyecto de Docker Compose llamado `bigdata-final`, con volúmenes persistentes y puertos publicados sólo en la interfaz local. Un cuarto servicio, `analisis`, es un cliente temporal de Python que contiene los drivers ODBC y PyMongo: se crea para cada tarea, se conecta a los gestores por la red interna de Compose y se elimina al terminar. Así no se instalan drivers en macOS y la configuración queda descrita en archivos versionables.

| Servicio | Contenedor | Datos | Puerto en el Mac |
|---|---|---|---|
| SQL Server 2022 | bigdata-sqlserver | Base Northwind | 127.0.0.1:1434 |
| Oracle Database Free | bigdata-oracle | Esquema HR, servicio FREEPDB1 | 127.0.0.1:1522 |
| MongoDB 8.0 | bigdata-mongodb | bigdata_final.sensores_datos | 127.0.0.1:27020 |
| Cliente de análisis | analisis (temporal) | Python, ODBC 18, Instant Client, PyMongo | Sin puerto |

Todas las operaciones se ejecutan con `docker/lab.py`, que envuelve los comandos de Docker Compose, espera a que cada gestor esté saludable, oculta las contraseñas en la salida e impide reinstalar una base que ya contiene datos.

## 3.1 Preparación de Docker Desktop

SQL Server 2022 y el cliente con el driver ODBC de Oracle sólo se distribuyen para arquitectura amd64. En un Mac con Apple Silicon, Docker Desktop los ejecuta mediante emulación; se activó **Rosetta** con el *Apple Virtualization framework* para acelerarla [10]. Oracle y MongoDB usan imágenes ARM64 nativas.

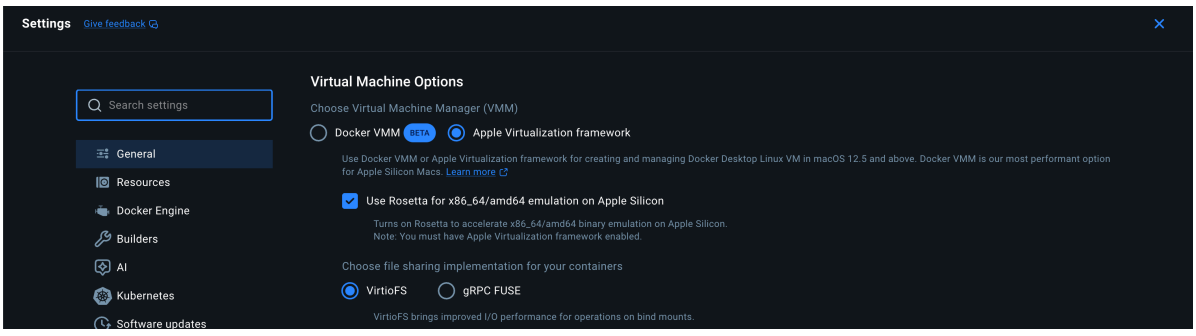



***Figura 1.*** *Configuración de Docker Desktop: Apple Virtualization framework con Rosetta para emular amd64.*

```
dancarrion@MacBook-Pro proyecto-final-db % docker version
Client:
 Version:           29.8.2
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        7fc2dff
 Built:             Wed Sep 30 19:32:03 2026
 OS/Arch:           darwin/arm64
 Context:           desktop-linux

Server: Docker Desktop 4.94.0 (241994)
 Engine:
  Version:          29.8.2
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.8
  Git commit:       8af9fe3
  Built:            Wed Sep 30 19:31:50 2026
  OS/Arch:          linux/arm64
  Experimental:     false
 containerd:
  Version:          v2.3.6
  GitCommit:        ee2735368117d2eb259779949d5e75cdafec9761
 runc:
  Version:          1.5.1
  GitCommit:        v1.5.1-0-g8f2685a4
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
dancarrion@MacBook-Pro proyecto-final-db % docker compose version
Docker Compose version v5.5.1
dancarrion@MacBook-Pro proyecto-final-db % python3 --version
Python 3.11.9
```

***Figura 2.*** *Versiones de Docker Engine, Docker Compose y Python utilizadas en el laboratorio.*

## 3.2 Configuración de credenciales

Las contraseñas no se escriben en el código ni en los archivos que se suben al repositorio. `lab.py configurar` genera `docker/.env` con cuatro contraseñas aleatorias distintas (SQL Server, administrador de Oracle, usuario HR y administrador de MongoDB) y permisos 600; no se imprimen ni se incluyen en la entrega. `lab.py validar` comprueba la sintaxis de Compose y las variables sin crear contenedores.

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py configurar
docker/.env ya existe y es válido; se conserva.
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py validar
Compose y configuración válidos.
```

***Figura 3.*** *Creación y validación de la configuración del laboratorio.*

## 3.3 Cadenas de conexión del cliente

Dentro de la red de Compose el cliente usa los nombres de servicio en lugar de `localhost`. El archivo `docker/cliente.py` construye las cadenas a partir de las variables de entorno, sin escribir contraseñas en disco:

```
SQLSERVER_ODBC = 'DRIVER={ODBC Driver 18 for SQL Server};SERVER=sqlserver,1433;'
                 'DATABASE=Northwind;UID=sa;PWD=***;Encrypt=yes;TrustServerCertificate=yes'
ORACLE_ODBC    = 'DRIVER={Oracle Instant Client ODBC};DBQ=oracle:1521/FREEPDB1;UID=hr;PWD=***'
MONGODB_URI    = 'mongodb://bigdata_admin:***@mongodb:27017/?authSource=admin'
```

Cada cadena indica qué driver usar (`DRIVER`), dónde está el gestor (`SERVER` o `DBQ`, con el nombre del contenedor y su puerto interno), a qué base o servicio entrar (`DATABASE`, o `FREEPDB1` en Oracle) y con qué usuario (`UID`). En SQL Server, `Encrypt=yes` cifra la comunicación y `TrustServerCertificate=yes` acepta el certificado que el propio laboratorio genera. En MongoDB, la URI incluye el usuario, el host, el

puerto y la base donde se valida el usuario (`authSource=admin`). Las contraseñas aparecen aquí como `***`.

La verificación (`src/conexiones.py verificar`) abre cada conexión y ejecuta una consulta sobre una tabla del trabajo; sólo registra éxito si el conteo es mayor que cero. El resultado se guarda con fecha UTC y conector en `evidencias/conexiones.json`.

```
def connect(motor):
    if motor in ('sqlserver', 'oracle'):
        import pyodbc
        return pyodbc.connect(os.environ[{'sqlserver': 'SQLSERVER_ODBC',
                                          'oracle': 'ORACLE_ODBC'}[motor]], timeout=10)
    from pymongo import MongoClient
    client = MongoClient(os.environ['MONGODB_URI'], serverSelectionTimeoutMS=5000)
    client.admin.command('ping')
    return client
```

## 3.4 Cliente de análisis con drivers ODBC

La imagen del cliente parte de `python:3.11-slim-bookworm` e instala unixODBC, Microsoft ODBC Driver 18 desde el repositorio de Microsoft [3] y Oracle Instant Client Basic y ODBC [4]. Los ZIP de Oracle se verifican con sus hashes SHA-256 antes de registrarse en `/etc/odbcinst.ini`. Luego se instalan las librerías de `requirements.txt`.

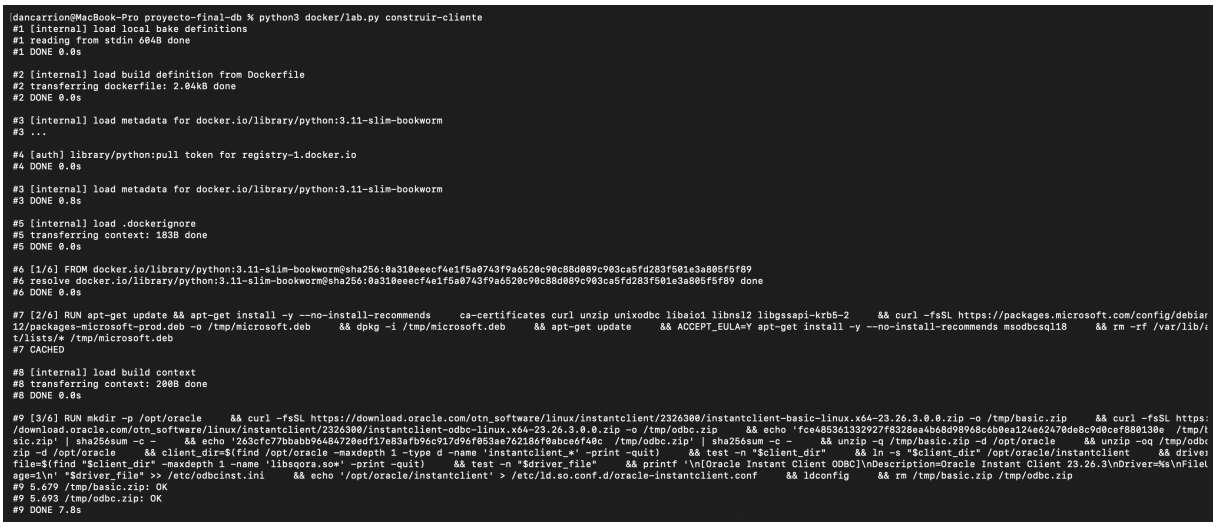



***Figura 4.*** *Construcción del cliente: instalación de msodbcsql18 y verificación de los paquetes de Oracle Instant Client (/tmp/basic.zip: OK, /tmp/odbc.zip: OK).*

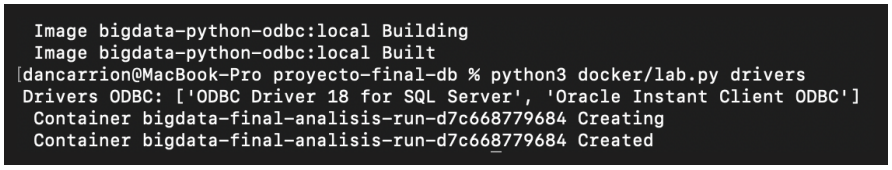



***Figura 5.*** *Imagen del cliente construida y drivers ODBC registrados: ODBC Driver 18 for SQL Server y Oracle Instant Client ODBC.*

## 3.5 Estado de los tres gestores

`lab.py estado` lista los contenedores del laboratorio. La columna STATUS indica cuánto tiempo llevan en ejecución y si su *healthcheck* responde (*healthy*), y la columna PORTS muestra qué puerto del Mac se conecta con el puerto interno de cada gestor (por ejemplo, `127.0.0.1:1434->1433/tcp` para SQL Server).

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py estado
NAME              IMAGE                                                COMMAND                  SERVICE     CREATED          STATUS                    PORTS
bigdata-mongodb   mongo:8.0                                            "docker-entrypoint.s…"   mongodb     15 minutes ago   Up 15 minutes (healthy)   127.0.0.1:27020->27017/tcp
bigdata-oracle    container-registry.oracle.com/database/free:23.26.3.0-lite-arm64   "/bin/bash -c $ORACL…"   oracle      28 minutes ago   Up 28 minutes (healthy)   127.0.0.1:1522->1521/tcp
bigdata-sqlserver mcr.microsoft.com/mssql/server:2022-latest           "/opt/mssql/bin/laun…"   sqlserver   53 minutes ago   Up 53 minutes (healthy)   127.0.0.1:1434->1433/tcp
```

***Figura 6.*** *Los tres contenedores en ejecución y con estado healthy, con sus puertos locales.*

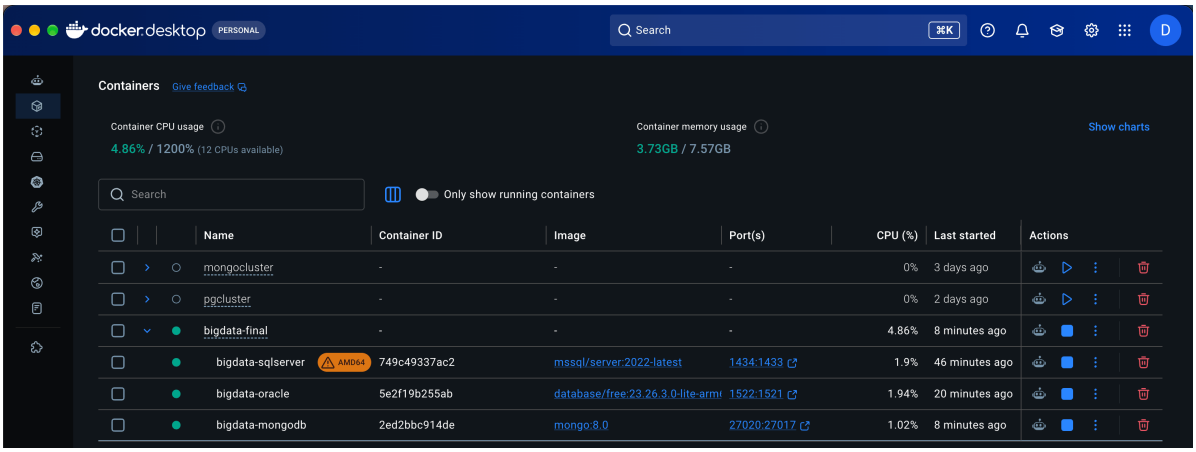



***Figura 7.*** *Vista de Docker Desktop: el proyecto bigdata-final con SQL Server (emulado, AMD64), Oracle y MongoDB.*

# 4. SQL Server y Northwind

## 4.1 Descripción de la base de datos

Northwind es la base de ejemplo de Microsoft que modela una distribuidora de alimentos: clientes, empleados, pedidos, líneas de pedido, productos, categorías, proveedores y transportistas [1]. Sus pedidos cubren del 4 de julio de 1996 al 6 de mayo de 1998. El análisis usa la cadena Orders → Order Details → Products → Categories, junto con Orders → Customers y Orders → Employees; Products → Suppliers aporta el análisis de precios. Todas las uniones son de muchos a uno y se comprueba que conserven las 2,155 líneas.

La diferencia entre las dos tablas centrales es importante: **Orders** tiene una fila por pedido (fecha, cliente y empleado), mientras que **Order Details** tiene una fila por cada producto dentro de un pedido (precio, cantidad y descuento). Un pedido con tres productos ocupa una fila en Orders y tres en Order Details; en promedio, cada pedido tiene unas 2.6 líneas. Por eso las ventas se calculan sumando líneas y los pedidos se cuentan en Orders. Esta es la primera línea de Order Details: el pedido 10248 incluye 12 unidades del producto 11 (Queso Cabrales) a 14.00, sin descuento.

```
OrderID  ProductID  UnitPrice  Quantity  Discount
10248    11         14.00      12        0.0
```

| Tabla | Filas | Uso en el análisis |
|---|---|---|
| Orders | 830 | Fecha del pedido, cliente y empleado |
| Order Details | 2,155 | Precio histórico, cantidad y descuento |
| Customers | 91 | Empresa y país del cliente |
| Products / Categories | 77 / 8 | Catálogo y clasificación comercial |
| Employees / Suppliers | 9 / 29 | Ventas por empleado y precios por proveedor |

## 4.2 Instalación y carga

`lab.py iniciar sqlserver` crea el contenedor a partir de la imagen oficial de SQL Server para Linux [8] y espera a que el *healthcheck* ejecute `SELECT 1` con sqlcmd. Después, `lab.py cargar sqlserver` crea la base vacía `Northwind` y ejecuta el script oficial `instnwnd.sql`; al terminar verifica los conteos de las siete tablas principales.

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py iniciar sqlserver
 Network bigdata-final_default Creating
 Volume bigdata-final_sqlserver-data Creating
 Volume bigdata-final_sqlserver-data Creating
 Network bigdata-final_default Creating
 Volume bigdata-final_sqlserver-data Created
 Volume bigdata-final_sqlserver-data Created
 Network bigdata-final_default Created
 Network bigdata-final_default Created
 Container bigdata-sqlserver Creating
 Container bigdata-sqlserver Created
 Container bigdata-sqlserver Starting
 Container bigdata-sqlserver Started
 Container bigdata-sqlserver Waiting
 Container bigdata-sqlserver Healthy
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py estado
NAME              IMAGE                                        COMMAND                  SERVICE     CREATED          STATUS                    PORTS
bigdata-sqlserver   mcr.microsoft.com/mssql/server:2022-latest   "/opt/mssql/bin/laun…"   sqlserver   50 seconds ago   Up 49 seconds (healthy)   127.0.0.1:1434->1433/tcp
```

***Figura 8.*** *Creación del contenedor de SQL Server y estado healthy en el puerto 1434.*

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py cargar sqlserver
Northwind cargada: 830 pedidos y 2155 líneas.
```

***Figura 9.*** *Carga de Northwind: 830 pedidos y 2,155 líneas.*

## 4.3 Conexión ODBC y evidencia

La conexión usa **pyodbc** con **Microsoft ODBC Driver 18**, con cifrado activo y confianza en el certificado autofirmado del laboratorio. La prueba consulta `DB_NAME()` y `COUNT(*)` de `dbo.Orders`: devolvió la base Northwind con 830 pedidos. Última verificación registrada: 2026-10-12 00:41:35 UTC.

**Cómo leer la salida de la verificación** (igual para los tres gestores): la primera línea muestra la fecha y hora UTC de la prueba; la segunda, el resultado (`CONEXIÓN VERIFICADA`); la tercera es el registro que se guarda en `evidencias/conexiones.json`, con el gestor, el conector (ODBC o PyMongo), la fecha, `"verified": true` y el conteo devuelto por la consulta (aquí, `"orders": 830`). Las líneas «Container … Creating/Created» corresponden al cliente temporal de análisis que se crea para hacer la prueba.

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py verificar sqlserver
VERIFICACIÓN REAL DE CONEXIONES — 2026-10-11T23:31:24.420664+00:00
sqlserver: CONEXIÓN VERIFICADA
{"motor": "sqlserver", "connector": "ODBC", "timestamp_utc": "2026-10-11T23:31:24.305046+00:00", "verified": true, "database": "Northwind", "orders": 830}
 Container bigdata-final-analisis-run-38302d894b8a Creating
 Container bigdata-final-analisis-run-38302d894b8a Created
```

***Figura 10.*** *Conexión exitosa a SQL Server mediante ODBC Driver 18 (CONEXIÓN VERIFICADA, 830 pedidos).*

## 4.4 Visualizaciones

Las diez figuras siguientes se calculan sobre las tablas extraídas de SQL Server. Las consultas equivalentes están en `sql/northwind.sql`. Los importes son ventas netas (después de descuentos), sin costos, impuestos ni flete; julio de 1996 y mayo de 1998 son meses parciales.

### N01 · Ventas mensuales y tendencia

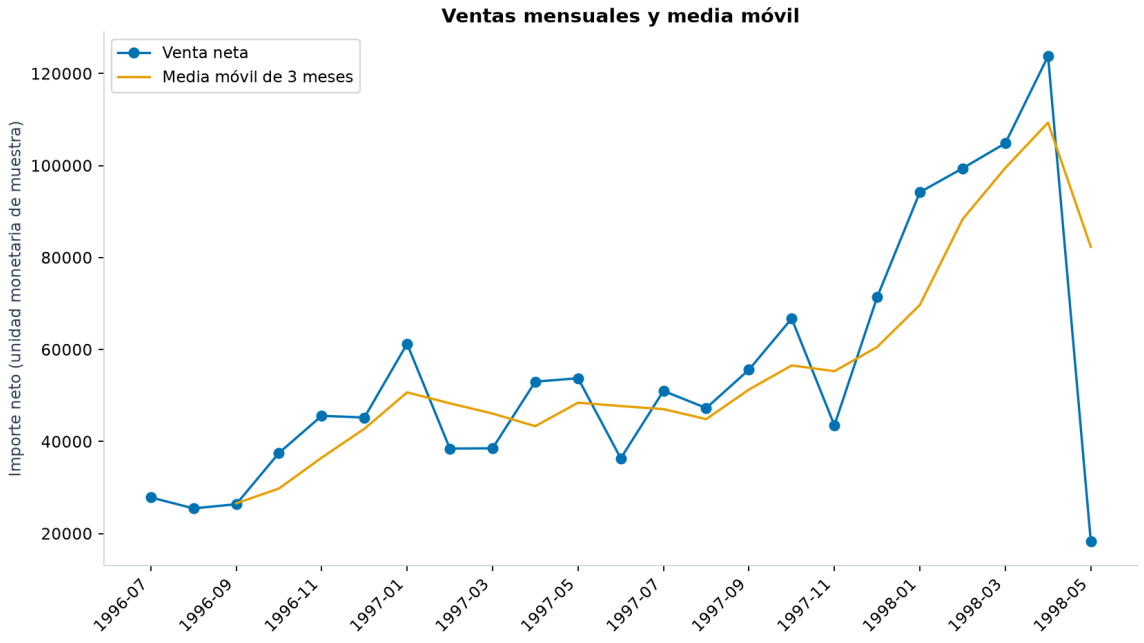



***Figura 11.*** *Ventas mensuales y tendencia (N01). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Dashboard de ventas mensuales con tendencias».

**Por qué este gráfico.** Una línea muestra la evolución en el tiempo; la media móvil de tres meses suaviza las oscilaciones para que se vea la tendencia.

**Cómo leerlo.** La línea azul es la venta neta de cada mes y la naranja, la media móvil. La caída del último punto se debe a que mayo de 1998 sólo tiene pedidos hasta el día 6.

**Construcción.** Se suma precio de línea × cantidad × (1 - descuento) por mes. La tendencia promedia tres observaciones mensuales consecutivas; conserva los períodos parciales y requiere tres observaciones.

**Resultado observado.** El total neto es 1,265,793.04; el máximo mensual corresponde a 1998-04, con 123,798.68.

**Insight y decisión que apoya.** Ajustar inventario y capacidad para meses de mayor demanda, confirmando el patrón con un período más amplio.

**Límite de interpretación.** Julio de 1996 y mayo de 1998 son períodos parciales. Las ventas no incluyen flete, impuestos ni costos. No se interpreta el último mes como caída económica.

**Consulta** (`sql/northwind.sql`):

```
-- N01: ventas mensuales y media móvil de 3 meses.
WITH mensual AS (
 SELECT DATEFROMPARTS(YEAR(o.OrderDate),MONTH(o.OrderDate),1) AS mes,
 SUM(CAST(d.UnitPrice AS decimal(18,4))*d.Quantity*(1-CAST(d.Discount AS decimal(9,4)))) AS
    neto
 FROM dbo.Orders o JOIN dbo.[Order Details] d ON o.OrderID=d.OrderID
 GROUP BY YEAR(o.OrderDate), MONTH(o.OrderDate))
SELECT mes,neto,CASE WHEN COUNT(*) OVER(ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT
    ROW)=3
 THEN AVG(neto) OVER(ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) END AS media_3m
FROM mensual ORDER BY mes;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
monthly = d.groupby('month').net.sum()
trend = monthly.rolling(3, min_periods=3).mean()
```

## N02 · Mapa de calor: categoría y país

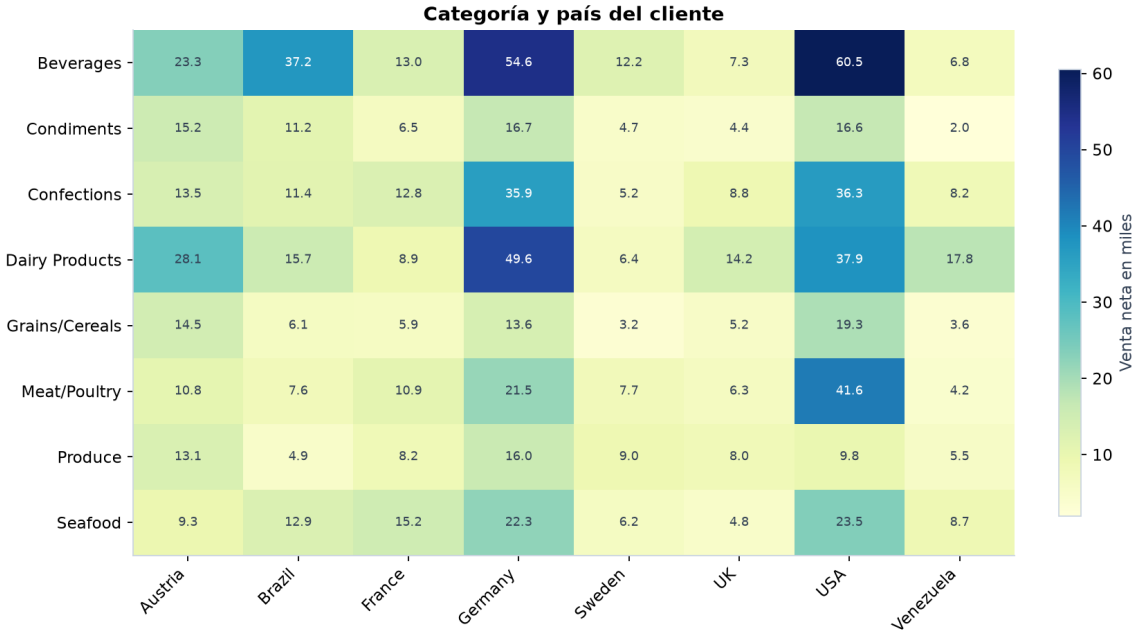



***Figura 12.*** *Mapa de calor: categoría y país (N02). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Mapa de calor de ventas por categoría y región».

**Por qué este gráfico.** Un mapa de calor compara dos variables categóricas a la vez (categoría de producto y país) en una sola cuadrícula; el color resalta las combinaciones más fuertes sin necesidad de leer cada número.

**Cómo leerlo.** Cada fila es una categoría de producto y cada columna uno de los ocho países con más compras. El número de cada celda es la venta neta en miles y el color se oscurece cuanto mayor es: Beverages en USA (60.5 mil) y en Germany (54.6 mil) son las celdas más oscuras. Se usa el país del cliente como «región» porque es la ubicación que registra Northwind.

**Construcción.** Se cruzan categorías con los ocho países con mayor venta neta. El color representa importe en miles; país es el domicilio del cliente.

**Resultado observado.** Beverages lidera por categoría con 267,868.18, equivalente al 21.2% del total.

**Insight y decisión que apoya.** Adaptar el portafolio a las combinaciones de mercado y categoría con mayor facturación.

**Límite de interpretación.** No se incluye el resto de países en la matriz. Una celda elevada puede resultar de pocos pedidos grandes; no equivale a mayor penetración de mercado.

**Consulta** (`sql/northwind.sql`):

```
-- N02: categoría por país del cliente; filtrar top 8 en Python.
SELECT ca.CategoryName,c.Country,SUM(d.UnitPrice*d.Quantity*(1-d.Discount)) AS neto
FROM dbo.[Order Details] d JOIN dbo.Orders o ON d.OrderID=o.OrderID
JOIN dbo.Products p ON d.ProductID=p.ProductID JOIN dbo.Categories ca ON
    ca.CategoryID=p.CategoryID
JOIN dbo.Customers c ON c.CustomerID=o.CustomerID GROUP BY ca.CategoryName,c.Country;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
d.pivot_table(index='CategoryName', columns='Country',
              values='net', aggfunc='sum', fill_value=0)
```

## N03 · Top 10 clientes por volumen monetario

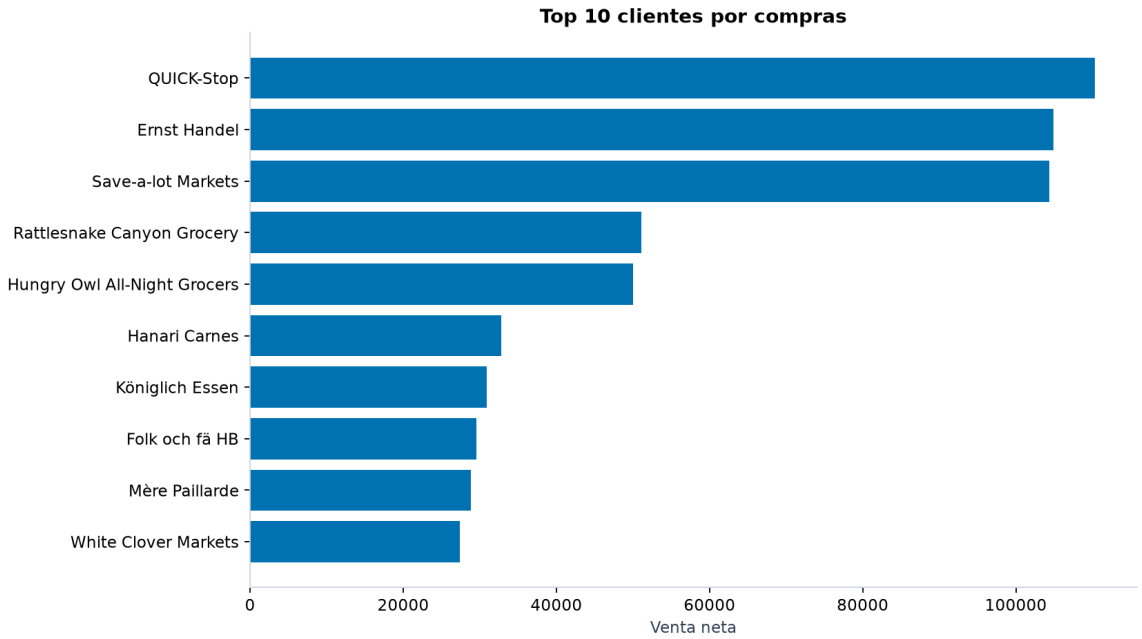



***Figura 13.*** *Top 10 clientes por volumen monetario (N03). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Gráfico de barras: Top 10 clientes por volumen de compras».

**Por qué este gráfico.** Las barras horizontales ordenadas facilitan comparar magnitudes y leer nombres largos de empresas.

**Cómo leerlo.** Cada barra es el total comprado por un cliente; el «volumen» se mide en dinero (venta neta), no en unidades.

**Construcción.** Se agrupan las líneas por cliente y se ordena la suma de venta neta. Volumen se define aquí como importe, no unidades.

**Resultado observado.** QUICK-Stop encabeza las compras con 110,277.30. Los diez primeros concentran 45.0% de las ventas.

**Insight y decisión que apoya.** Diseñar seguimiento de cuentas clave y evaluar riesgo de concentración.

**Límite de interpretación.** El ranking histórico no mide fidelidad ni valor futuro. Dos clientes del catálogo no tienen pedidos y aportan cero al análisis comercial.

**Consulta** (`sql/northwind.sql`):

```
-- N03: top clientes por importe neto.
SELECT TOP (10) c.CustomerID,c.CompanyName,SUM(d.UnitPrice*d.Quantity*(1-d.Discount)) AS neto
FROM dbo.Orders o JOIN dbo.[Order Details] d ON d.OrderID=o.OrderID
JOIN dbo.Customers c ON c.CustomerID=o.CustomerID
GROUP BY c.CustomerID,c.CompanyName ORDER BY neto DESC,c.CustomerID;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
clients = d.groupby('CompanyName').net.sum().nlargest(10)
```

## N04 · Evolución de ventas trimestrales

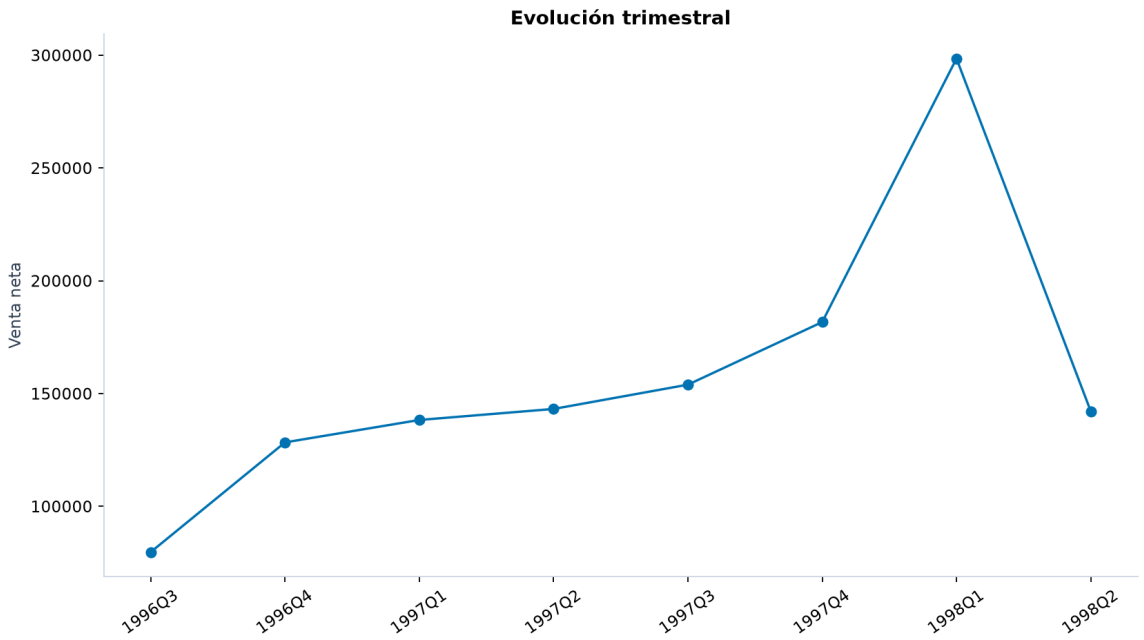



***Figura 14.*** *Evolución de ventas trimestrales (N04). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Línea de tiempo: Evolución de ventas trimestrales».

**Por qué este gráfico.** Agrupar por trimestre reduce el ruido mensual y muestra la evolución de mediano plazo.

**Cómo leerlo.** El primer y el último punto (1996Q3 y 1998Q2) son trimestres incompletos, por eso se ven más bajos.

**Construcción.** Cada pedido se asigna al trimestre de OrderDate; la suma se calcula a partir de líneas, sin multiplicar pedidos por uniones adicionales.

**Resultado observado.** El trimestre de mayor importe es 1998Q1, con 298,491.55.

**Insight y decisión que apoya.** Comparar los trimestres completos de 1997 antes de utilizar la serie para presupuestar.

**Límite de interpretación.** 1996-Q3 y 1998-Q2 tienen cobertura parcial. Una diferencia de duración impide compararlos directamente con trimestres completos.

**Consulta** (`sql/northwind.sql`):

```
-- N04: trimestre; cobertura parcial se documenta en el informe.
SELECT YEAR(o.OrderDate) AS anio,DATEPART(QUARTER,o.OrderDate) AS trimestre,
SUM(d.UnitPrice*d.Quantity*(1-d.Discount)) AS neto
FROM dbo.Orders o JOIN dbo.[Order Details] d ON d.OrderID=o.OrderID
GROUP BY YEAR(o.OrderDate),DATEPART(QUARTER,o.OrderDate) ORDER BY anio,trimestre;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
q = d.assign(quarter=d.OrderDate.dt.to_period('Q').astype(str))
q = q.groupby('quarter').net.sum()
```

## N05 · Treemap de productos por categoría

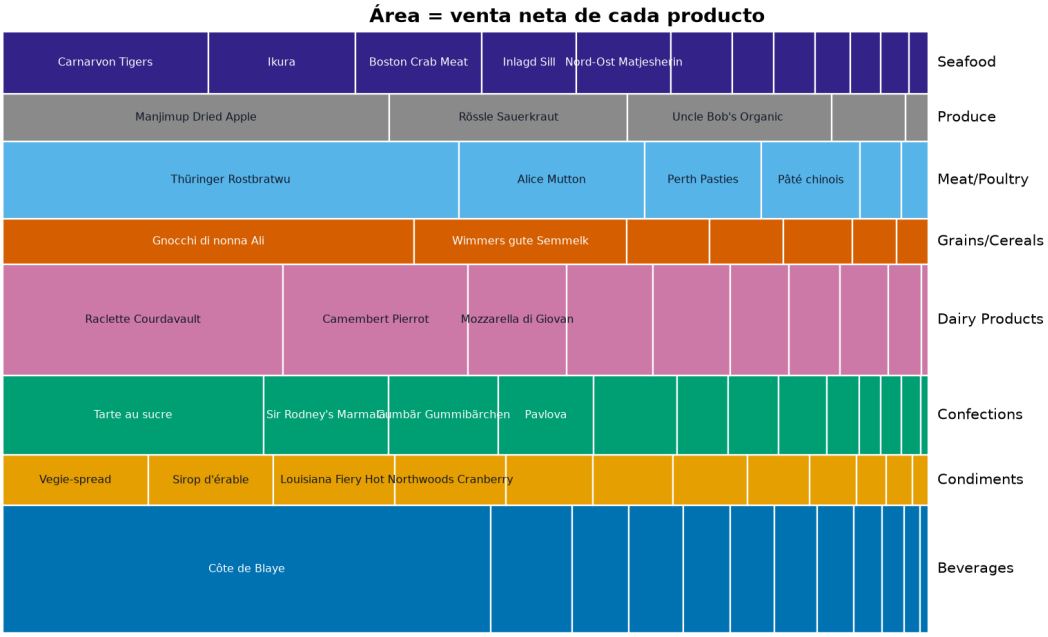



***Figura 15.*** *Treemap de productos por categoría (N05). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Treemap: Distribución de productos por categoría».

**Por qué este gráfico.** Un treemap muestra a la vez una jerarquía (categoría y producto) y el peso de cada elemento: el área de cada rectángulo es proporcional a lo que vende.

**Cómo leerlo.** Cada franja de color es una categoría y cada rectángulo dentro de ella, un producto. Cuanto más grande el rectángulo, mayor la venta neta de ese producto: Côte de Blaye, en Beverages, es el más grande. En el dashboard se puede hacer clic en una categoría para ver sólo sus productos.

**Construcción.** Cada área es proporcional a venta neta. Los productos se anidan en sus categorías; el dashboard permite abrir cada categoría.

**Resultado observado.** Côte de Blaye aporta 141,396.73, el mayor importe de un producto de la muestra.

**Insight y decisión que apoya.** Priorizar disponibilidad de productos de gran contribución y contrastar su margen cuando se disponga de costos.

**Límite de interpretación.** La muestra no contiene costo de compra ni utilidad. Este gráfico mide contribución a ingresos y no rentabilidad.

**Consulta** (`sql/northwind.sql`):

```
-- N05: treemap; ingresos, no utilidad.
SELECT ca.CategoryName,p.ProductName,SUM(d.UnitPrice*d.Quantity*(1-d.Discount)) AS neto
FROM dbo.[Order Details] d JOIN dbo.Products p ON p.ProductID=d.ProductID
JOIN dbo.Categories ca ON ca.CategoryID=p.CategoryID GROUP BY ca.CategoryName,p.ProductName;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
prod = d.groupby(['CategoryName','ProductName'], as_index=False).net.sum()
px.treemap(prod, path=['CategoryName','ProductName'], values='net')
```

## N06 · Dispersión: descuento y cantidad

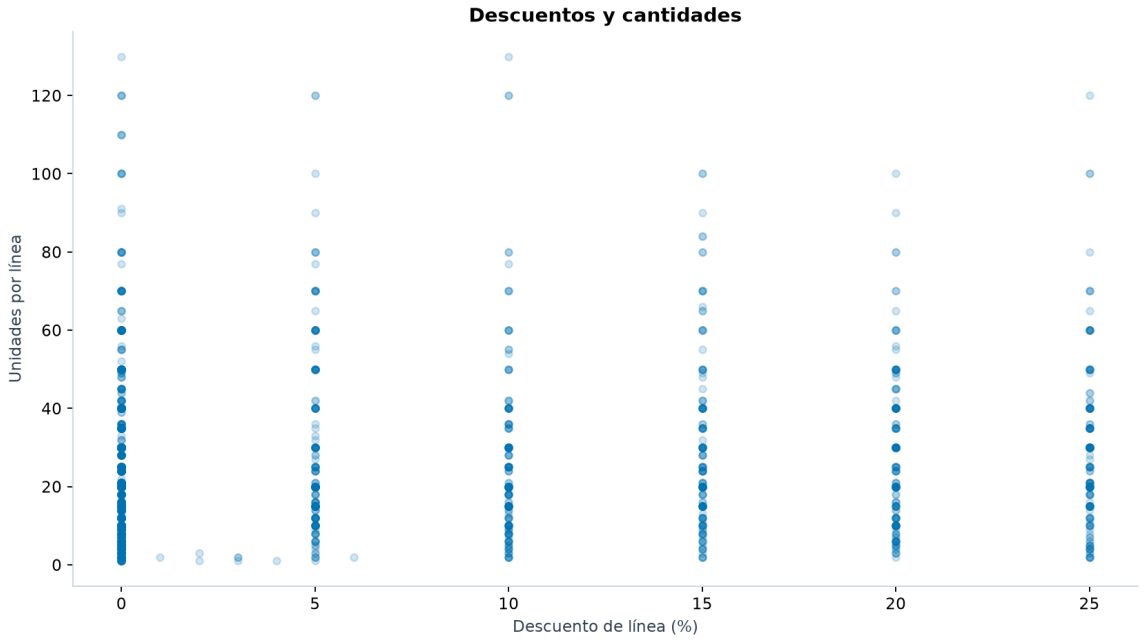



***Figura 16.*** *Dispersión: descuento y cantidad (N06). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Scatter plot: Relación entre descuentos y cantidades vendidas».

**Por qué este gráfico.** El gráfico de dispersión es la forma estándar de ver si dos variables numéricas se relacionan: si los descuentos aumentaran las cantidades, los puntos subirían de izquierda a derecha.

**Cómo leerlo.** Cada punto es una línea de pedido. El eje horizontal es el descuento aplicado, en porcentaje, y el vertical las unidades vendidas en esa línea. Los puntos forman columnas porque casi todos los descuentos son 0, 5, 10, 15, 20 o 25%; sólo unas pocas líneas tienen descuentos de 1 a 6%. Las columnas tienen alturas parecidas, lo que indica que el descuento casi no cambia la cantidad (r = 0.129).

**Construcción.** Cada punto corresponde a una línea. Se calcula Pearson entre descuento fraccional y cantidad; los descuentos repetidos producen columnas de puntos.

**Resultado observado.** La correlación observada es r = 0.129 en 2,155 líneas.

**Insight y decisión que apoya.** Explorar descuentos por producto y cliente antes de evaluar una política promocional.

**Límite de interpretación.** Una asociación no prueba que el descuento cause más ventas. Mezcla categorías, productos y condiciones de compra.

**Consulta** (`sql/northwind.sql`):

```
-- N06: observaciones para dispersión, una fila por línea.
SELECT d.OrderID,d.ProductID,d.Discount,d.Quantity,ca.CategoryName
FROM dbo.[Order Details] d JOIN dbo.Products p ON p.ProductID=d.ProductID
JOIN dbo.Categories ca ON ca.CategoryID=p.CategoryID;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
r = d.Discount.corr(d.Quantity)
px.scatter(d, x='Discount', y='Quantity', color='CategoryName')
```

## N07 · Distribución de países de clientes

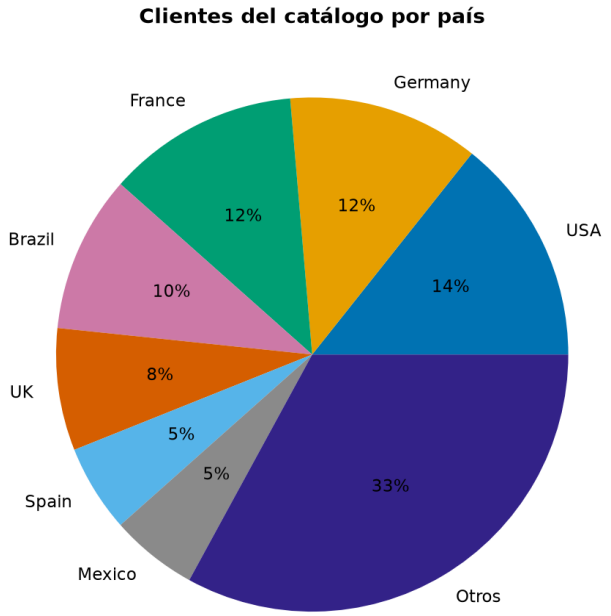



***Figura 17.*** *Distribución de países de clientes (N07). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Gráfico circular: Distribución de países de clientes».

**Por qué este gráfico.** El gráfico circular muestra cómo se reparten los 91 clientes entre países; los países menores se agrupan en «Otros».

**Cómo leerlo.** Cada porción es el porcentaje de clientes de un país; se cuentan clientes registrados, no ventas.

**Construcción.** Se cuenta cada CustomerID del catálogo una vez. Los siete países con más clientes se muestran por separado y los demás se agrupan en Otros.

**Resultado observado.** Hay 91 clientes de 21 países. USA tiene 13 clientes (14.3%).

**Insight y decisión que apoya.** Usar la distribución para contextualizar la cobertura comercial junto con importes de venta.

**Límite de interpretación.** Representa clientes registrados; no población, participación de mercado ni clientes activos. Otros combina países distintos.

**Consulta** (`sql/northwind.sql`):

```
-- N07: catálogo de clientes (incluye aquellos sin pedidos).
SELECT Country,COUNT(*) AS clientes FROM dbo.Customers GROUP BY Country ORDER BY clientes
    DESC;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
c = customers.groupby('Country').size()
```

## N08 · Box plot de precios por proveedor

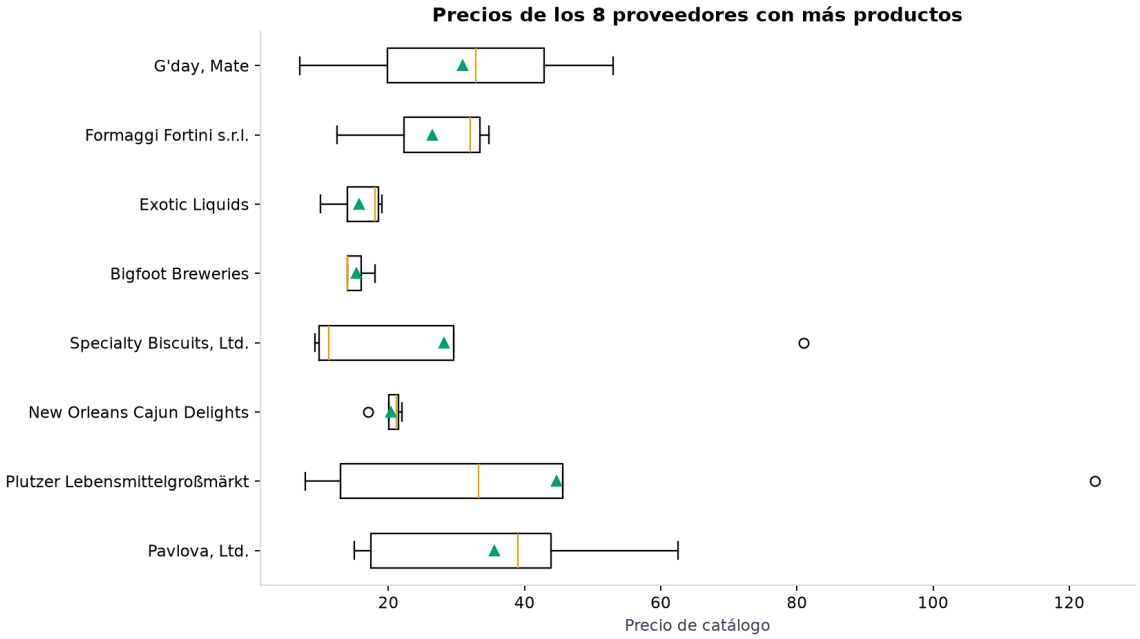



***Figura 18.*** *Box plot de precios por proveedor (N08). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Box plot: Análisis de precios por proveedor».

**Por qué este gráfico.** El diagrama de caja resume en una sola figura la distribución de precios de cada proveedor —centro, dispersión y valores extremos— y permite compararlos lado a lado.

**Cómo leerlo.** Cada fila es un proveedor (los ocho con más productos). La caja abarca el 50% central de sus precios; la línea naranja dentro de la caja es la mediana y el triángulo verde, el promedio. Los bigotes llegan hasta los precios más altos y más bajos que no son atípicos, y los círculos aislados son precios atípicos, como el producto de más de 120 de Plutzer. Se usa el precio de catálogo (tabla Products).

**Construcción.** Se usa Products.UnitPrice, distinto del precio histórico de cada venta. Se eligen ocho proveedores por cantidad de productos y se muestran mediana, cuartiles y puntos extremos.

**Resultado observado.** El catálogo contiene 77 productos y 29 proveedores; el precio mediano es 19.50.

**Insight y decisión que apoya.** Revisar proveedores con dispersión elevada para entender mezcla y posicionamiento del portafolio.

**Límite de interpretación.** Las muestras por proveedor son pequeñas y las categorías difieren. Un precio extremo no es automáticamente un error.

**Consulta** (`sql/northwind.sql`):

```
-- N08: precios del catálogo por proveedor.
SELECT s.CompanyName,p.ProductID,p.UnitPrice FROM dbo.Products p
JOIN dbo.Suppliers s ON s.SupplierID=p.SupplierID;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
sub = products.merge(suppliers, on='SupplierID')
px.box(sub, x='UnitPrice', y='CompanyName', points='all')
```

## N09 · Áreas: ventas acumuladas por empleado

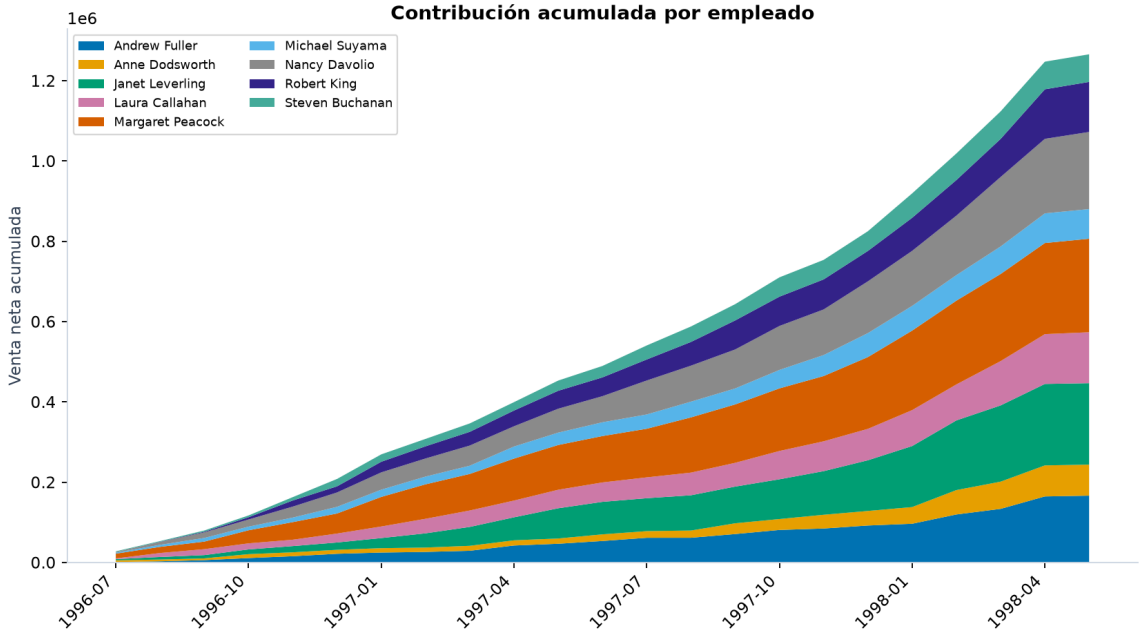



***Figura 19.*** *Áreas: ventas acumuladas por empleado (N09). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Gráfico de áreas: Acumulado de ventas por empleado».

**Por qué este gráfico.** El área apilada muestra a la vez el total acumulado (la altura de la figura) y cuánto aporta cada empleado (el grosor de su banda) a lo largo del tiempo.

**Cómo leerlo.** El eje horizontal son los meses y el vertical la venta neta acumulada desde julio de 1996, en millones (la marca «1e6» significa 1,000,000). Cada color es un empleado y el grosor de su banda en un mes es lo que lleva vendido hasta ese momento. Al final, la altura total es 1.27 millones, la venta neta de todo el período, y la banda más ancha es la de Margaret Peacock (18.4%).

**Construcción.** Se suman las ventas por empleado y mes, se completan celdas ausentes con cero y se aplica acumulado temporal. Las áreas se apilan; el valor individual es el espesor de cada banda.

**Resultado observado.** Margaret Peacock registra 232,890.85, equivalente al 18.4% del importe neto.

**Insight y decisión que apoya.** Combinar la contribución con número de pedidos, cartera asignada y metas para evaluar desempeño.

**Límite de interpretación.** La posición superior de una banda no representa su valor individual. Las ventas no miden productividad ajustada por oportunidades.

**Consulta** (`sql/northwind.sql`):

```
-- N09: acumulado individual, apilar en Python.
WITH mensual AS (
 SELECT o.EmployeeID,DATEFROMPARTS(YEAR(o.OrderDate),MONTH(o.OrderDate),1) AS mes,
 SUM(d.UnitPrice*d.Quantity*(1-d.Discount)) AS neto
 FROM dbo.Orders o JOIN dbo.[Order Details] d ON d.OrderID=o.OrderID
 GROUP BY o.EmployeeID,YEAR(o.OrderDate),MONTH(o.OrderDate))
SELECT EmployeeID,mes,neto,SUM(neto) OVER(PARTITION BY EmployeeID ORDER BY mes ROWS UNBOUNDED
    PRECEDING) AS acumulado
FROM mensual ORDER BY EmployeeID,mes;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
emp = d.pivot_table(index='month', columns='employee',
 values='net', aggfunc='sum', fill_value=0).cumsum()
```

## N10 · Heatmap temporal de pedidos

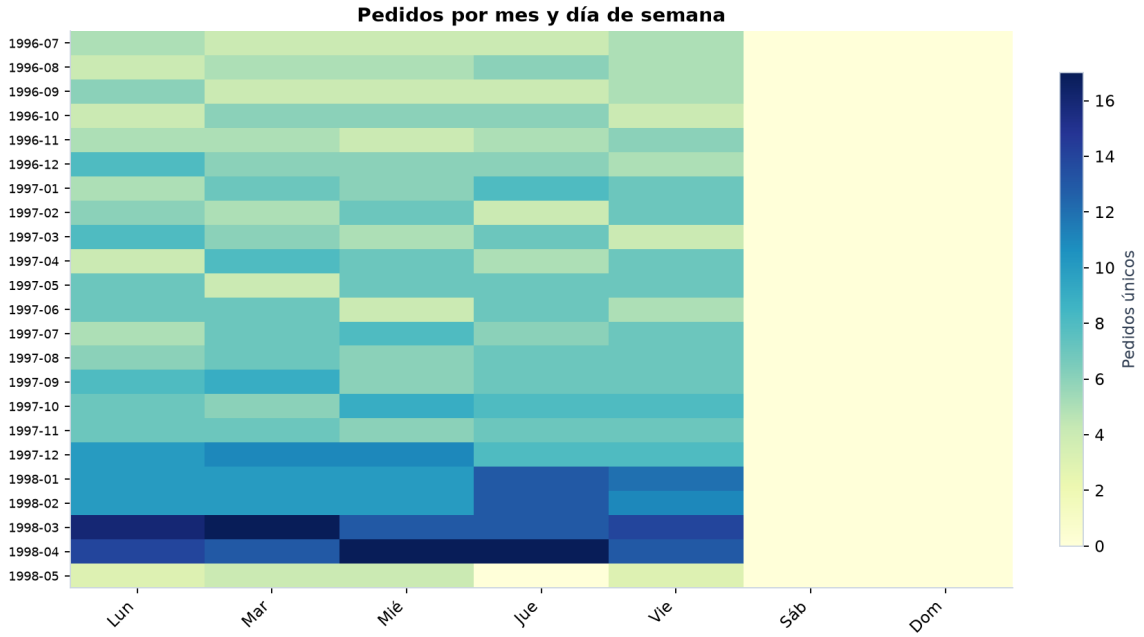



***Figura 20.*** *Heatmap temporal de pedidos (N10). Fuente: SQL Server / Northwind.*

**Requisito de la consigna.** «Heatmap: Patrones temporales de pedidos».

**Por qué este gráfico.** Un mapa de calor de mes por día de la semana revela patrones de calendario que una línea no muestra, como días sin actividad o meses de mayor carga.

**Cómo leerlo.** Cada fila es un mes y cada columna un día de la semana; el color indica cuántos pedidos se registraron (más oscuro, más pedidos). Sábado y domingo están vacíos porque Northwind no registra pedidos en fin de semana. Los meses de 1998 son los más oscuros, y el martes (Mar) acumula más pedidos en total (168).

**Construcción.** Se cuenta Orders.OrderID en la tabla de pedidos, una sola vez, por mes y día de semana. Se incluyen fines de semana con cero.

**Resultado observado.** Hay 830 pedidos únicos. Mar acumula 168 pedidos, el mayor conteo por día de semana.

**Insight y decisión que apoya.** Planificar personal para los días con más actividad de registro, contrastando pedidos y tiempo de procesamiento.

**Límite de interpretación.** El conteo agregado no ajusta por cuántas veces aparece cada día en cada mes. No representa hora de compra ni fecha de entrega.

**Consulta** (`sql/northwind.sql`):

```
-- N10: no unir líneas; lunes=0 independientemente de SET DATEFIRST.
SELECT YEAR(OrderDate) AS anio,MONTH(OrderDate) AS mes,
((DATEDIFF(DAY,CONVERT(date,'19000101',112),OrderDate)%7)+7)%7 AS dia_semana,
COUNT(*) AS pedidos FROM dbo.Orders GROUP BY YEAR(OrderDate),MONTH(OrderDate),
((DATEDIFF(DAY,CONVERT(date,'19000101',112),OrderDate)%7)+7)%7 ORDER BY anio,mes,dia_semana;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
orders.groupby([orders.OrderDate.dt.to_period('M'),
                orders.OrderDate.dt.dayofweek]).size()
```

## 4.5 Insights obtenidos de Northwind

- **Crecimiento y estacionalidad.** Las ventas crecen durante 1997 y alcanzan su máximo en abril de 1998 (123,798.68); 1998-Q1 es el trimestre más alto (298,491.55). Para presupuestar conviene comparar sólo trimestres completos.
- **Concentración de clientes.** Los diez principales clientes generan el 45.0% de la venta neta; QUICK-Stop aporta 110,277.30. Es un riesgo de dependencia que justifica un seguimiento de cuentas clave.
- **Mezcla de producto.** Beverages es la categoría líder (21.2% del total) y Côte de Blaye el producto de mayor ingreso (141,396.73). Sin costos no se puede afirmar que sean los más rentables.
- **Descuentos.** La correlación entre descuento y cantidad es débil (r = 0.129): los descuentos no se asocian claramente con pedidos más grandes y requieren un análisis por producto y cliente.
- **Operación.** Margaret Peacock concentra el 18.4% de las ventas, y el martes registra más pedidos (168). Ambos datos ayudan a planificar personal y metas, combinados con la cartera asignada.

# 5. Oracle y recursos humanos

## 5.1 Descripción de la base de datos

El esquema HR de Oracle modela una organización. El análisis usa la cadena employees → departments → locations → countries → regions, junto con employees → jobs; job_history guarda los puestos y departamentos anteriores de algunos empleados [2]. Se utilizó la versión oficial del repositorio oracle-samples, que contiene cinco regiones.

En términos simples: cada **empleado** tiene un salario, una fecha de ingreso, un **puesto** (job) y un **departamento**; cada departamento está en una **ubicación** (ciudad), que pertenece a un **país**, que a su vez pertenece a una **región**. El campo manager_id indica quién es el supervisor de cada empleado, lo que permite reconstruir el organigrama. La tabla **jobs** define para cada puesto un salario mínimo y uno máximo (bandas salariales).

| Tabla | Filas | Uso en el análisis |
|---|---|---|
| employees | 107 | Salario, fecha de ingreso, puesto y supervisor |
| departments / jobs | 27 / 19 | Asignación organizacional y bandas salariales |
| locations / countries / regions | 23 / 25 / 5 | Ubicación geográfica de los departamentos |
| job_history | 10 | Cambios de puesto de 7 empleados |

HR no contiene edad, género, competencias, evaluaciones de desempeño, bajas ni salarios históricos. Por eso la "distribución demográfica" se aborda con la ubicación de los departamentos, el radar compara métricas estructurales de los puestos y el waterfall se presenta como escenario de bandas salariales, no como una carrera real.

## 5.2 Instalación y carga

El primer arranque de Oracle descarga la imagen Lite del registro oficial de Oracle [9] y crea la base de datos; el *healthcheck* comprueba que la PDB `FREEPDB1` responda. `lab.py cargar oracle` crea el usuario HR y ejecuta como HR los scripts oficiales `hr_create.sql`, `hr_populate.sql` y `hr_code.sql`.




***Figura 21.*** *Descarga de la imagen de Oracle Database Free Lite y contenedor healthy.*

**Incidencia y solución.** La primera carga falló con `ORA-00959: tablespace 'USERS' does not exist`: la variante Lite de la imagen no incluye ese tablespace, que el esquema HR usa por defecto. El error ocurrió antes de crear el usuario, por lo que no quedaron objetos a medias. Se modificó el cargador para crear el tablespace cuando falta y se repitió la carga:

```
CREATE TABLESPACE users
  DATAFILE '/opt/oracle/oradata/FREE/FREEPDB1/users01.dbf'
  SIZE 100M AUTOEXTEND ON NEXT 50M MAXSIZE 2G;
```

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py cargar oracle
La operación Docker falló. CREATE USER hr IDENTIFIED BY "[oculto]" DEFAULT TABLESPACE users QUOTA UNLIMITED ON users
*
ERROR at line 1:
ORA-00959: tablespace 'USERS' does not exist
Help: https://docs.oracle.com/error-help/db/ora-00959/
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py cargar oracle
HR ya contiene las siete tablas con los conteos esperados; se conserva.
```

***Figura 22.*** *Primer intento con ORA-00959 y carga posterior: HR contiene las siete tablas con los conteos esperados.*

## 5.3 Conexión ODBC y evidencia

La conexión usa **pyodbc** con el driver **Oracle Instant Client ODBC** y el descriptor `oracle:1521/FREEPDB1` (nombre de servicio, no SID). La prueba consulta el esquema actual y `COUNT(*)` de `employees`: devolvió el esquema HR con 107 empleados. Última verificación registrada: 2026-10-12 00:41:35 UTC.

**Incidencia y solución.** La primera verificación devolvió `ORA-01017: invalid credential`. La contraseña era correcta, pero el cliente la enviaba entre llaves (`PWD={…}`), un formato que acepta el driver de Microsoft pero que el driver ODBC de Oracle interpreta como parte de la contraseña. Se envió la contraseña sin llaves, lo cual es seguro porque el generador sólo usa letras, números, guion y guion bajo.

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py verificar oracle
VERIFICACIÓN REAL DE CONEXIONES – 2026-10-11T23:34:07.506560+00:00
oracle: CONEXIÓN VERIFICADA
{"motor": "oracle", "connector": "ODBC", "timestamp_utc": "2026-10-11T23:34:07.335537+00:00", "verified": true, "schema": "HR", "employees": 107}
 Container bigdata-final-analisis-run-472f68add770 Creating
 Container bigdata-final-analisis-run-472f68add770 Created
```

***Figura 23.*** *Conexión exitosa a Oracle mediante Oracle Instant Client ODBC (CONEXIÓN VERIFICADA, esquema HR, 107 empleados).*

## 5.4 Visualizaciones

Las diez figuras siguientes se calculan sobre las siete tablas de HR extraídas de Oracle. Las consultas están en `sql/hr.sql`. El salario se expresa en la unidad de la muestra (mensual, sin moneda documentada) y la antigüedad se mide a la fecha fija del 08/10/2026.

## H01 · Histograma salarial por departamento

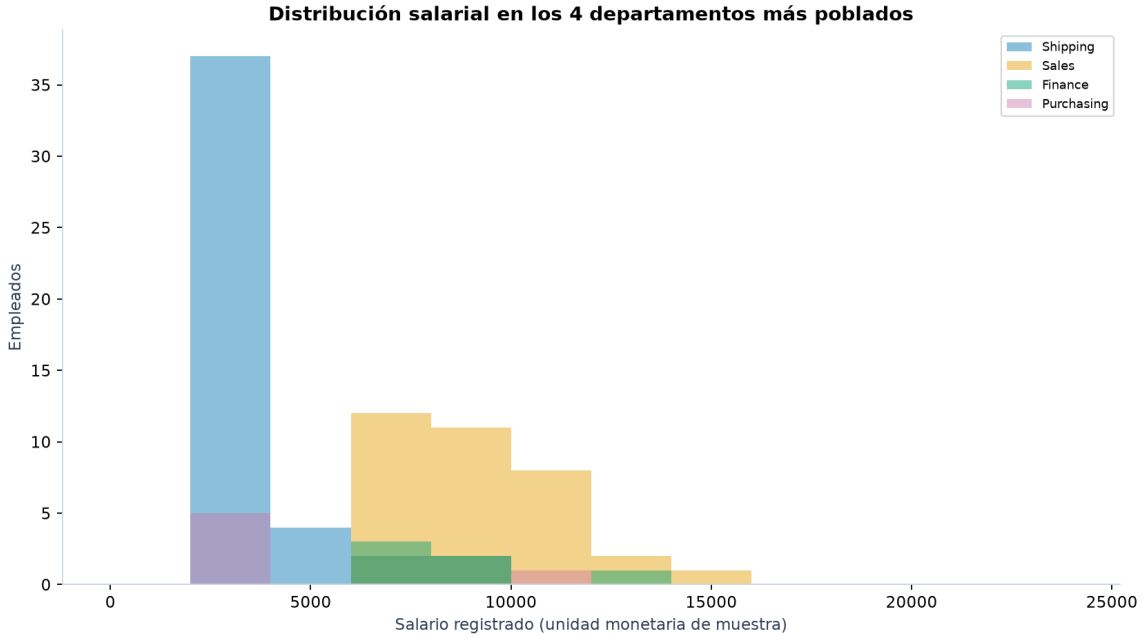



***Figura 24.*** *Histograma salarial por departamento (H01). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Histograma: Distribución de salarios por departamento».

**Por qué este gráfico.** Un histograma muestra cómo se reparten los salarios: en qué rangos se concentran los empleados y qué tan dispersos están. Superponer departamentos permite compararlos.

**Cómo leerlo.** El eje horizontal es el salario mensual agrupado en rangos y el vertical, cuántos empleados caen en cada rango. Cada color es uno de los cuatro departamentos con más empleados. Shipping (azul) se concentra en los salarios más bajos, mientras que Sales (amarillo) ocupa rangos más altos. En el dashboard se pueden mostrar todos los departamentos desde la leyenda o el filtro.

**Construcción.** La versión estática superpone cuatro departamentos por tamaño. La versión interactiva permite mostrar u ocultar todos los departamentos desde la leyenda.

**Resultado observado.** Los 107 empleados tienen salario mediano 6,200.00, promedio 6,461.83 y máximo 24,000.00.

**Insight y decisión que apoya.** Revisar bandas de compensación por puesto y departamento antes de explicar diferencias.

**Límite de interpretación.** HR es una muestra académica y documenta salarios mensuales. La moneda no se atribuye; la dispersión no permite inferir discriminación o desempeño.

**Consulta** (`sql/hr.sql`):

```
-- H01: salario por departamento; NULL se conserva como Sin departamento.
SELECT e.employee_id,NVL(d.department_name,'Sin departamento') AS departamento,e.salary
FROM employees e LEFT JOIN departments d ON d.department_id=e.department_id;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
px.histogram(h, x='salary', color='department_name',
             nbins=12, barmode='overlay')
```

## H02 · Radar de perfiles de puesto

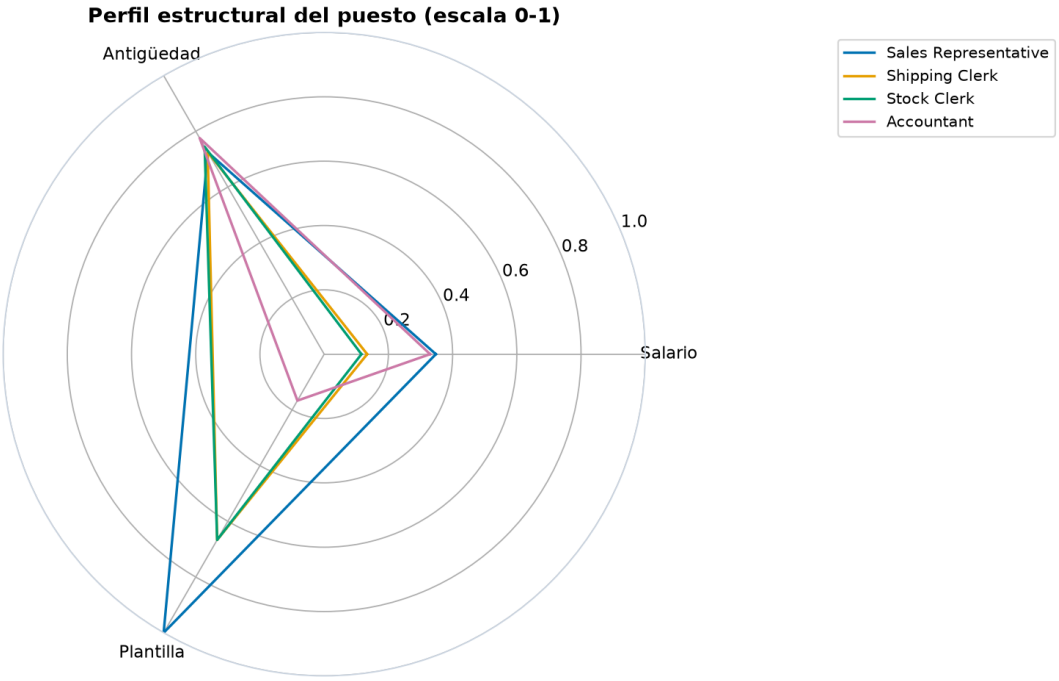



***Figura 25.*** *Radar de perfiles de puesto (H02). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Gráfico de radar: Competencias por tipo de trabajo».

**Por qué este gráfico.** El radar compara varios perfiles en varias dimensiones a la vez. Como HR no contiene competencias, se usan tres dimensiones que sí existen en los datos: salario medio, antigüedad media y plantilla (número de empleados) de cada puesto.

**Cómo leerlo.** Cada eje es una dimensión y cada línea de color, un puesto (los cuatro con más empleados). Los valores se escalan de 0 a 1 dividiendo por el máximo de cada dimensión, para comparar dimensiones con unidades distintas. Sales Representative destaca en plantilla (30 empleados, el máximo), y las antigüedades de los cuatro puestos son parecidas.

**Construcción.** Se comparan cuatro puestos por tamaño. Salario medio, antigüedad media y plantilla se dividen por el máximo de cada variable en todos los puestos.

**Resultado observado.** Sales Representative es el puesto de mayor dotación entre los perfiles mostrados: 30 empleados.

**Insight y decisión que apoya.** Comparar dimensiones organizacionales y utilizar hover para inspeccionar perfiles relativos.

**Límite de interpretación.** No existen evaluaciones de competencias en HR. El radar es un perfil estructural y no mide habilidad, calidad de trabajo ni puntuación de desempeño.

**Consulta** (`sql/hr.sql`):

```
-- H02: perfil estructural. No hay competencias en HR.
SELECT j.job_title,AVG(e.salary) AS salario_medio,
AVG((DATE '2026-10-08'-e.hire_date)/365.25) AS antiguedad_media,COUNT(*) AS plantilla
FROM employees e JOIN jobs j ON j.job_id=e.job_id GROUP BY j.job_title;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
metrics = h.groupby('job_title').agg(
 Salario=('salary','mean'), Antigüedad=('tenure','mean'),
 Plantilla=('employee_id','size'))
normalized = metrics / metrics.max()
```

## H03 · Organigrama interactivo

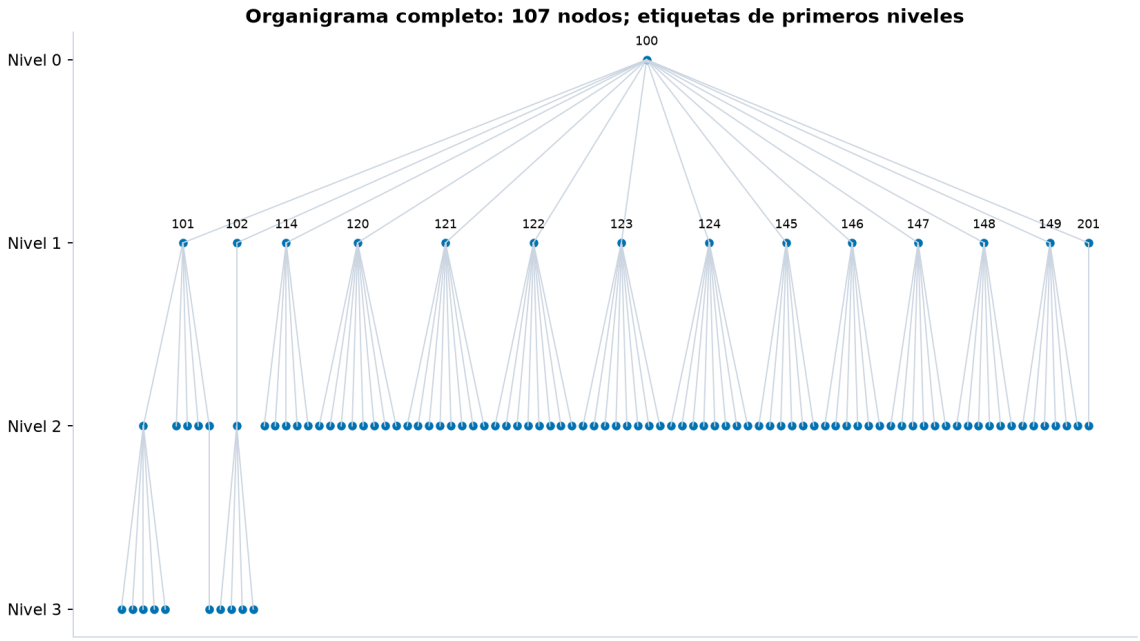



***Figura 26.*** *Organigrama interactivo (H03). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Organigrama interactivo de la empresa».

**Por qué este gráfico.** Un árbol jerárquico es la representación directa de quién reporta a quién: muestra la profundidad de la organización y cuántas personas supervisa cada gerente.

**Cómo leerlo.** Cada punto es un empleado, identificado por su employee_id, y cada línea lo une con su supervisor. El nivel 0 es el presidente (100), el nivel 1 sus reportes directos y así sucesivamente hasta el nivel 3. En el dashboard, al pasar el cursor por un punto se ven el nombre y el puesto del empleado.

**Construcción.** Se construye un árbol a partir de employee_id y manager_id. Se dibujan todos los empleados; el dashboard muestra nombre y puesto al pasar el cursor.

**Resultado observado.** La estructura contiene 107 nodos, 106 relaciones y 1 raíz. La profundidad máxima es 3 enlaces.

**Insight y decisión que apoya.** Inspeccionar amplitud de supervisión y cadenas de reporte antes de proponer cambios de estructura.

**Límite de interpretación.** Una relación jerárquica no indica carga de trabajo ni autoridad informal. La coordenada horizontal sólo separa ramas del árbol.

**Consulta** (`sql/hr.sql`):

```
-- H03: organigrama, una relación por empleado.
SELECT employee_id,manager_id,first_name,last_name,job_id FROM employees ORDER BY employee_id;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
edges = h[['employee_id','manager_id']].dropna()
# Posiciones calculadas por recorrido del árbol; hover por empleado.
```

## H04 · Mapa de calor de dotación por ubicación

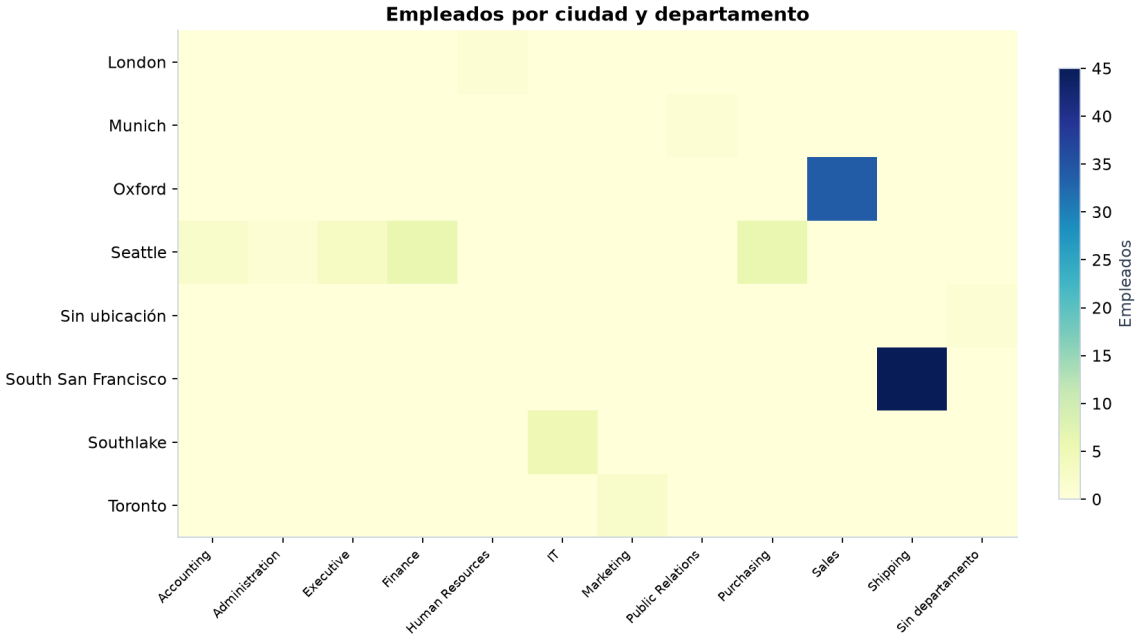



***Figura 27.*** *Mapa de calor de dotación por ubicación (H04). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Mapa de calor: Densidad de empleados por ubicación».

**Por qué este gráfico.** El mapa de calor cruza dos categorías (ciudad y departamento) y usa el color para la cantidad de empleados, de modo que las concentraciones saltan a la vista.

**Cómo leerlo.** Cada fila es la ciudad donde está el departamento y cada columna, un departamento; el color indica cuántos empleados hay en esa combinación. Las celdas más oscuras son Shipping en South San Francisco (45) y Sales en Oxford (34). «Densidad» significa aquí número de empleados por ubicación, no por superficie.

**Construcción.** Se une employees con departments, locations, countries y regions mediante uniones izquierdas. Los empleados sin departamento conservan una categoría explícita.

**Resultado observado.** South San Francisco concentra 45 empleados. Hay 1 empleado sin ubicación derivable.

**Insight y decisión que apoya.** Identificar ubicaciones con concentración de personal y revisar registros sin asignación organizacional.

**Límite de interpretación.** La densidad aquí es un conteo, no empleados por superficie. La ubicación corresponde al departamento y no al domicilio personal.

**Consulta** (`sql/hr.sql`):

```
-- H04: dotación por ciudad y departamento.
SELECT NVL(l.city,'Sin ubicación') AS ciudad,NVL(d.department_name,'Sin departamento') AS
    departamento,
COUNT(*) AS empleados FROM employees e LEFT JOIN departments d ON
    d.department_id=e.department_id
LEFT JOIN locations l ON l.location_id=d.location_id GROUP BY l.city,d.department_name;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
h.groupby(['city','department_name']).size().unstack(fill_value=0)
```

## H05 · Violín salarial por antigüedad

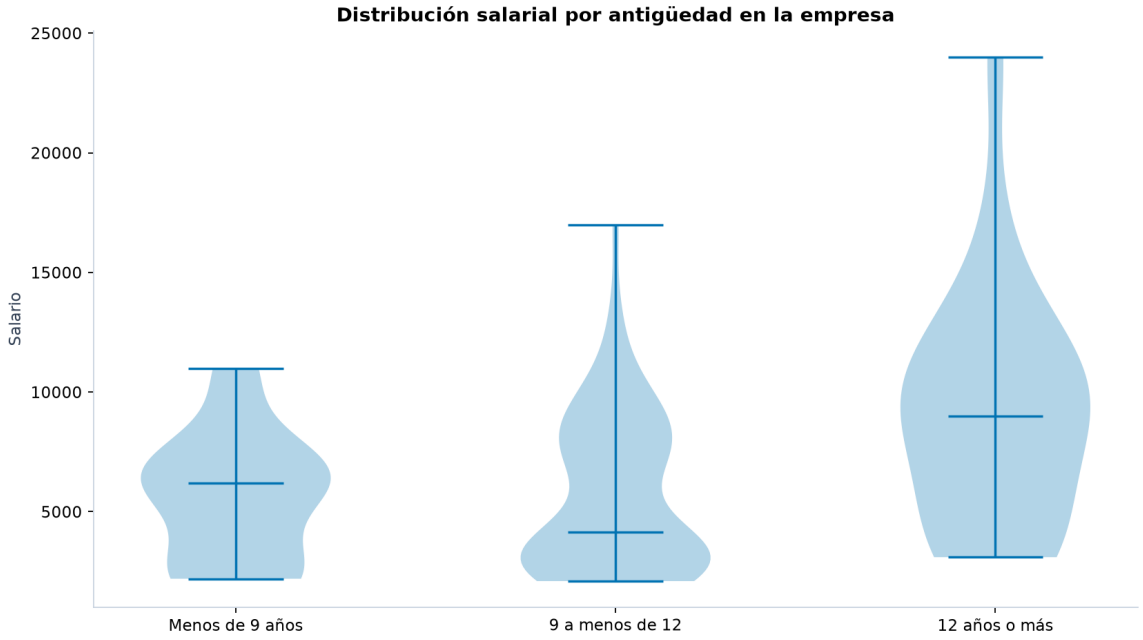



***Figura 28.*** *Violín salarial por antigüedad (H05). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Gráfico de violín: Análisis salarial por nivel de experiencia».

**Por qué este gráfico.** El violín muestra la forma completa de la distribución de salarios de cada grupo, no sólo su promedio, y permite comparar grupos lado a lado. Como HR no registra la experiencia laboral total, se usa la antigüedad en la empresa.

**Cómo leerlo.** Cada violín es un grupo de antigüedad: menos de 9 años, de 9 a menos de 12, y 12 años o más. El ancho del violín a cada altura indica cuántos empleados tienen ese salario; la línea horizontal del

centro es la mediana y los extremos marcan el mínimo y el máximo. El grupo de 12 años o más tiene salarios más altos y más dispersos.

**Construcción.** La antigüedad se calcula entre hire_date y 08/10/2026, en días / 365.25. Se crean tres bandas y se muestra una densidad estimada con mediana.

**Resultado observado.** La antigüedad mediana es 10.76 años. Las bandas contienen Menos de 9 años: 17, 9 a menos de 12: 66, 12 años o más: 24.

**Insight y decisión que apoya.** Comparar salarios dentro del mismo puesto antes de interpretar el efecto de la antigüedad.

**Límite de interpretación.** La antigüedad en la empresa no equivale a experiencia profesional total. La densidad depende de la muestra y del suavizado.

**Consulta** (`sql/hr.sql`):

```
-- H05: salario por antigüedad, proxy de permanencia en la empresa.
SELECT employee_id,salary,(DATE '2026-10-08'-hire_date)/365.25 AS antiguedad FROM employees;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
h['tenure'] = (pd.Timestamp('2026-10-08') - h.hire_date).dt.days / 365.25
px.violin(h, x='level', y='salary', box=True)
```

## H06 · Sankey de cambios de departamento

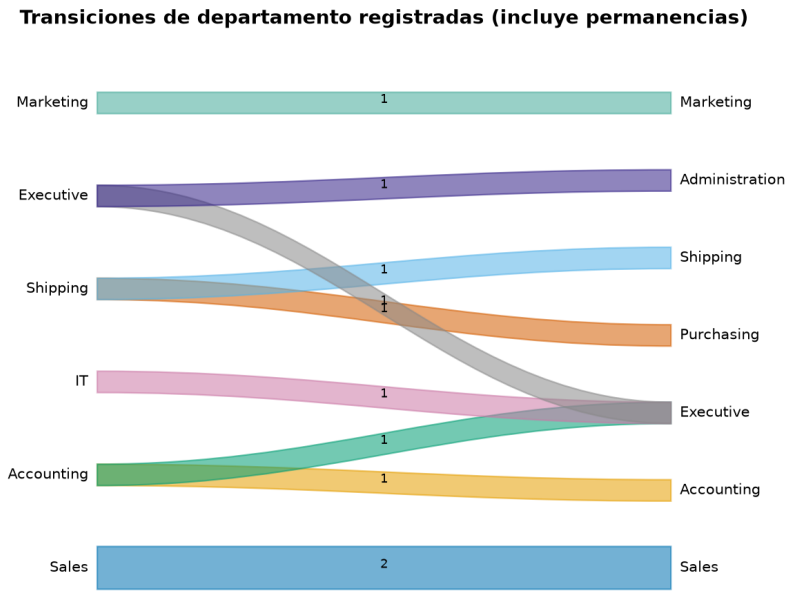



***Figura 29.*** *Sankey de cambios de departamento (H06). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Sankey diagram: Flujo de cambios de departamento».

**Por qué este gráfico.** Un diagrama de Sankey representa flujos entre estados: cada banda conecta un departamento de origen con uno de destino y su grosor indica cuántas transiciones hubo.

**Cómo leerlo.** A la izquierda está el departamento anterior y a la derecha, el siguiente (o el actual). Una banda que une el mismo departamento a ambos lados es un cambio de puesto dentro del departamento, como las dos transiciones de Sales. Las bandas que cruzan, como Executive → Administration o Shipping → Purchasing, son cambios de departamento.

**Construcción.** Por empleado se ordena job_history por start_date. Cada departamento se conecta con el siguiente registro y, al final, con el departamento actual. Los lados separan estado anterior y posterior.

**Resultado observado.** Se reconstruyen 10 transiciones: 4 cambian de departamento y 6 permanecen en el mismo. El histórico contiene 7 empleados.

**Insight y decisión que apoya.** Usar el flujo para formular preguntas sobre movilidad interna y cambios de puesto.

**Límite de interpretación.** No existen bajas ni una población histórica completa. No se estima rotación de salida; un enlace al estado actual no documenta su fecha exacta.

**Consulta** (`sql/hr.sql`):

```
-- H06: histórico y asignación actual; reconstruir sucesiones por employee_id en Python.
SELECT employee_id,start_date,end_date,job_id,department_id FROM job_history ORDER BY
    employee_id,start_date;
SELECT employee_id,department_id FROM employees;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
seq = historial.sort_values('start_date').department_id.tolist()
seq += [departamento_actual]
transiciones = list(zip(seq, seq[1:]))
```

## H07 · Burbujas: salario y antigüedad

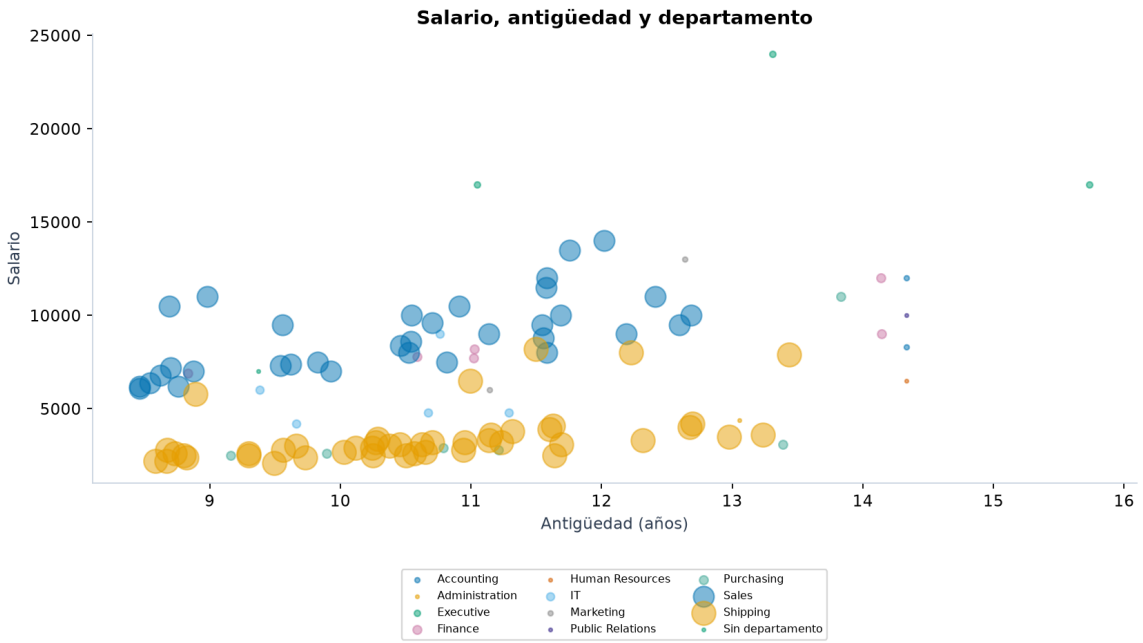



***Figura 30.*** *Burbujas: salario y antigüedad (H07). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Bubble chart: Salario vs. Antigüedad vs. Departamento».

**Por qué este gráfico.** El gráfico de burbujas representa más de dos variables en un plano: posición horizontal, posición vertical, color y tamaño.

**Cómo leerlo.** Cada burbuja es un empleado. El eje horizontal es la antigüedad en años, el vertical el salario mensual, el color el departamento y el tamaño de la burbuja, cuántos empleados tiene ese departamento. Las burbujas grandes amarillas (Shipping) se agrupan abajo y las azules (Sales) más arriba; en general, el salario sube con la antigüedad (r = 0.415).

**Construcción.** Cada punto es un empleado. El área de burbuja representa tamaño del departamento; el color representa departamento. Las coordenadas son antigüedad y salario.

**Resultado observado.** La correlación de Pearson entre antigüedad y salario es 0.415.

**Insight y decisión que apoya.** Examinar casos de compensación alta o baja dentro de grupos de puestos comparables.

**Límite de interpretación.** El área repite el tamaño del departamento para cada empleado y no debe sumarse. La correlación está afectada por puesto, jerarquía y composición.

**Consulta** (`sql/hr.sql`):

```
-- H07: salario, antigüedad y tamaño departamental.
SELECT employee_id,salary,(DATE '2026-10-08'-hire_date)/365.25 AS antiguedad,
department_id,COUNT(*) OVER(PARTITION BY department_id) AS plantilla_departamento FROM
    employees;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
h['department_size'] = h.department_name.map(h.department_name.value_counts())
px.scatter(h, x='tenure', y='salary', size='department_size',
           color='department_name')
```

## H08 · Densidad de contrataciones por mes

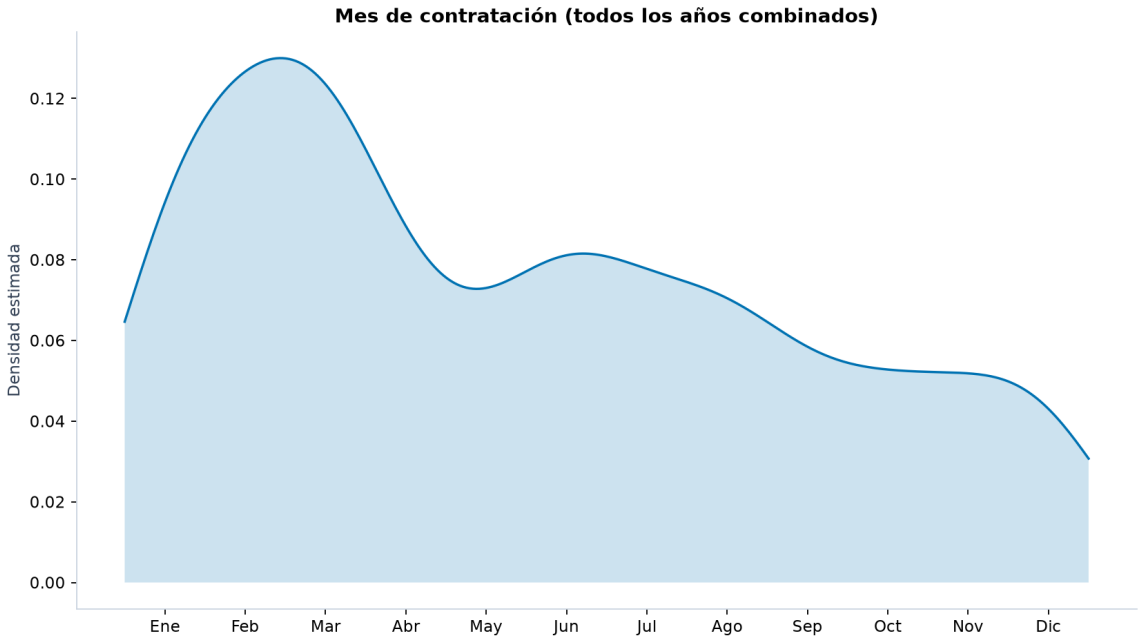



***Figura 31.*** *Densidad de contrataciones por mes (H08). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Gráfico de densidad: Concentración de contrataciones por mes».

**Por qué este gráfico.** Una curva de densidad suaviza el conteo de contrataciones por mes y muestra en qué época del año se concentran, sin depender de un año en particular.

**Cómo leerlo.** El eje horizontal son los meses del año (combinando todos los años) y la altura de la curva, la concentración relativa de contrataciones. El pico está entre febrero y marzo; marzo es el mes con más contrataciones (17). La curva no conecta diciembre con enero, aunque sean meses consecutivos.

**Construcción.** Se combinan meses de contratación de todos los años. La curva aplica KDE con ancho relativo 0.25; es una aproximación suavizada de una variable discreta.

**Resultado observado.** El mes con más contrataciones observadas es 3, con 17 empleados del catálogo actual.

**Insight y decisión que apoya.** Examinar el conteo por año y mes antes de atribuir estacionalidad al proceso de contratación.

**Límite de interpretación.** Enero y diciembre son adyacentes en un calendario, pero esta KDE no es circular. Sólo incluye empleados actuales y no permite estimar tasa histórica de contratación.

**Consulta** (`sql/hr.sql`):

```
-- H08: meses de contratación; KDE se calcula en Python.
SELECT EXTRACT(MONTH FROM hire_date) AS mes,COUNT(*) AS contrataciones
FROM employees GROUP BY EXTRACT(MONTH FROM hire_date) ORDER BY mes;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
months = h.hire_date.dt.month
kde = gaussian_kde(months, bw_method=0.25)
```

## H09 · Donut de empleados por región

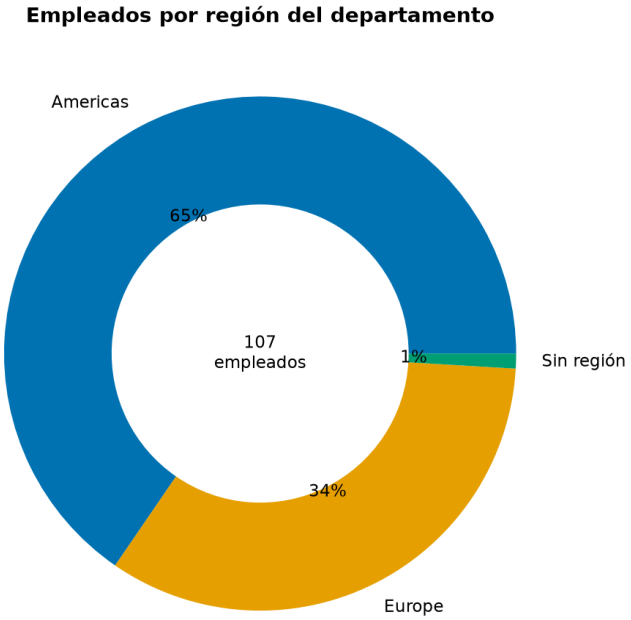



***Figura 32.*** *Donut de empleados por región (H09). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Donut chart: Distribución de empleados por región».

**Por qué este gráfico.** El donut muestra cómo se reparte el total de empleados entre pocas regiones y deja el centro para el total.

**Cómo leerlo.** La región corresponde a la ubicación del departamento, no a la nacionalidad del empleado; «Sin región» es el empleado sin departamento.

**Construcción.** Cada empleado se asigna a la región de su departamento mediante la cadena de ubicaciones. Se conserva Sin región para el registro sin departamento.

**Resultado observado.** Americas concentra 70 de 107 empleados (65.4%).

**Insight y decisión que apoya.** Evaluar concentración territorial del catálogo organizacional y calidad de asignaciones.

**Límite de interpretación.** La región es una categoría geográfica corporativa de esta versión de HR. No aporta nacionalidad, edad, sexo ni otras variables demográficas personales.

**Consulta** (`sql/hr.sql`):

```
-- H09: región de departamento, no nacionalidad personal.
SELECT NVL(r.region_name,'Sin región') AS region,COUNT(*) AS empleados
FROM employees e LEFT JOIN departments d ON d.department_id=e.department_id
LEFT JOIN locations l ON l.location_id=d.location_id LEFT JOIN countries c ON
    c.country_id=l.country_id
LEFT JOIN regions r ON r.region_id=c.region_id GROUP BY r.region_name;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
h.region_name.fillna('Sin región').value_counts()
```

## H10 · Waterfall de bandas salariales: escenario ilustrativo

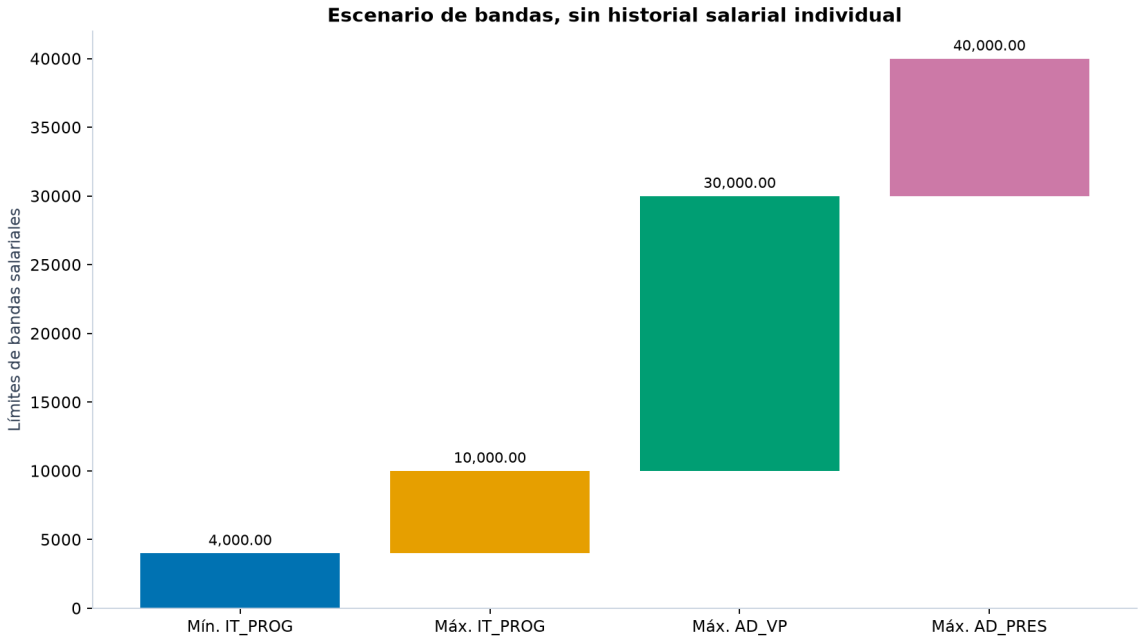



***Figura 33.*** *Waterfall de bandas salariales: escenario ilustrativo (H10). Fuente: Oracle / HR.*

**Requisito de la consigna.** «Waterfall chart: Progresión salarial en carrera».

**Por qué este gráfico.** Un waterfall muestra cómo un valor avanza por etapas sucesivas. HR no guarda el historial de salarios de cada empleado, así que se construye un escenario ilustrativo con las bandas salariales oficiales de tres puestos.

**Cómo leerlo.** Cada barra es un escalón de la escala salarial: el mínimo de programador (IT_PROG, 4,000), el máximo de programador (10,000), el máximo de vicepresidente (AD_VP, 30,000) y el máximo de presidente (AD_PRES, 40,000). La altura de cada barra es el aumento respecto al escalón anterior. Es un escenario de bandas, no la carrera real de una persona.

**Construcción.** Se encadenan el mínimo y máximo de IT_PROG y los máximos de AD_VP y AD_PRES. Cada escalón representa diferencia entre límites de bandas definidos en jobs.

**Resultado observado.** El escenario va de 4,000.00 a 40,000.00; la diferencia es 36,000.00.

**Insight y decisión que apoya.** Ilustrar cómo se comparan bandas de puestos; reemplazarlo por un historial real si se desea analizar progresión de una carrera.

**Límite de interpretación.** No representa ascensos observados, salarios pasados ni trayectoria de un empleado. La secuencia de puestos es hipotética y las bandas son datos oficiales.

**Consulta** (`sql/hr.sql`):

```
-- H10: escenario de bandas de puestos. No hay sueldos históricos individuales.
SELECT job_id,job_title,min_salary,max_salary FROM jobs
WHERE job_id IN ('IT_PROG','AD_VP','AD_PRES') ORDER BY max_salary;
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
jobs = read('hr_jobs').set_index('job_id')
limits = [jobs.loc['IT_PROG','min_salary'], jobs.loc['IT_PROG','max_salary'],
          jobs.loc['AD_VP','max_salary'], jobs.loc['AD_PRES','max_salary']]
```

## 5.5 Insights obtenidos de HR

- **Compensación.** El salario mediano es 6,200.00 y el promedio 6,461.83, con un máximo de 24,000.00: la distribución tiene cola hacia salarios altos concentrados en puestos directivos.
- **Antigüedad y salario.** La correlación es moderada (r = 0.415). La antigüedad mediana es 10.76 años; la mayoría de la plantilla (66 empleados) tiene entre 9 y 12 años.
- **Estructura.** El organigrama tiene una sola raíz y tres niveles por debajo del presidente para 107 empleados: una estructura plana con amplitudes de supervisión grandes en algunos gerentes.
- **Ubicación.** South San Francisco concentra 45 empleados y Americas el 65.4% de la plantilla; un empleado no tiene departamento ni ubicación, lo que señala un problema de calidad de datos.
- **Movilidad.** Sólo 7 empleados tienen historial (10 transiciones, 4 con cambio de departamento): no alcanza para estimar rotación, pero sí para formular preguntas sobre movilidad interna.

# 6. MongoDB y sensores

## 6.1 Descripción de la base de datos

La colección `sensores_datos` de la base `bigdata_final` guarda una lectura por documento: timestamp (BSON Date), city, station, sensor_id, type, value, unit, latitude, longitude e injected_anomaly. Los datos son una **simulación propia** y reproducible (semilla 5122026) con ciclos diarios, ruido y 103 anomalías inyectadas; la etiqueta injected_anomaly sólo se usa para evaluar el detector, no para detectar.

Este es un documento real de la colección, la primera lectura del sensor de temperatura de la estación SDQ-1 en Santo Domingo:

```
{
  "timestamp": { "$date": "2026-09-01T00:00:00-04:00" },
  "city": "Santo Domingo",  "station": "SDQ-1",
  "sensor_id": "SDQ-1-temperatura",  "type": "temperatura",
  "value": 24.345,  "unit": "°C",
  "latitude": 18.4861,  "longitude": -69.9312,
  "injected_anomaly": false
}
```

Cada estación tiene tres sensores (temperatura, humedad y CO2), y el identificador del sensor combina estación y tipo. El modelo documental encaja con este tipo de datos: cada lectura es un evento independiente que trae consigo todo su contexto (ubicación, unidad y coordenadas), sin necesidad de unir tablas.

| Propiedad | Configuración |
|---|---|
| Ubicaciones | Santo Domingo, Santiago y Punta Cana; dos estaciones por ciudad |
| Magnitudes | Temperatura (°C), humedad (%) y CO2 (ppm) |
| Frecuencia y período | Una lectura por hora y sensor, septiembre de 2026 |
| Tamaño | 18 sensores × 720 horas = 12,960 documentos |
| Índices | Único (sensor_id, timestamp); compuesto (type, timestamp, city) |

## 6.2 Instalación y carga

`lab.py iniciar mongodb` crea una instancia con autenticación. `lab.py cargar mongodb` aplica un validador de esquema y los índices, e importa `sensores.jsonl` con `mongoimport` en modo *upsert* por (sensor_id, timestamp), de modo que repetir la carga no duplica lecturas. Las fechas se importan como BSON Date mediante Extended JSON [6].

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py iniciar mongodb
 Volume bigdata-final_mongodb-data Creating
 Volume bigdata-final_mongodb-data Creating
 Volume bigdata-final_mongodb-data Created
 Volume bigdata-final_mongodb-data Created
 Container bigdata-mongodb Creating
 Container bigdata-mongodb Created
 Container bigdata-mongodb Starting
 Container bigdata-mongodb Started
 Container bigdata-mongodb Waiting
 Container bigdata-mongodb Healthy
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py cargar mongodb
{
  database: 'bigdata_final',
  collection: 'sensores_datos',
  documents: 0
}
2026-10-11T23:09:18.376+0000    connected to: mongodb://localhost/
2026-10-11T23:09:18.805+0000    12960 document(s) imported successfully. 0 document(s) failed to import.
{
  database: 'bigdata_final',
  collection: 'sensores_datos',
  documents: 12960
}
Importación por clave completada: repetirla conserva una lectura por sensor y timestamp.
```

***Figura 34.*** *Creación del contenedor de MongoDB e importación de 12,960 documentos en sensores_datos.*

## 6.3 Conexión con PyMongo y evidencia

La conexión usa **PyMongo** con autenticación contra la base `admin` [5]. La prueba ejecuta `ping` y cuenta los documentos de `sensores_datos`: devolvió 12,960 documentos. Última verificación registrada: 2026-10-12 00:41:35 UTC.

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py verificar mongodb
VERIFICACIÓN REAL DE CONEXIONES – 2026-10-11T23:34:46.931016+00:00
mongodb: CONEXIÓN VERIFICADA
{"motor": "mongodb", "connector": "PyMongo", "timestamp_utc": "2026-10-11T23:34:46.855690+00:00", "verified": true, "database": "bigdata_final", "documents": 12960}
 Container bigdata-final-analisis-run-e79e7c5ef1b0 Creating
 Container bigdata-final-analisis-run-e79e7c5ef1b0 Created
```

***Figura 35.*** *Conexión exitosa a MongoDB mediante PyMongo (CONEXIÓN VERIFICADA, 12,960 documentos).*

## 6.4 Visualizaciones

Las diez figuras siguientes se calculan sobre los documentos extraídos de MongoDB. Las consultas están en `mongo/consultas.js`; en todas, `c` es la colección `sensores_datos` y `tz` la zona `America/Santo_Domingo`. Las magnitudes con unidades distintas nunca se suman entre sí.

## S01 · Serie temporal de temperatura por ciudad

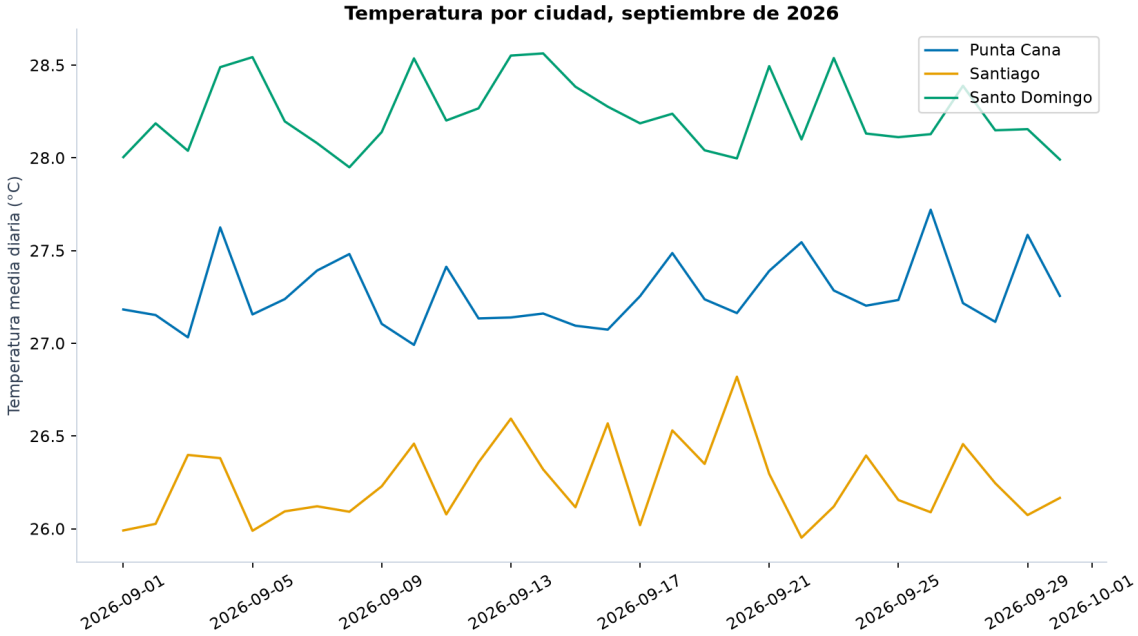



***Figura 36.*** *Serie temporal de temperatura por ciudad (S01). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Time series: Evolución de temperatura por ciudad».

**Por qué este gráfico.** Una línea por ciudad permite comparar el nivel y la variación diaria de la temperatura.

**Cómo leerlo.** Cada punto es el promedio diario de las dos estaciones de la ciudad y de las 24 horas.

**Construcción.** Se promedian dos estaciones y 24 lecturas horarias por día y ciudad. La fecha se obtiene después de convertir UTC a America/Santo_Domingo.

**Resultado observado.** Santo Domingo tiene la mayor temperatura media: 28.24 °C.

**Insight y decisión que apoya.** Inspeccionar cambios diarios y confirmar si responden a señales persistentes o a puntos anómalos.

**Límite de interpretación.** Todas las lecturas son sintéticas; las diferencias entre ciudades se definieron en el generador y no describen el clima real.

**Consulta** (`mongo/consultas.js`):

```
// S01: media diaria por ciudad, sólo temperatura.
c.aggregate([
  { $match: { type: 'temperatura' } },
  { $group: { _id: { date: { $dateTrunc: { date: '$timestamp', unit: 'day', timezone: tz } },
                     city: '$city' },
              mean: { $avg: '$value' }, n: { $sum: 1 } } },
  { $sort: { '_id.date': 1 } }
]);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
temp = s[s.type == 'temperatura']
daily = temp.groupby(['date','city'], as_index=False).value.mean()
```

## S02 · Mapa de calor geográfico de lecturas

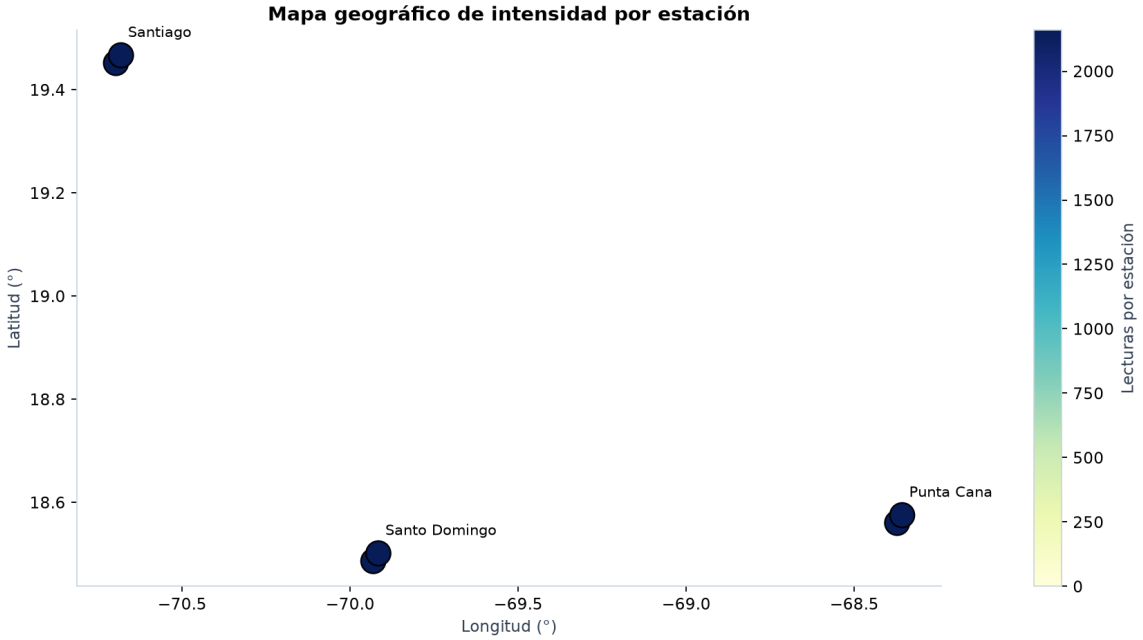



***Figura 37.*** *Mapa de calor geográfico de lecturas (S02). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Mapas de calor geográficos: Distribución de lecturas».

**Por qué este gráfico.** Ubicar las estaciones en sus coordenadas muestra la distribución geográfica de la red de sensores; el color indica cuántas lecturas aporta cada punto, lo que permite detectar zonas con menos cobertura.

**Cómo leerlo.** El eje horizontal es la longitud y el vertical la latitud, a la misma escala, de modo que las posiciones respetan la geografía: Santiago al noroeste, Santo Domingo al sur y Punta Cana al este. Cada círculo es una estación (dos por ciudad, muy cercanas entre sí) y su color, la cantidad de lecturas. Todas tienen el mismo color porque cada estación registró 2,160 lecturas.

**Construcción.** Se agregan lecturas por coordenada de estación. La figura estática ubica puntos coloreados en latitud/longitud; el dashboard muestra las estaciones en longitud/latitud con la misma escala, coloreadas por lecturas, sin depender de un mapa externo.

**Resultado observado.** Las seis estaciones tienen 2,160 lecturas cada una; la cobertura de observación es uniforme.

**Insight y decisión que apoya.** Utilizar el mapa para reconocer distribución de estaciones y revisar diferencias de cobertura cuando se incorporen datos reales.

**Límite de interpretación.** El mapa representa conteo y no temperatura. Las coordenadas de estaciones son ilustrativas; no corresponden a instalaciones verificadas.

**Consulta** (`mongo/consultas.js`):

```
// S02: intensidad de lecturas por ubicación.
c.aggregate([
  { $group: { _id: { city: '$city', station: '$station', lat: '$latitude', lon: '$longitude'
    },
              n: { $sum: 1 } } }
]);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
geo = s.groupby(['city','station','latitude','longitude'],
 as_index=False).agg(readings=('value','size'))
```

## S03 · Violines por tipo de sensor

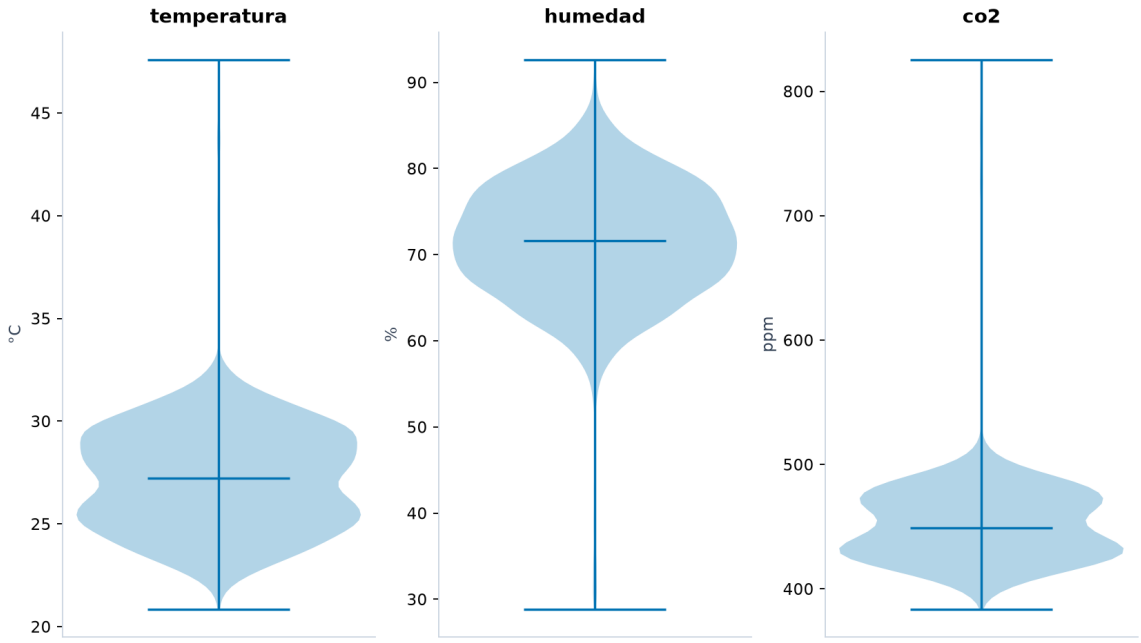



***Figura 38.*** *Violines por tipo de sensor (S03). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Gráfico de violín: Distribución de valores por tipo de sensor».

**Por qué este gráfico.** El violín compara la forma de la distribución de cada magnitud. Se usa un panel por tipo porque temperatura, humedad y CO2 tienen unidades y escalas distintas y no pueden compartir un eje.

**Cómo leerlo.** Cada panel es un tipo de sensor con su unidad (°C, % y ppm). El ancho del violín indica dónde se concentran las lecturas y la línea central, la mediana: 27.23 °C, 71.64% y 449.10 ppm. Las colas largas hacia arriba (temperatura y CO2) o hacia abajo (humedad) corresponden a las anomalías.

**Construcción.** Se dibujan tres distribuciones en paneles con escalas y unidades independientes. Las colas incluyen anomalías inyectadas.

**Resultado observado.** Las medianas son temperatura 27.23 °C, humedad 71.64% y CO2 449.10 ppm.

**Insight y decisión que apoya.** Comparar dispersión dentro de cada magnitud y revisar extremos con el detector robusto.

**Límite de interpretación.** Las variables tienen unidades distintas. Las alturas entre paneles no deben compararse; la forma depende del ancho de suavizado.

**Consulta** (`mongo/consultas.js`):

```
// S03: observaciones separadas por tipo y unidad; nunca sumar magnitudes diferentes.
c.find({}, { _id: 0, type: 1, value: 1, unit: 1 });
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
for typ in ['temperatura','humedad','co2']:
    values = s.loc[s.type == typ, 'value']
    # Un panel independiente y su unidad para cada magnitud.
```

## S04 · Matriz de dispersión entre sensores

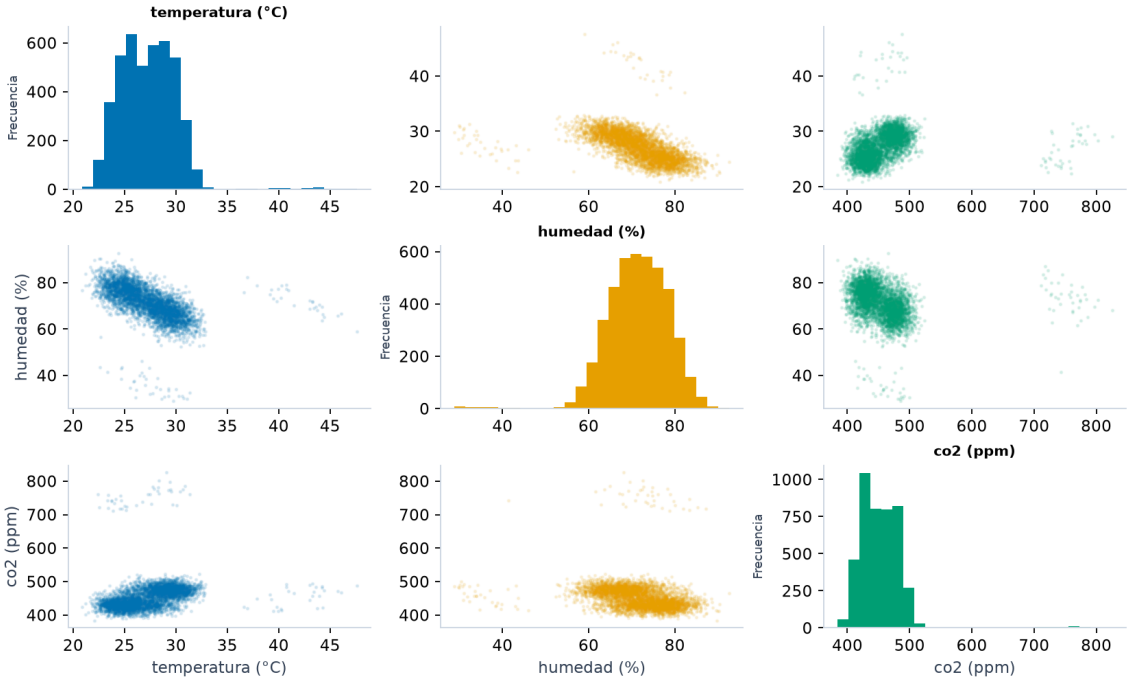



*Figura 39. Matriz de dispersión entre sensores (S04). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Scatter plot matrix: Correlación entre múltiples sensores».

**Por qué este gráfico.** La matriz de dispersión muestra en una sola figura todas las parejas de variables, lo que permite ver qué magnitudes se mueven juntas.

**Cómo leerlo.** En la diagonal están los histogramas de cada magnitud. Fuera de la diagonal, cada panel cruza dos magnitudes medidas en la misma estación y a la misma hora; cada punto es una hora. La nube temperatura-humedad baja de izquierda a derecha (relación inversa, r = −0.590) y la de temperatura-CO2 sube levemente (r = 0.341). Los puntos aislados, lejos de la nube, son anomalías.

**Construcción.** Se alinea cada estación y timestamp antes de pivotar tipos. No se correlacionan listas de lecturas sin correspondencia temporal.

**Resultado observado.** La correlación temperatura-humedad es -0.590; temperatura-CO2 es 0.341.

**Insight y decisión que apoya.** Inspeccionar dependencias y diferencias entre estaciones, y recalcular correlaciones excluyendo anomalías para analizar sensibilidad.

**Límite de interpretación.** La relación inversa entre humedad y temperatura fue introducida por el generador. Los puntos extremos alteran Pearson y no constituyen evidencia física de campo.

**Consulta** (`mongo/consultas.js`):

```
// S04: alineación por estación y timestamp; los tipos se pivotan en Python.
c.aggregate([
  { $group: { _id: { station: '$station', timestamp: '$timestamp' },
              readings: { $push: { type: '$type', value: '$value' } } } },
  { $sort: { '_id.timestamp': 1 } }
]);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
wide = s.pivot(index=['station','timestamp'],
               columns='type', values='value')
correlations = wide.corr()
```

## S05 · Áreas apiladas por ubicación

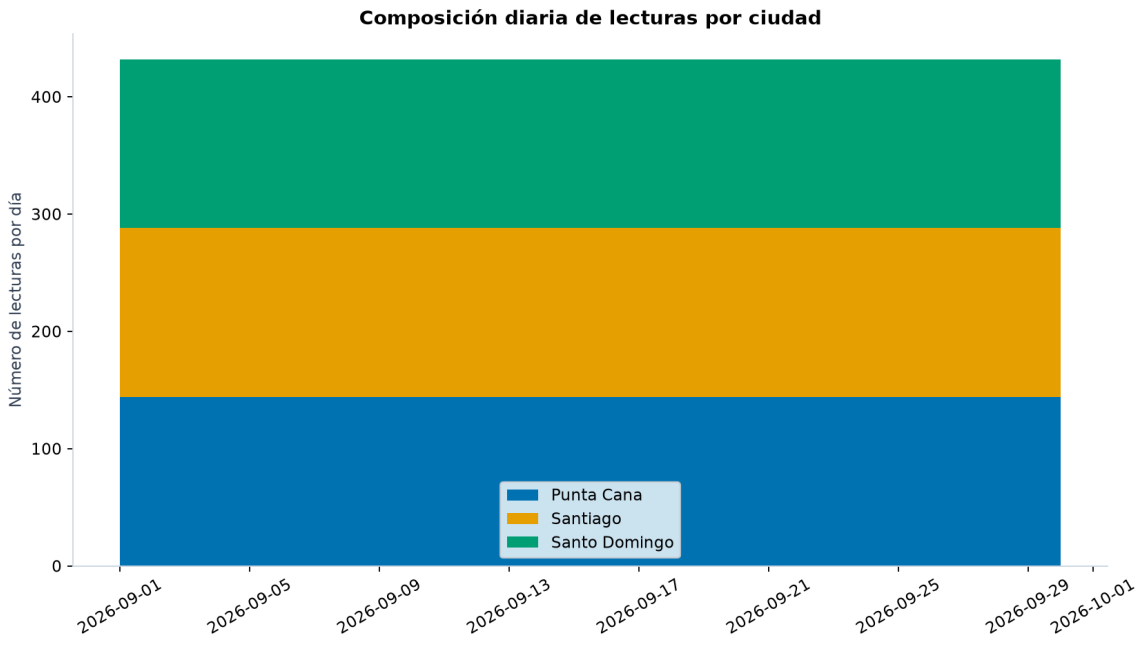



***Figura 40.*** *Áreas apiladas por ubicación (S05). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Gráfico de áreas apiladas: Composición de lecturas por ubicación».

**Por qué este gráfico.** El área apilada muestra el total diario y cuánto aporta cada ciudad, lo que sirve para detectar si una ubicación deja de enviar datos.

**Cómo leerlo.** Las bandas planas y del mismo grosor (144 lecturas por ciudad y día) indican que ningún sensor dejó de reportar.

**Construcción.** Se apilan conteos diarios por ciudad. Se usa conteo para evitar sumar temperatura, humedad y CO2, magnitudes incompatibles.

**Resultado observado.** Cada ciudad aporta 4,320 lecturas en 30 días; la participación diaria es 33.3% por ciudad.

**Insight y decisión que apoya.** Identificar pérdida de cobertura si una banda disminuye en un despliegue real.

**Límite de interpretación.** La estabilidad procede del muestreo sintético uniforme. La suma representa eventos y no intensidad de una variable física.

**Consulta** (`mongo/consultas.js`):

```
// S05: conteo de eventos diarios por ciudad.
c.aggregate([
  { $group: { _id: { date: { $dateTrunc: { date: '$timestamp', unit: 'day', timezone: tz } },
                     city: '$city' },
              n: { $sum: 1 } } },
  { $sort: { '_id.date': 1 } }
]);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
counts = s.groupby(['date','city']).size().unstack(fill_value=0)
```

## S06 · Detección de anomalías con MAD

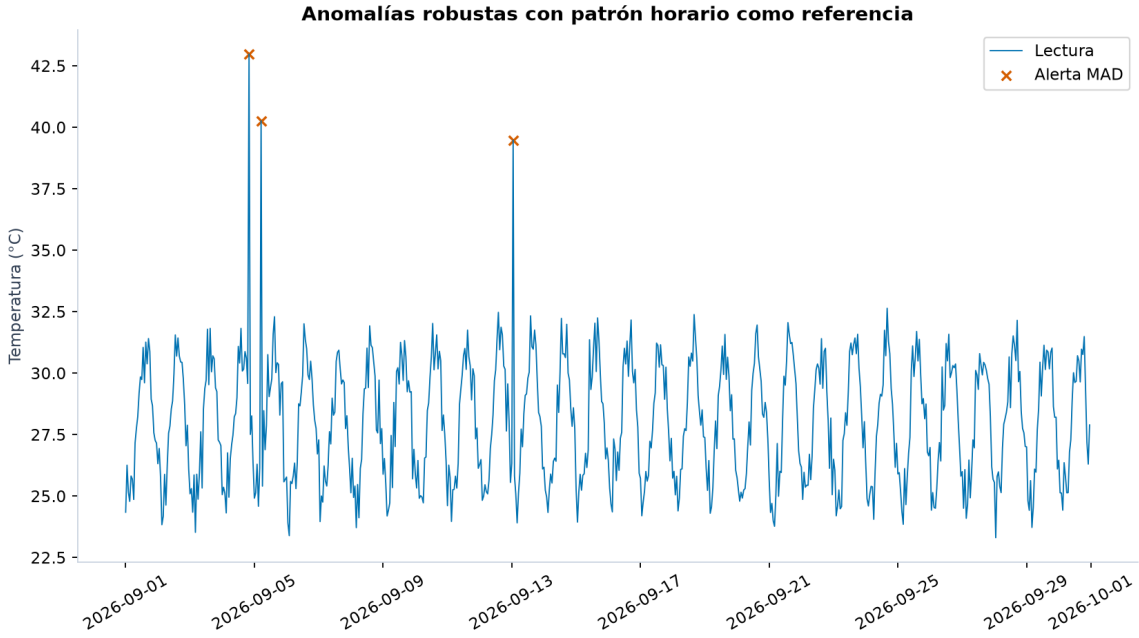



***Figura 41.*** *Detección de anomalías con MAD (S06). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Anomaly detection: Identificación de valores atípicos».

**Por qué este gráfico.** Una serie con las alertas marcadas muestra cuándo ocurren los valores atípicos frente al comportamiento normal del sensor.

**Cómo leerlo.** Se muestra el sensor de temperatura SDQ-1 durante septiembre. La línea azul es la lectura horaria, con su ciclo de día y noche; las X naranjas son las lecturas que el detector marcó como atípicas, como los picos de más de 40 °C. El detector compara cada lectura con lo normal para esa hora del día, por eso no confunde el calor de la tarde con una anomalía. En el dashboard se puede elegir otro sensor.

**Construcción.** Por sensor y hora del día se calcula mediana y MAD. Se alerta si |valor - mediana| / (1.4826 × MAD) > 3.5. La etiqueta inyectada sólo se usa para evaluar.

**Resultado observado.** En las 12,960 lecturas se detectan 137 alertas: 103 verdaderos positivos, 34 falsos positivos y 0 falsos negativos. Precisión 75.2%; sensibilidad 100.0%.

**Insight y decisión que apoya.** Revisar alertas antes de eliminar datos y calibrar el umbral con una ventana histórica independiente.

**Límite de interpretación.** La referencia usa todo septiembre: es detección retrospectiva, no validación predictiva. En producción debe entrenarse sobre datos previos y controlar MAD igual a cero.

**Consulta** (`mongo/consultas.js`):

```
// S06: series para mediana/MAD por sensor y hora local (cálculo en Python).
c.aggregate([
  { $project: { _id: 0, sensor_id: 1, value: 1, timestamp: 1,
                hour: { $hour: { date: '$timestamp', timezone: tz } } } },
  { $sort: { sensor_id: 1, timestamp: 1 } }
]);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
s['score'] = abs(s.value - s.baseline) / (1.4826 * s.mad)
s['detected'] = s.score > 3.5
# injected_anomaly NO participa en la regla.
```

## S07 · Calendario de patrones diarios

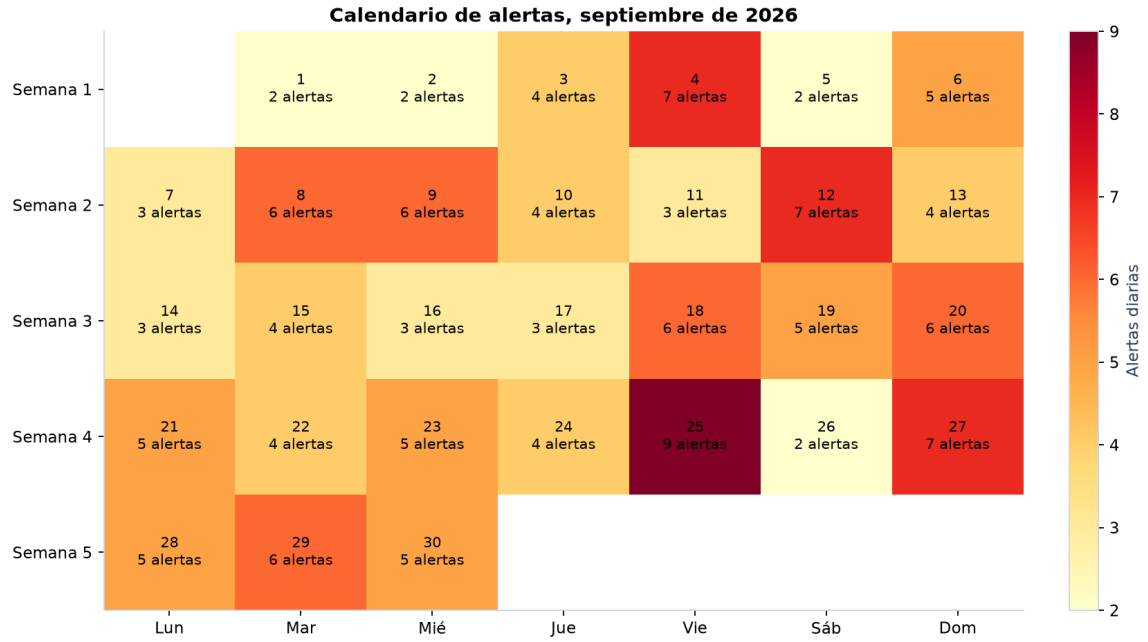



***Figura 42.*** *Calendario de patrones diarios (S07). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Gráfico de calendario: Patrones diarios/semanales».

**Por qué este gráfico.** El calendario organiza los días en semanas (filas) y días de la semana (columnas), lo que permite ver a la vez el patrón diario y el semanal.

**Cómo leerlo.** Cada celda es un día de septiembre de 2026, con su número y el total de alertas de ese día; el color se intensifica cuantas más alertas hay. El 25 de septiembre (viernes) es el día más oscuro, con 9 alertas. Las celdas vacías son días fuera del mes.

**Construcción.** Se suman alertas por día local y se organizan en semanas de lunes a domingo. Celdas fuera de septiembre permanecen vacías.

**Resultado observado.** El mayor conteo corresponde al 2026-09-25: 9 alertas sobre 432 lecturas diarias.

**Insight y decisión que apoya.** Comparar alertas diarias con volumen de lecturas para localizar jornadas de revisión.

**Límite de interpretación.** Sólo hay un mes de datos simulados. El máximo no confirma un patrón semanal estable ni una falla real.

**Consulta** (`mongo/consultas.js`):

```
// S07: datos para el calendario; las alertas se derivan en Python.
c.find({}, { _id: 0, timestamp: 1, sensor_id: 1, value: 1 });
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
daily_alerts = s.groupby('date').detected.sum()
# Matriz calendario con huecos fuera del mes.
```

## S08 · Red de relaciones entre sensores

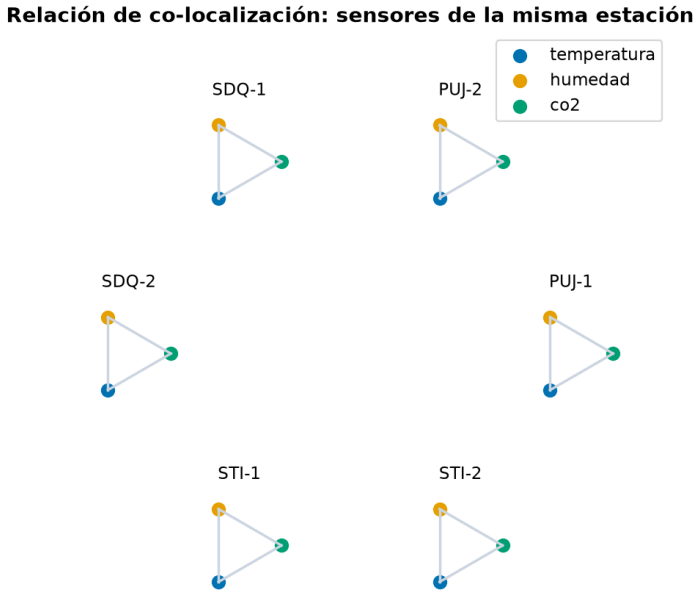



***Figura 43.*** *Red de relaciones entre sensores (S08). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Network graph: Relaciones entre sensores conectados».

**Por qué este gráfico.** Un grafo de red representa elementos (nodos) y sus relaciones (enlaces). Como los datos no registran conexiones físicas, la relación que se modela es la co-localización: dos sensores están enlazados si están en la misma estación.

**Cómo leerlo.** Cada punto es un sensor, coloreado por tipo (temperatura, humedad o CO2), y cada línea une sensores de la misma estación. Se forman seis triángulos, uno por estación (por ejemplo, SDQ-1 en Santo Domingo), con tres sensores cada uno.

**Construcción.** Se crea un enlace entre dos sensores cuando comparten station. La posición es un diseño de red; no expresa coordenadas geográficas.

**Resultado observado.** La red tiene 18 sensores, 18 relaciones y seis grupos de tres sensores.

**Insight y decisión que apoya.** Examinar qué magnitudes se observan conjuntamente y organizar validaciones por estación.

**Límite de interpretación.** La relación es de co-localización definida en el esquema sintético. No representa conexiones físicas, tráfico de red ni causalidad.

**Consulta** (`mongo/consultas.js`):

```
// S08: co-localización de sensores, no conectividad física.
c.aggregate([
  { $group: { _id: { sensor_id: '$sensor_id', station: '$station', type: '$type' } } }
]);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
nodes = s[['sensor_id','station','type']].drop_duplicates()
# Enlace sólo si dos nodos comparten station.
```

## S09 · Dispersión 3D: valor, ubicación y tiempo

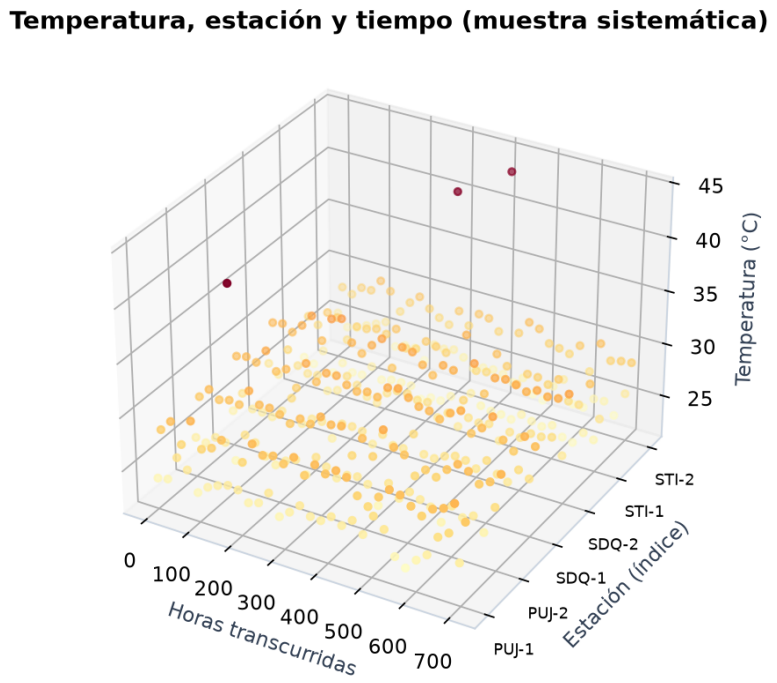



***Figura 44.*** *Dispersión 3D: valor, ubicación y tiempo (S09). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «3D scatter plot: Valor vs. Ubicación vs. Tiempo».

**Por qué este gráfico.** Un gráfico 3D permite ver tres variables a la vez; en el dashboard se puede rotar para explorar los puntos desde distintos ángulos.

**Cómo leerlo.** Un eje es el tiempo (horas transcurridas desde el 1 de septiembre), otro la ubicación (las seis estaciones) y el vertical, la temperatura. Se muestra una lectura de cada doce (360 puntos) para que la nube sea legible; los puntos más altos y oscuros son temperaturas anómalas.

**Construcción.** Se usa sólo temperatura para mantener una unidad coherente. Se toma una fila de cada doce y se codifica la estación como índice categórico.

**Resultado observado.** El gráfico contiene 360 puntos de 4,320 lecturas de temperatura.

**Insight y decisión que apoya.** Rotar y explorar el gráfico para localizar valores altos y comparar con las series temporales completas.

**Límite de interpretación.** La distancia entre índices de estación no es distancia geográfica. El muestreo reduce carga y puede omitir anomalías; el detector utiliza todas las lecturas.

**Consulta** (`mongo/consultas.js`):

```
// S09: sólo temperatura; muestreo sistemático reproducible en Python.
c.find({ type: 'temperatura' }, { _id: 0, station: 1, city: 1, value: 1, timestamp: 1 })
 .sort({ station: 1, timestamp: 1 });
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
sample = temp.iloc[::12].copy()
px.scatter_3d(sample, x='elapsed', y='station_index', z='value',
              color='city', hover_name='station')
```

## S10 · Dashboard de monitoreo simulado

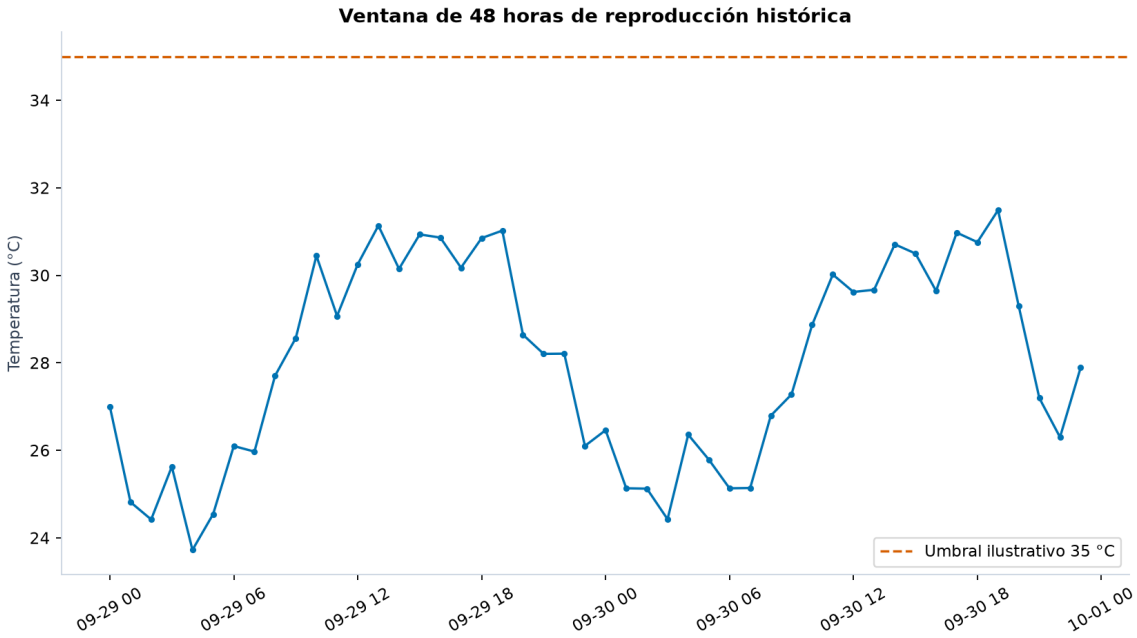



***Figura 45.*** *Dashboard de monitoreo simulado (S10). Fuente: MongoDB / sensores (simulación).*

**Requisito de la consigna.** «Dashboard en tiempo real: Monitoreo continuo».

**Por qué este gráfico.** Un monitor en tiempo real muestra las lecturas más recientes y las compara con un umbral de alerta. Como los datos son históricos, se simula: el dashboard reproduce la serie hora por hora.

**Cómo leerlo.** La figura muestra las últimas 48 horas de septiembre del sensor de temperatura SDQ-1; la línea punteada naranja marca el umbral ilustrativo de 35 °C. En el dashboard, el botón Reproducir agrega una lectura por segundo (una hora histórica) y la ventana avanza, como lo haría un monitor conectado a un flujo real.

**Construcción.** El dashboard incorpora controles de reproducir, pausar y reiniciar. Cada segundo agrega una lectura histórica de SDQ-1 a una ventana de 48 puntos.

**Resultado observado.** La última lectura histórica de SDQ-1 es 27.89 °C, registrada el 30/09/2026 23:00.

**Insight y decisión que apoya.** Usar la reproducción para probar controles, alertas visuales y actualización de una serie antes de conectar un flujo real.

**Límite de interpretación.** Un segundo de reproducción equivale a una hora histórica. No hay streaming desde MongoDB ni medición ambiental en vivo. El umbral de 35 °C es ilustrativo.

**Consulta** (`mongo/consultas.js`):

```
// S10: reproducción histórica; últimas 48 lecturas de un sensor.
c.find({ sensor_id: 'SDQ-1-temperatura' }, { _id: 0, value: 1, timestamp: 1 })
 .sort({ timestamp: -1 }).limit(48);
```

**Transformación y visualización en Python** (`src/analisis.py`):

```
history = temp[temp.station == 'SDQ-1']
# JavaScript avanza una lectura por segundo y conserva 48 puntos.
```

## 6.5 Insights obtenidos de los sensores

- **Patrones por ubicación.** Santo Domingo registra la mayor temperatura media (28.24 °C). La cobertura es uniforme: cada estación aporta 2,160 lecturas y cada ciudad el 33.3% diario.
- **Correlación entre sensores.** Temperatura y humedad se relacionan de forma inversa (r = −0.590) y temperatura y CO2 de forma directa (r = 0.341), relaciones definidas por el generador.
- **Anomalías.** El detector MAD identifica las 103 anomalías inyectadas (sensibilidad 100%) con 34 falsos positivos (precisión 75.2%): conviene revisar las alertas antes de descartar datos y calibrar el umbral.
- **Patrón diario.** El 25 de septiembre concentra 9 alertas sobre 432 lecturas, la jornada que más revisión requiere.
- **Monitoreo.** La reproducción histórica permite probar alertas visuales y controles antes de conectar un flujo real.

# 7. Extracción integrada, validación y dashboard

## 7.1 Extracción desde los tres gestores

`lab.py integrar` respalda los resultados anteriores y ejecuta `run.py --desde-bd` dentro del cliente. Este comando vuelve a verificar las tres conexiones, exporta las tablas de Northwind y HR por ODBC y la colección por PyMongo, registra la fecha y el hash SHA-256 de cada CSV en `proveniencia_fuentes.json`, recalcula las 30 visualizaciones, ejecuta las 17 comprobaciones y regenera el dashboard. Si una conexión falla, se detiene antes de modificar los datos.

El registro de procedencia funciona como un sello: al extraer cada tabla se guarda su hash SHA-256 (su «huella digital», ver glosario). Antes de usar los datos se recalcula la huella; si alguien modificara un CSV después de la extracción, la huella no coincidiría y el dashboard dejaría de presentar esos datos como extraídos de los gestores. Así se puede afirmar que las 30 figuras provienen de las bases y no de archivos editados a mano.

```
dancarrion@MacBook-Pro proyecto-final-db % python3 docker/lab.py integrar
Respaldo de resultados anteriores: /Users/dancarrion/respaldos_bigdata_final/20261012T004134Z
VERIFICACIÓN REAL DE CONEXIONES - 2026-10-12T00:41:36.057794+00:00
sqlserver: CONEXIÓN VERIFICADA
{"motor": "sqlserver", "connector": "ODBC", "timestamp_utc": "2026-10-12T00:41:35.696455+00:00", "verified": true, "database": "Northwind", "orders": 830}
oracle: CONEXIÓN VERIFICADA
{"motor": "oracle", "connector": "ODBC", "timestamp_utc": "2026-10-12T00:41:35.780996+00:00", "verified": true, "schema": "HR", "employees": 107}
mongodb: CONEXIÓN VERIFICADA
{"motor": "mongodb", "connector": "PyMongo", "timestamp_utc": "2026-10-12T00:41:35.986975+00:00", "verified": true, "database": "bigdata_final", "documents": 12960}
Extracción de sólo lectura completada: sqlserver
Extracción de sólo lectura completada: oracle
Extracción de sólo lectura completada: mongodb
Ejecutando analisis.py
```

***Figura 46.*** *Integración: respaldo, verificación de las tres conexiones y extracción de sólo lectura desde SQL Server, Oracle y MongoDB.*

```
Ejecutando dashboard.py
Dashboard listo: /workspace/dashboard/index.html
Ejecutando validar.py
17/17 comprobaciones correctas
```

***Figura 47.*** *Final de la integración: dashboard regenerado y 17/17 comprobaciones correctas.*

## 7.2 Dashboard interactivo

El dashboard (`dashboard/index.html`) organiza las 30 visualizaciones por base de datos. Un selector elige la base y otro la figura; los botones Anterior y Siguiente recorren las diez figuras. Cada vista muestra tres indicadores clave, el gráfico interactivo de Plotly (información al pasar el cursor, zoom, series activables desde la leyenda, treemap navegable y gráfico 3D rotable) y un panel con el método, el resultado, la decisión y el límite, además de la fórmula o el código [7].

Hay filtros específicos en N01 (país), H01 (departamento), S06 y S10 (sensor). El aviso superior indica la procedencia de los datos: tras la integración, informa que se extrajeron de SQL Server, Oracle y MongoDB. El archivo incluye los datos y una copia local de Plotly, por lo que funciona sin Internet ni servidor.

Para facilitar su consulta, el dashboard se publicó con GitHub Pages en https://dcarrioncordones.github.io/proyecto-final-db/dashboard/. Esa página sirve el mismo archivo del repositorio, con los datos extraídos de los tres gestores; no se conecta a las bases en tiempo real.

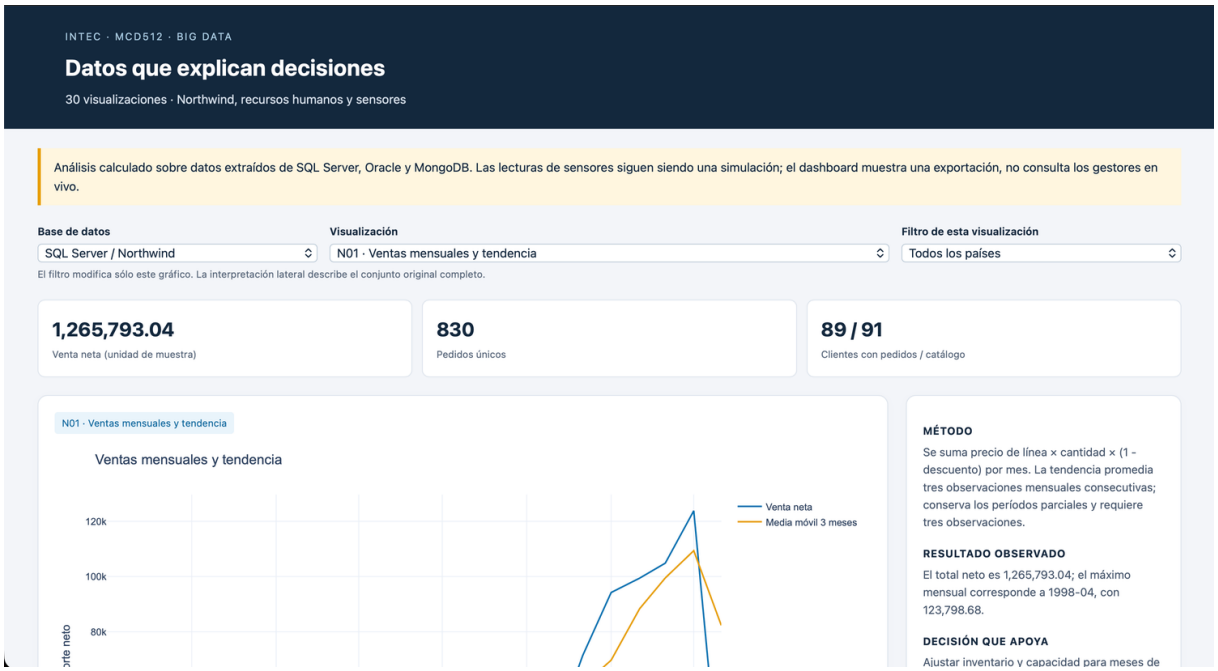



***Figura 48.*** *Vista general del dashboard: aviso de procedencia, selectores, filtro de país, indicadores y N01.*

El monitoreo en tiempo real se simula en S10: con Reproducir, Pausar y Reiniciar, cada segundo agrega una hora histórica de un sensor a una ventana de 48 lecturas y muestra el umbral de alerta. No es *streaming* desde MongoDB, sino una reproducción de la serie extraída.

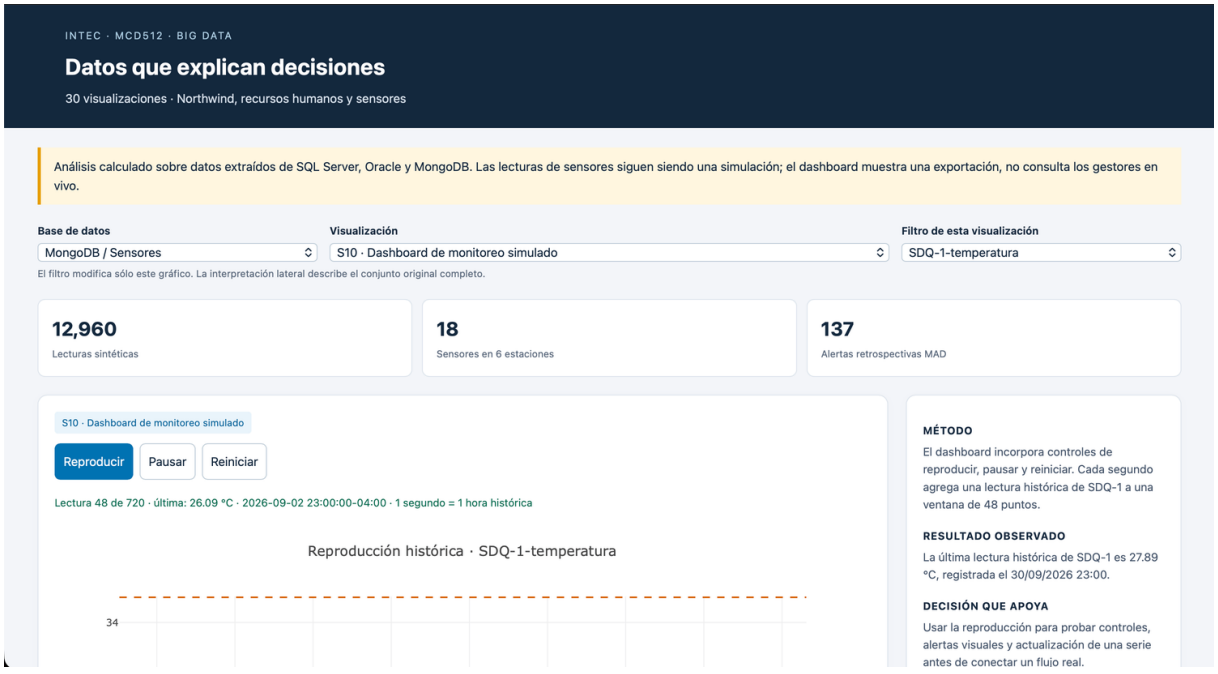



***Figura 49.*** *Dashboard de monitoreo simulado (S10) con controles de reproducción y filtro de sensor.*

## 7.3 Pruebas de usabilidad

Se recorrieron las 30 visualizaciones en el navegador, verificando que cada una se dibuja sin errores, y se probaron los filtros, la navegación y los controles de reproducción. En la revisión, el mapa de S02 aparecía en blanco: las seis estaciones tienen el mismo número de lecturas, por lo que la escala de densidad quedaba sin rango y Plotly no dibujaba nada. Se reemplazó por un gráfico de estaciones en longitud y latitud a igual escala, coloreado por número de lecturas, que además funciona sin mapas externos. El diseño apila los paneles en pantallas estrechas y los controles tienen etiquetas y foco visible para el teclado. El Anexo C reúne las capturas de las 30 vistas.

# 8. Discusión y recomendaciones

## 8.1 Interpretación de los resultados

La venta neta cuantifica ingresos después de descuentos, pero no utilidad: el treemap y el ranking de clientes señalan dónde concentrar el seguimiento, no dónde está el margen. El pico de abril de 1998 debe leerse en una serie cuyos extremos son meses incompletos; para reportes operativos conviene mostrar también los días cubiertos y la venta por día.

En HR, los histogramas y violines muestran la heterogeneidad salarial, pero explicar diferencias exige comparar puestos equivalentes. El historial de siete empleados describe cambios registrados y no permite estimar rotación. En los sensores, la MAD controla la influencia de los extremos; su precisión de 75.2% justifica una revisión manual, y la evaluación es retrospectiva: para medir generalización habría que calibrar con datos anteriores y probar con datos posteriores.

## 8.2 Escalabilidad y recomendaciones técnicas

Con volúmenes mayores, agregar en el gestor (por día, mes, departamento o ubicación) reduce la transferencia de datos; la extracción completa usada aquí es adecuada para el tamaño de las muestras, pero no se midió con millones de filas. Las siguientes mejoras se proponen como trabajo futuro, no como resultados comprobados:

| Componente | Mejora propuesta | Comprobación necesaria |
|---|---|---|
| SQL Server | Índices sobre fechas y claves de unión; preagregación mensual | Plan de ejecución y tiempos con volumen representativo |
| Oracle | Filtrar columnas y fechas; revisar índices de las relaciones | Plan de consulta y conteos después de filtros |
| MongoDB | Agregaciones temporales con índice compuesto; política de retención | explain(), selectividad y tamaño de la colección |
| Sensores | Referencia robusta entrenada sólo con datos anteriores | Prueba temporal separada: precisión y sensibilidad |
| Dashboard | Muestrear figuras densas conservando métricas completas | Tiempo de carga y pérdida visual por muestreo |

En cuanto a fechas, todas las agregaciones de sensores declaran la zona horaria explícitamente y Northwind usa la fecha del pedido sin hora, lo que evita desplazamientos de día al agrupar.

# 9. Conclusiones

## 9.1 Cumplimiento de la consigna

La tabla indica dónde se cumple cada requisito de la guía del trabajo final.

| Requisito de la consigna | Dónde se cumple |
|---|---|
| Portada: título, nombre, matrícula, fecha y asignatura | Portada |
| Introducción: objetivos y metodología | Secciones 1 y 2 |
| Descripción de cada base de datos | Secciones 4.1, 5.1 y 6.1 |
| Capturas de conexión exitosa | Secciones 4.3, 5.3 y 6.3, con el montaje del laboratorio en la sección 3 |
| 10 visualizaciones por gestor con explicación detallada | Secciones 4.4, 5.4 y 6.4; cada visualización indica qué ejemplo de la consigna cumple |
| Código y fórmulas utilizadas | Consulta y código Python en cada visualización; definiciones en la sección 2.2 |
| Insights obtenidos | Secciones 4.5, 5.5 y 6.5, y discusión en la sección 8 |
| Conexiones ODBC (SQL Server y Oracle) y conector de MongoDB | Secciones 2.1, 3.3, 3.4, 4.3, 5.3 y 6.3 |
| Scripts SQL, Oracle y MongoDB | `sql/northwind.sql`, `sql/hr.sql` y `mongo/consultas.js` (Anexo B) |
| Código Python para las visualizaciones | `src/` y `run.py` (Anexo B) |
| Archivos de configuración de conexión | `docker/` y `config/` (Anexo B) |
| Documentación de instalación de dependencias | `README.md`, `COMO_CORRER.md` y Anexo A |
| Dashboards interactivos (opcional) | Dashboard HTML publicado en línea y notebook de Jupyter ejecutable (sección 7.2) |
| Recomendaciones técnicas: consultas, fechas, escalabilidad, accesibilidad y rendimiento | Secciones 2.2, 7.2 y 8.2 |

## 9.2 Conclusiones

Se establecieron las tres conexiones requeridas: SQL Server y Oracle mediante ODBC (Microsoft ODBC Driver 18 y Oracle Instant Client ODBC) y MongoDB mediante PyMongo, todas verificadas con consultas reales sobre tablas y colecciones del trabajo. Los datos de las 30 visualizaciones provienen de esas conexiones, con procedencia trazable por hash.

Las visualizaciones cubren los requisitos de cada gestor: ventas, clientes, empleados, tendencias y productos en Northwind; compensación, ubicación, estructura, movilidad y comparaciones por departamento en HR; y series temporales, patrones por ubicación, anomalías, correlaciones y monitoreo simulado en los sensores. Cada figura explica su método, resultado, decisión y límite, y su precisión se respalda con 17 comprobaciones automáticas.

La principal lección metodológica es mantener coherencia entre lo que contiene la fuente y lo que comunica el gráfico: el ingreso no se presenta como utilidad, las métricas estructurales no se presentan como competencias, los cambios registrados no se extrapolan a rotación y la reproducción histórica no se presenta como *streaming*. El laboratorio en Docker, la extracción automatizada y el dashboard sin dependencias permiten reproducir todo el trabajo con unos pocos comandos.

# Referencias


[1] Microsoft. Northwind and pubs sample databases for SQL Server (instnwnd.sql). https://github.com/microsoft/sql-server-samples/blob/master/samples/databases/northwind-pubs/instnwnd.sql

[2] Oracle. Database Sample Schemas: Human Resources (HR). https://github.com/oracle-samples/db-sample-schemas/tree/main/human_resources

[3] Microsoft Learn. Install the Microsoft ODBC driver for SQL Server (Linux). https://learn.microsoft.com/en-us/sql/connect/odbc/linux-mac/installing-the-microsoft-odbc-driver-for-sql-server

[4] Oracle. Oracle Instant Client Downloads for Linux x86-64 (Basic y ODBC). https://www.oracle.com/database/technologies/instant-client/linux-x86-64-downloads.html

[5] MongoDB. PyMongo Driver: Connect to MongoDB. https://www.mongodb.com/docs/languages/python/pymongo-driver/current/connect/

[6] MongoDB. $dateTrunc (aggregation expression operator). https://www.mongodb.com/docs/manual/reference/operator/aggregation/dateTrunc/

[7] Plotly. Plotly Open Source Graphing Library for Python. https://plotly.com/python/

[8] Microsoft Learn. Deploy and connect to SQL Server Linux containers. https://learn.microsoft.com/en-us/sql/linux/sql-server-linux-docker-container-deployment

[9] Oracle. Oracle Database Free container images (Oracle Container Registry). https://container-registry.oracle.com/ords/ocr/ba/database/free

[10] Docker. Docker Desktop settings: Virtual Machine Options y Rosetta. https://docs.docker.com/desktop/settings-and-maintenance/settings/

[11] Leys, C., Ley, C., Klein, O., Bernard, P. y Licata, L. (2013). Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. Journal of Experimental Social Psychology, 49(4), 764–766. https://doi.org/10.1016/j.jesp.2013.03.013


Guía docente: Trabajo final de Big Data, MCD512, INTEC, Prof. José M. Aquino, marzo de 2026. Se utilizó para definir los entregables y criterios del trabajo.

# Anexo A. Reproducción del laboratorio

Desde la carpeta del proyecto, con Docker Desktop abierto y Rosetta activado, la secuencia completa es la siguiente. Cada comando debe terminar correctamente antes de ejecutar el siguiente; la guía detallada está en `COMO_CORRER.md`.

```
python3 docker/lab.py configurar          # contraseñas en docker/.env
python3 docker/lab.py validar
python3 docker/lab.py iniciar sqlserver
python3 docker/lab.py cargar sqlserver
python3 docker/lab.py iniciar oracle
python3 docker/lab.py cargar oracle
python3 docker/lab.py iniciar mongodb
python3 docker/lab.py cargar mongodb
python3 docker/lab.py construir-cliente
python3 docker/lab.py drivers
python3 docker/lab.py verificar sqlserver   # también oracle y mongodb
python3 docker/lab.py integrar             # extracción, análisis, validación y dashboard
python3 docker/lab.py detener              # detiene los gestores conservando los datos
```

Para ejecutar el análisis sin Docker sobre los CSV ya extraídos se usa un entorno local de Python:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python run.py --usar-exportados
```

El notebook `Trabajo_final_Big_Data.ipynb` ejecuta las mismas etapas y muestra las figuras; detecta si los datos provienen de los gestores y, en ese caso, no los reemplaza por las muestras.

# Anexo B. Código fuente y configuración

## B.1 Repositorio

El proyecto completo está publicado en https://github.com/dcarrioncordones/proyecto-final-db. La tabla resume los archivos principales.

| Archivo | Contenido |
|---|---|
| sql/northwind.sql | Consultas de SQL Server por visualización (N01–N10) |
| sql/hr.sql | Consultas de Oracle por visualización (H01–H10) |
| mongo/consultas.js | Agregaciones de MongoDB por visualización (S01–S10) |
| mongo/preparar_coleccion.js | Validador de esquema e índices de la colección |
| src/conexiones.py | Verificación de conexiones (ODBC y PyMongo) y extracción de sólo lectura |
| src/analisis.py | Preparación analítica y las 30 figuras (PNG y Plotly) |
| src/validar.py | 17 comprobaciones de datos y resultados |
| src/dashboard.py, dashboard_template.html | Construcción del dashboard interactivo |
| src/origen.py | Registro y comprobación de procedencia por hash |
| run.py | Orquestación del análisis: --desde-bd y --usar-exportados |
| Trabajo_final_Big_Data.ipynb | Notebook de Jupyter ejecutable |
| docker/compose.yaml, Dockerfile | Gestores, volúmenes, puertos y cliente con drivers ODBC |
| docker/lab.py, docker/cliente.py | Operación del laboratorio y cadenas de conexión |
| config/.env.example, odbc.ini.example | Plantillas de conexión sin credenciales reales |
| requirements.txt, README.md, COMO_CORRER.md | Dependencias e instrucciones de instalación y ejecución |

## B.2 Dashboard en línea

El dashboard (`dashboard/index.html`) está publicado con GitHub Pages en https://dcarrioncordones.github.io/proyecto-final-db/dashboard/. También puede abrirse localmente con doble clic desde la carpeta del proyecto, sin Internet ni servidor.

# Anexo C. Capturas del dashboard

Las capturas siguientes muestran la versión interactiva de cada visualización en el dashboard, con su panel explicativo. Se tomaron en Safari después de la integración con los tres gestores. Cada base comienza con una vista general que incluye el aviso de procedencia, los selectores y sus tres indicadores clave, comunes a sus diez visualizaciones.

## C.1 SQL Server / Northwind

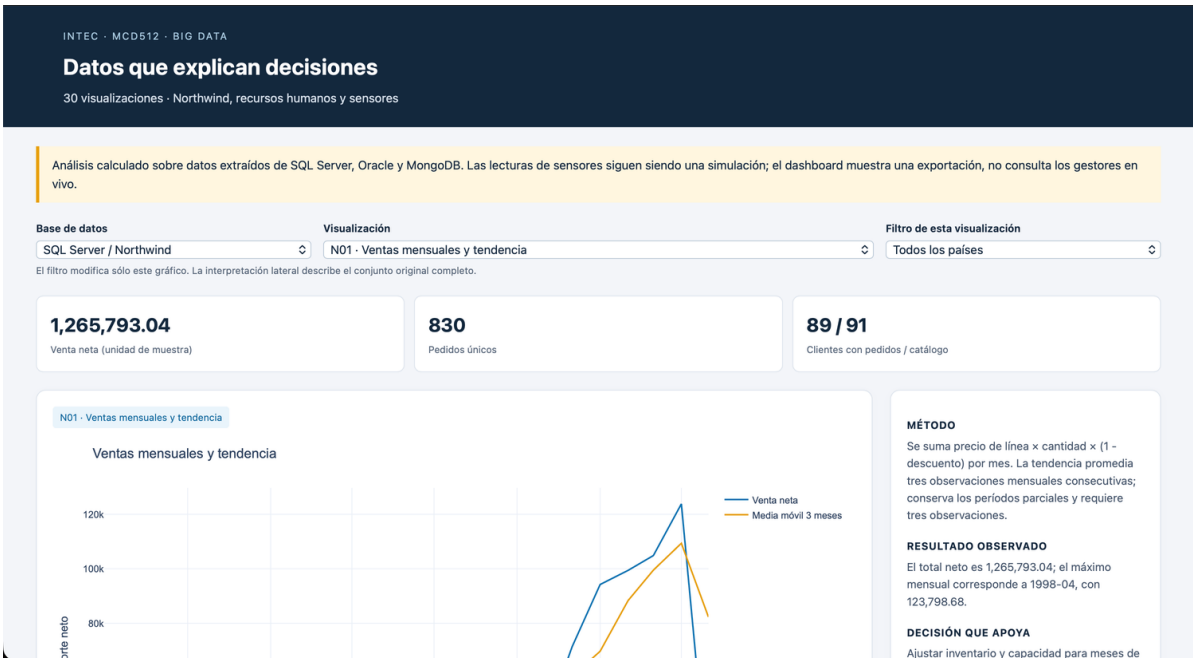



***Figura 50.*** *Dashboard: vista general de SQL Server / Northwind con indicadores clave.*

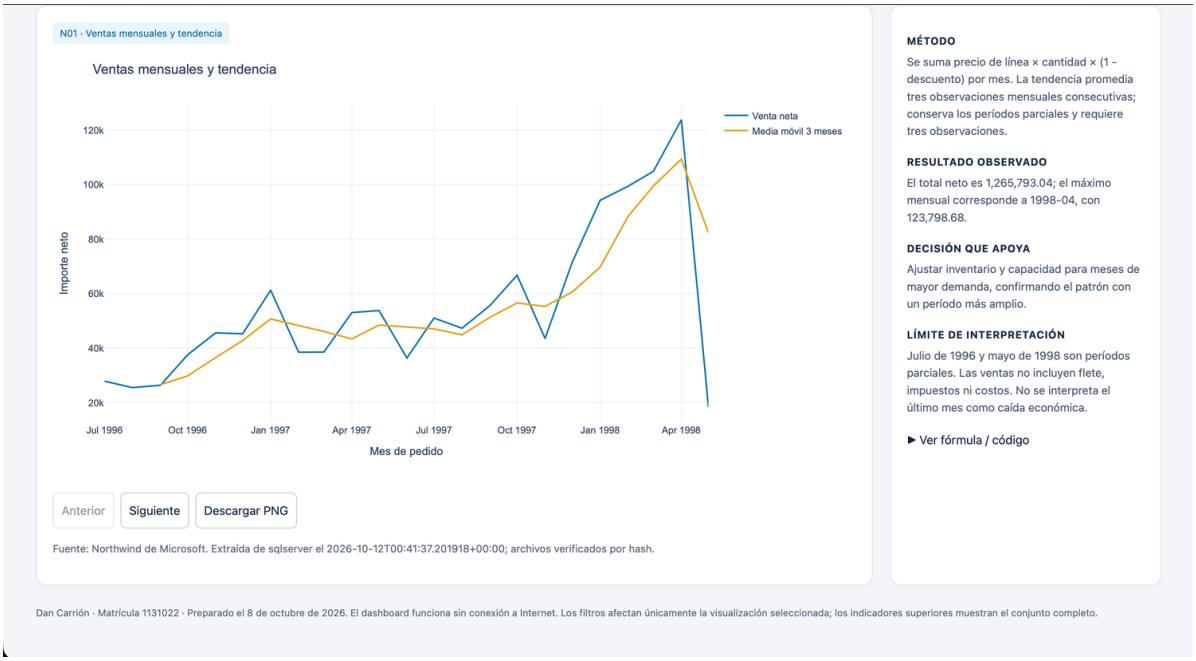



***Figura 51.*** *Dashboard: N01 · Ventas mensuales y tendencia.*

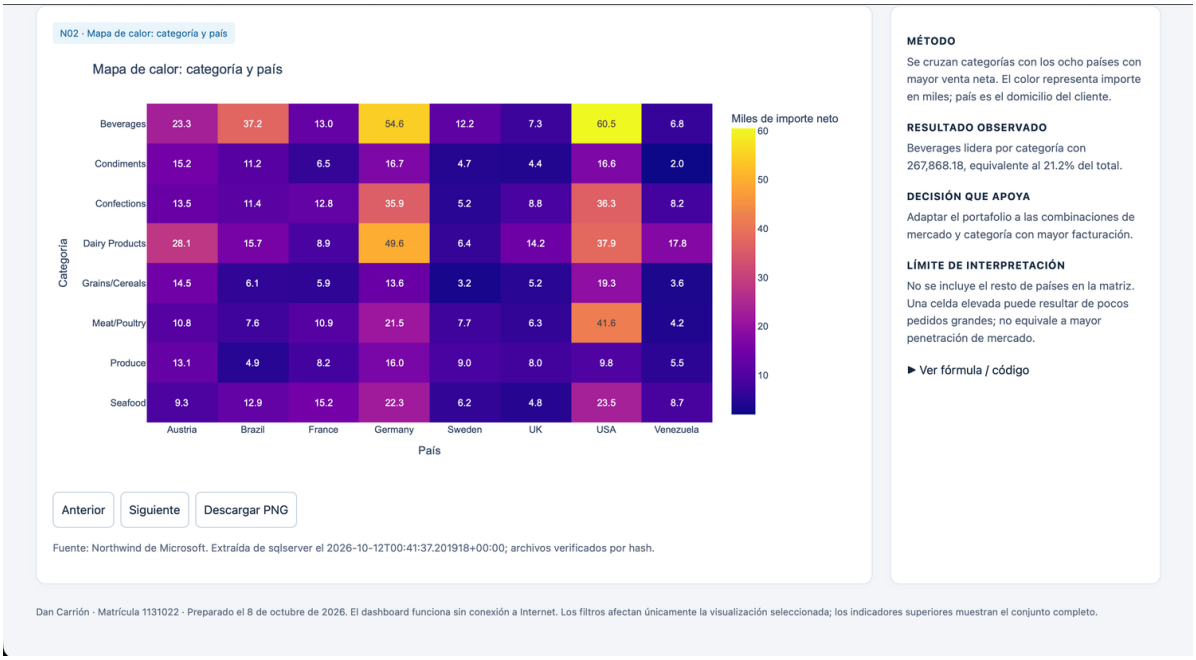



***Figura 52.*** *Dashboard: N02 · Mapa de calor: categoría y país.*

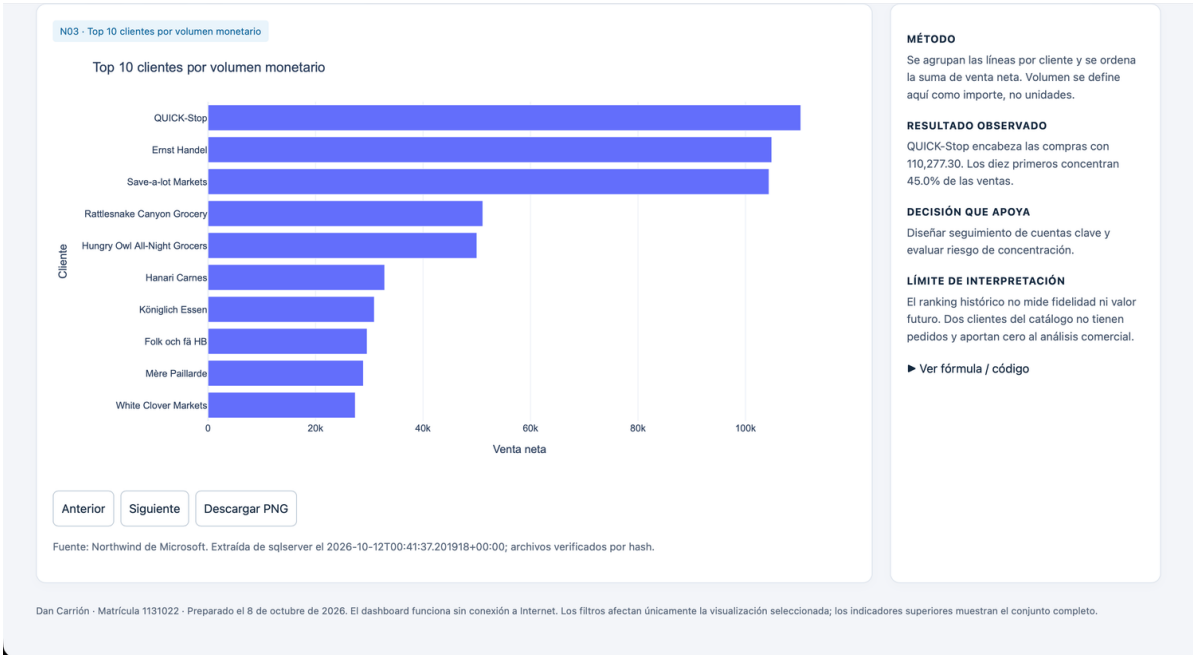



***Figura 53.*** *Dashboard: N03 · Top 10 clientes por volumen monetario.*

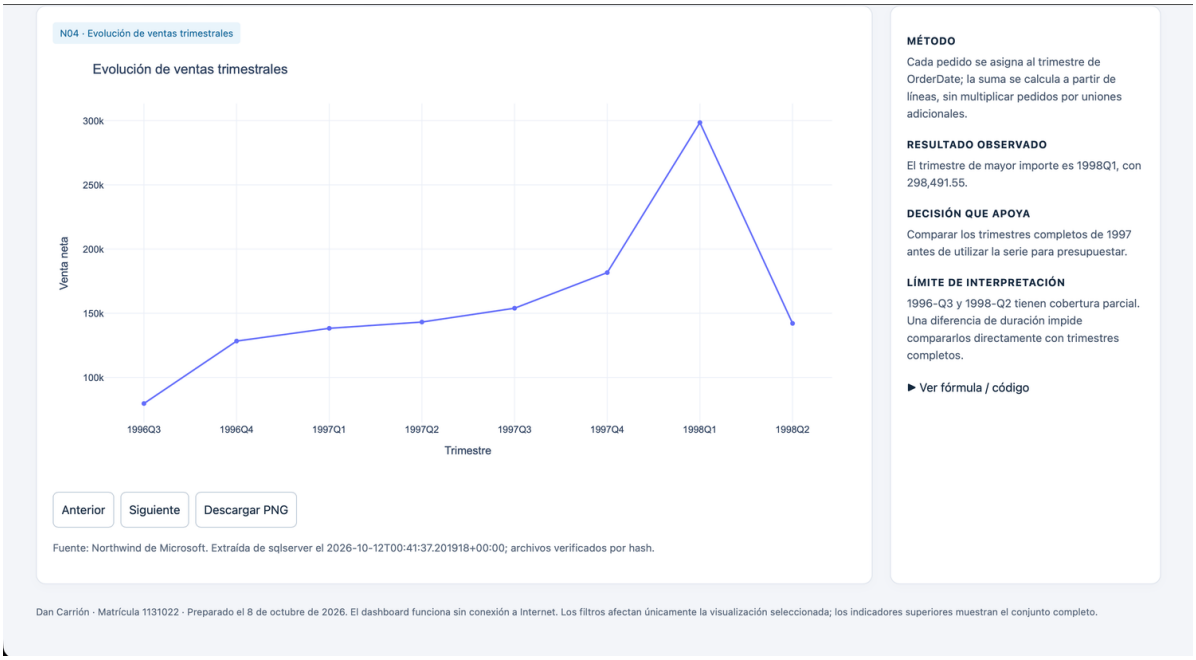



***Figura 54.*** *Dashboard: N04 · Evolución de ventas trimestrales.*

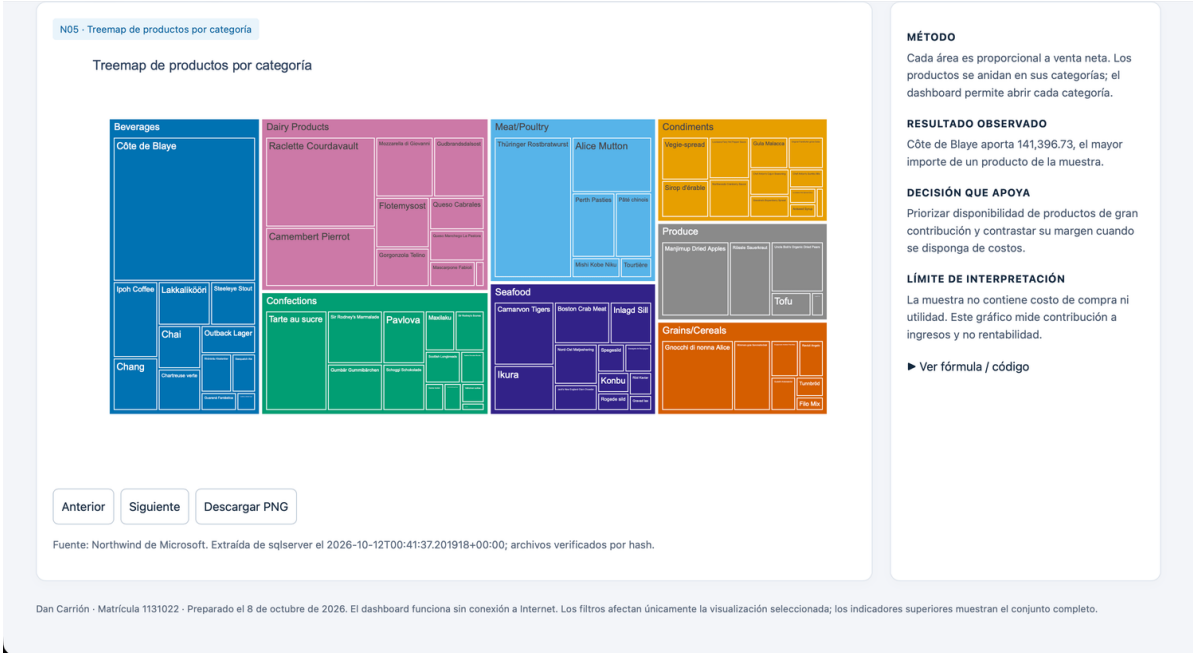



***Figura 55.*** *Dashboard: N05 · Treemap de productos por categoría.*

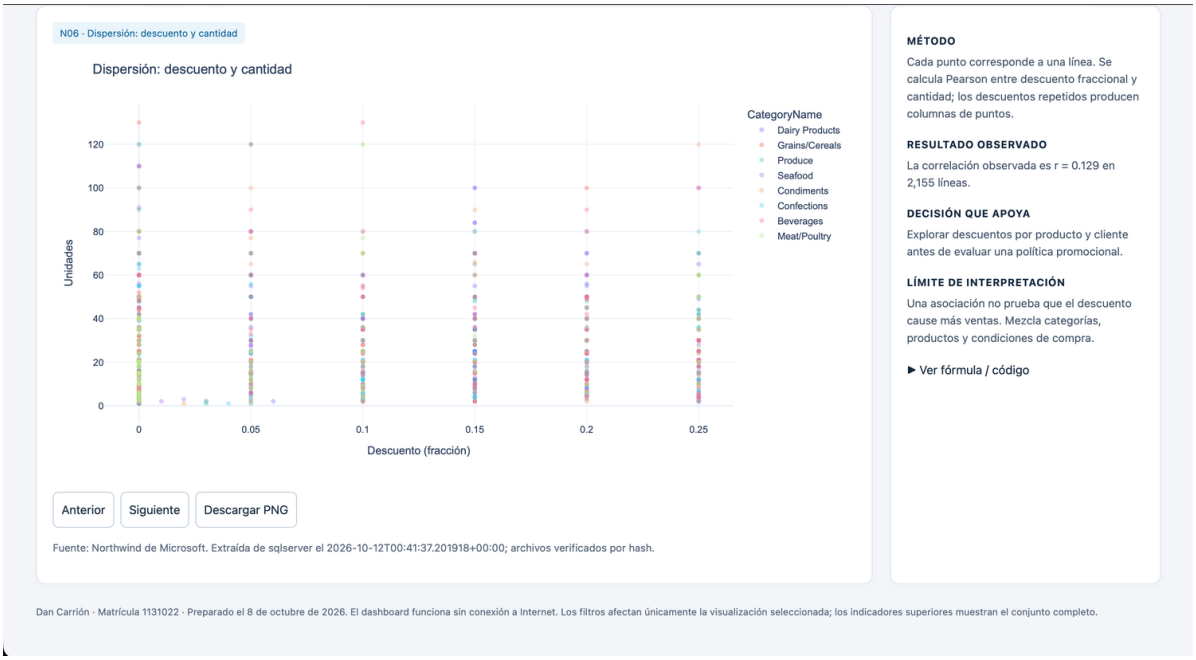



***Figura 56.*** *Dashboard: N06 · Dispersión: descuento y cantidad.*

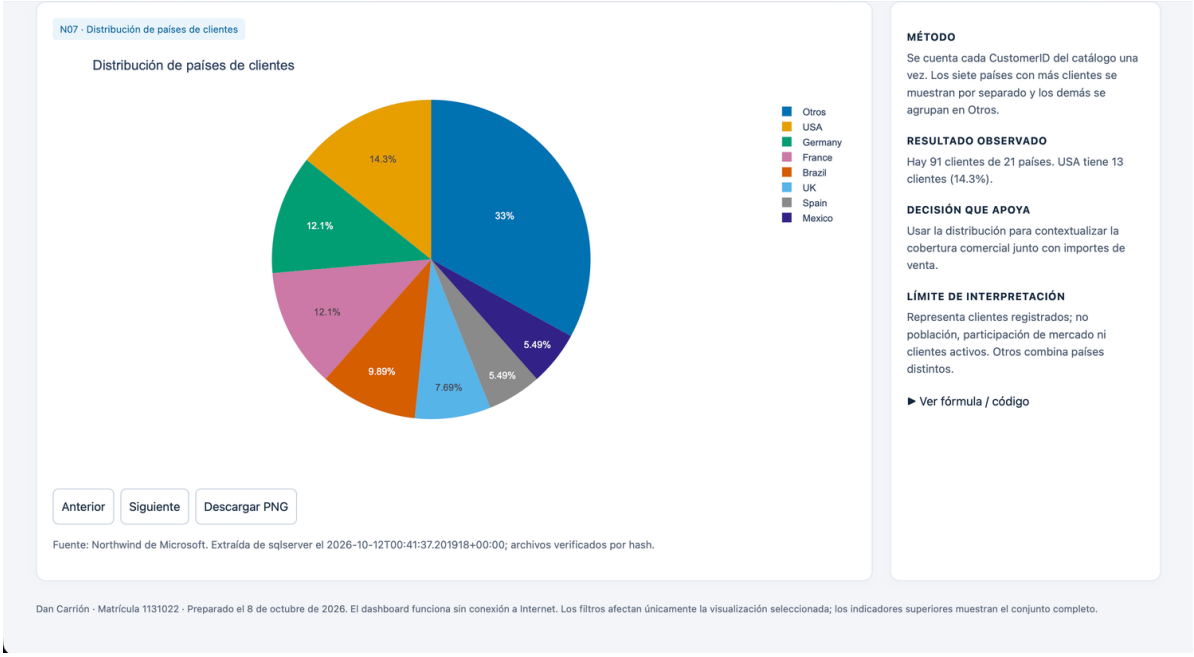



***Figura 57.*** *Dashboard: N07 · Distribución de países de clientes.*

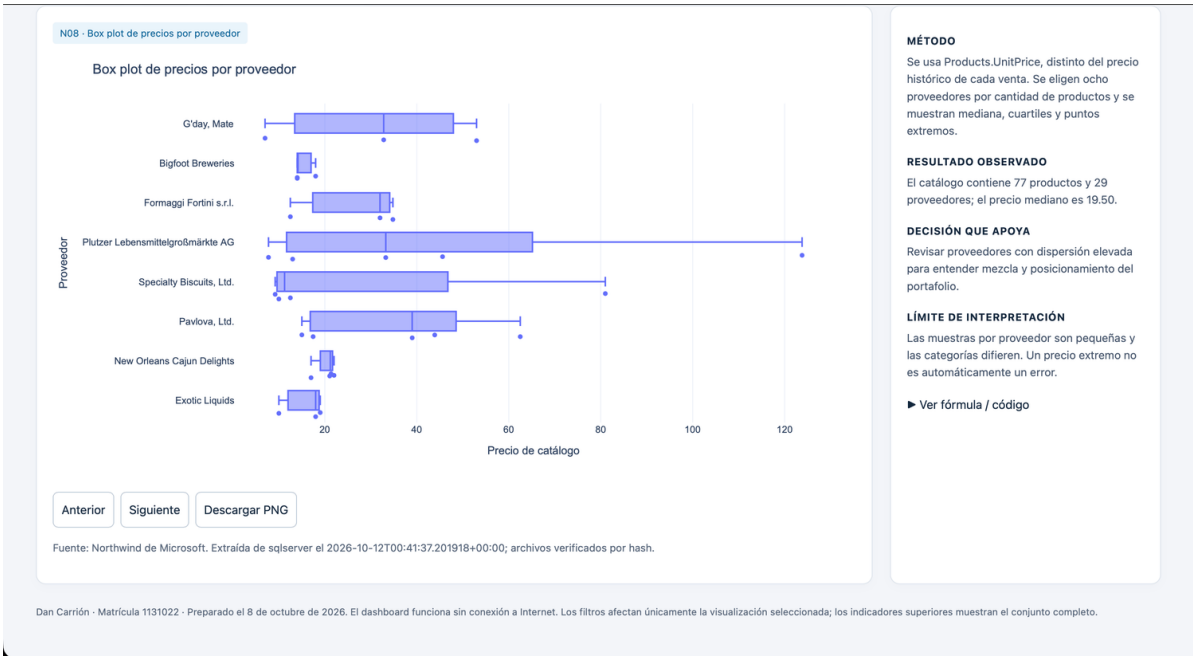



***Figura 58.*** *Dashboard: N08 · Box plot de precios por proveedor.*

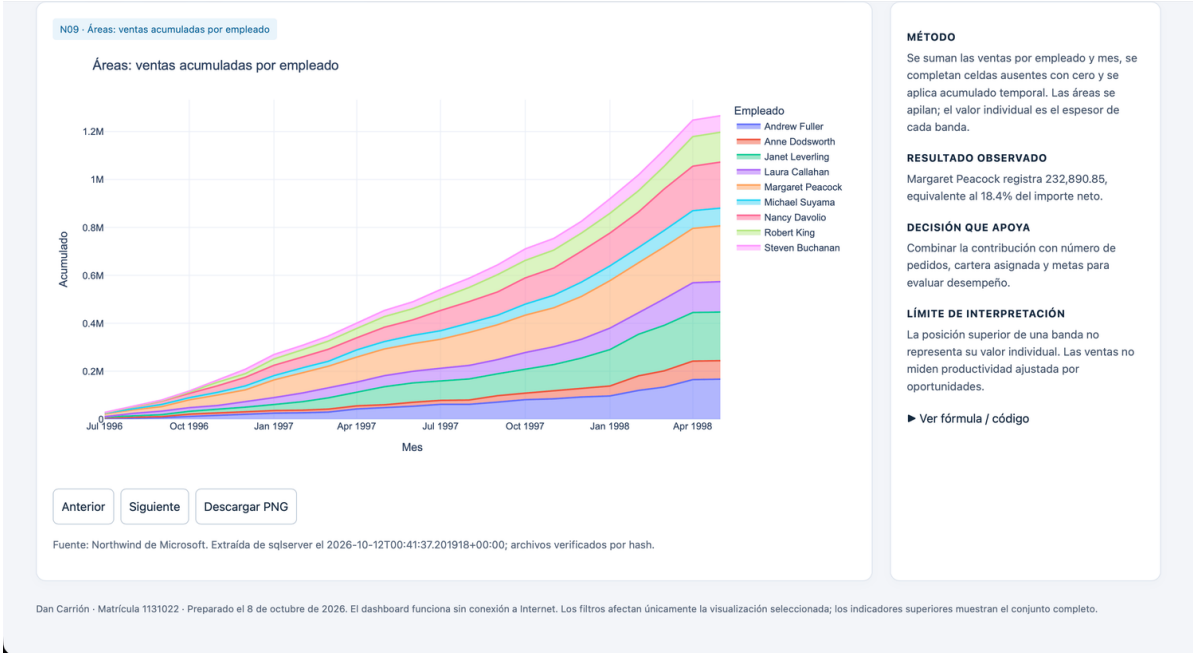



***Figura 59.*** *Dashboard: N09 · Áreas: ventas acumuladas por empleado.*

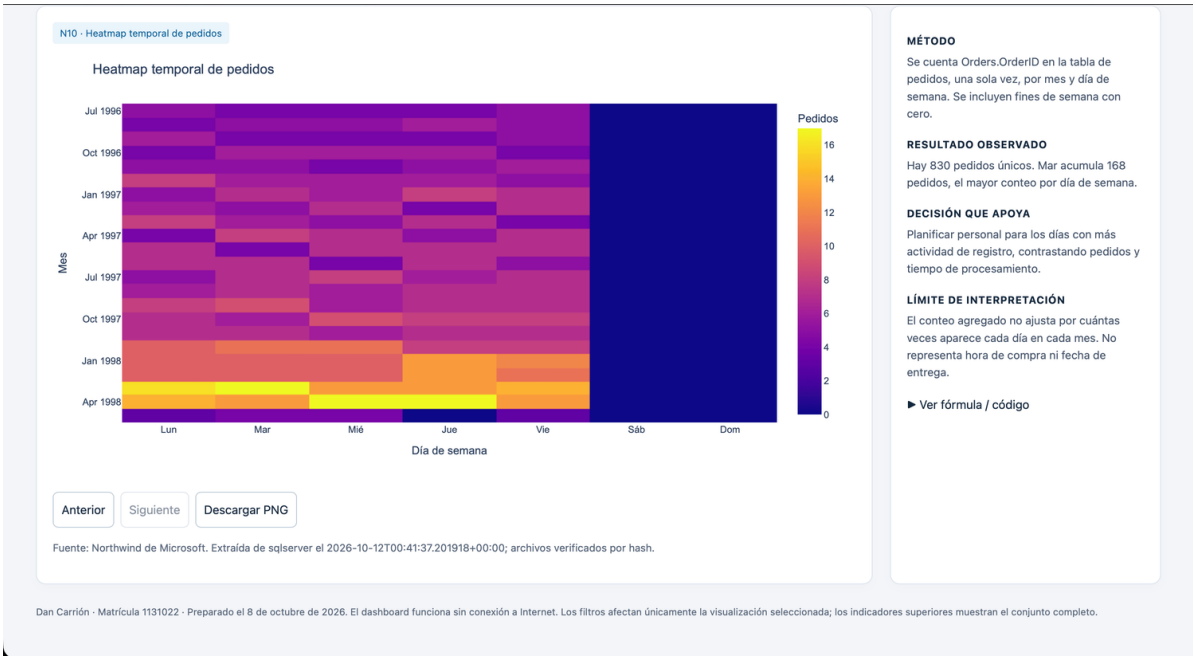



***Figura 60.*** *Dashboard: N10 · Heatmap temporal de pedidos.*

## C.2 Oracle / HR

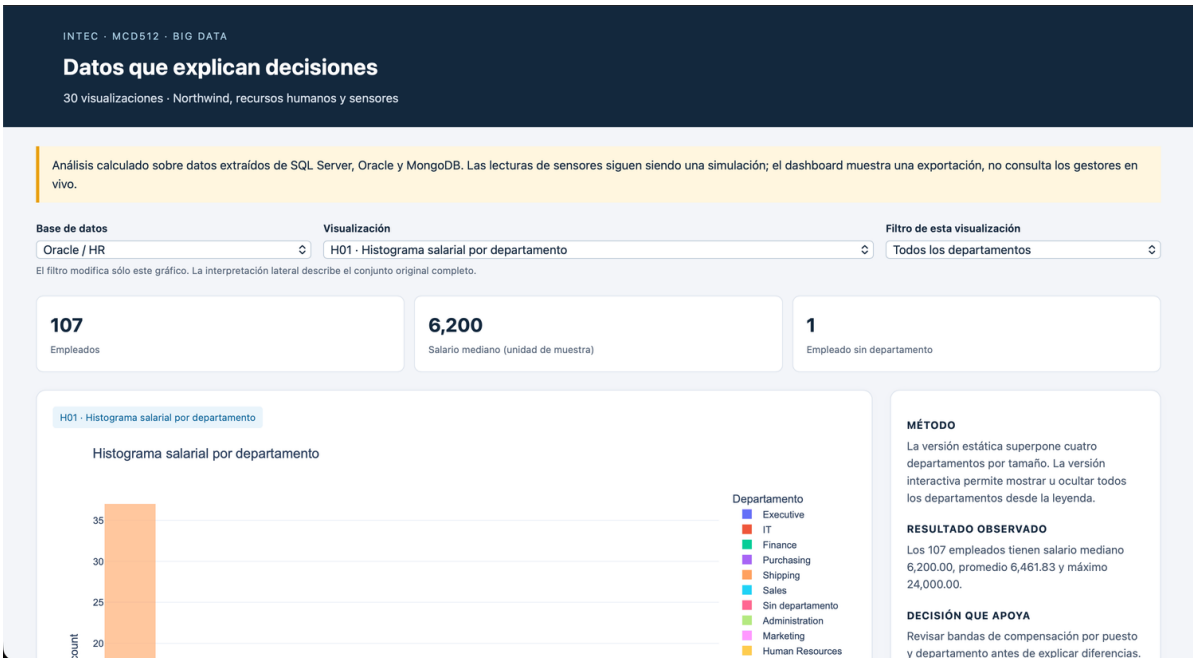



***Figura 61.*** *Dashboard: vista general de Oracle / HR con indicadores clave.*

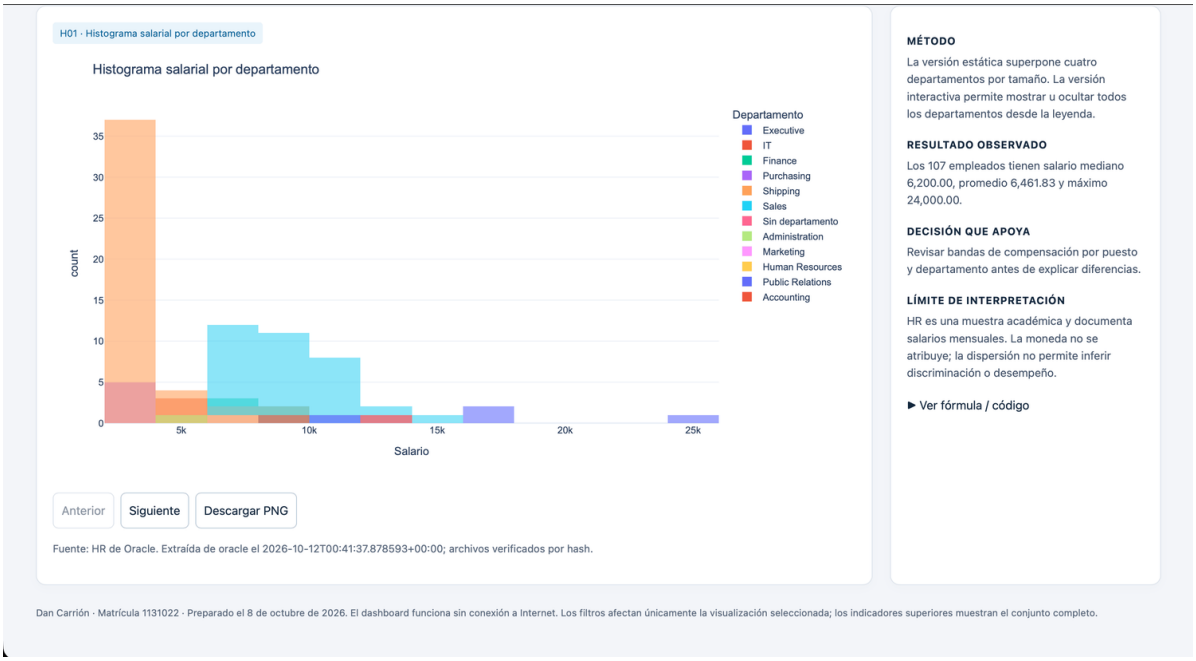



***Figura 62.*** *Dashboard: H01 · Histograma salarial por departamento.*

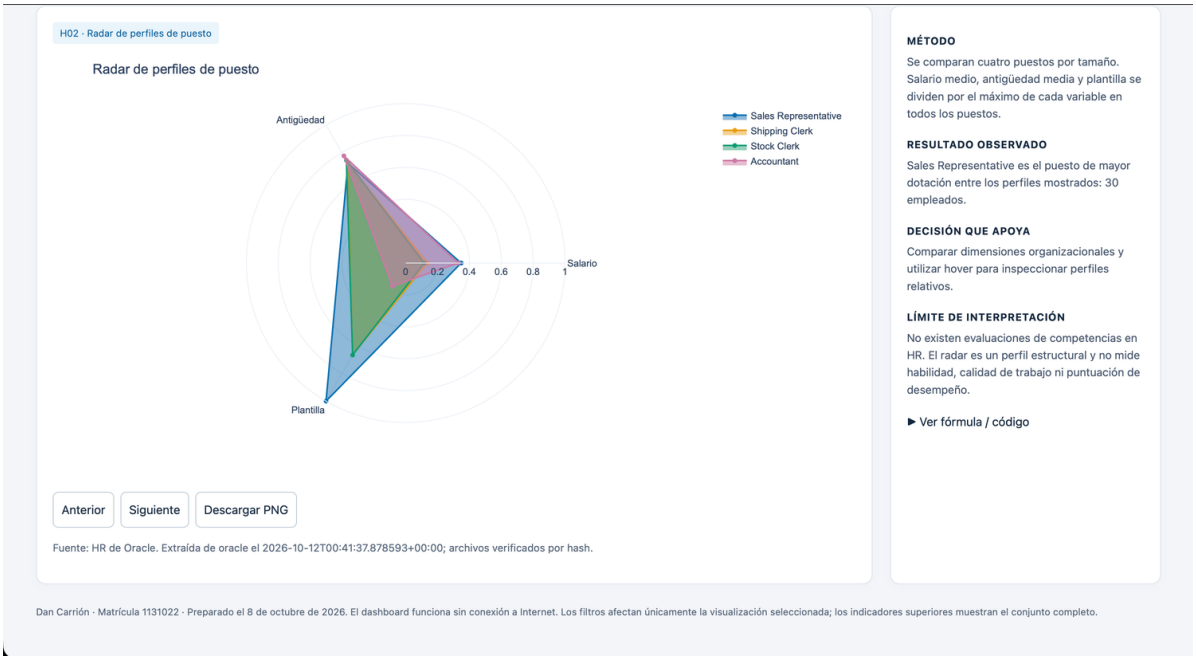



***Figura 63.*** *Dashboard: H02 · Radar de perfiles de puesto.*

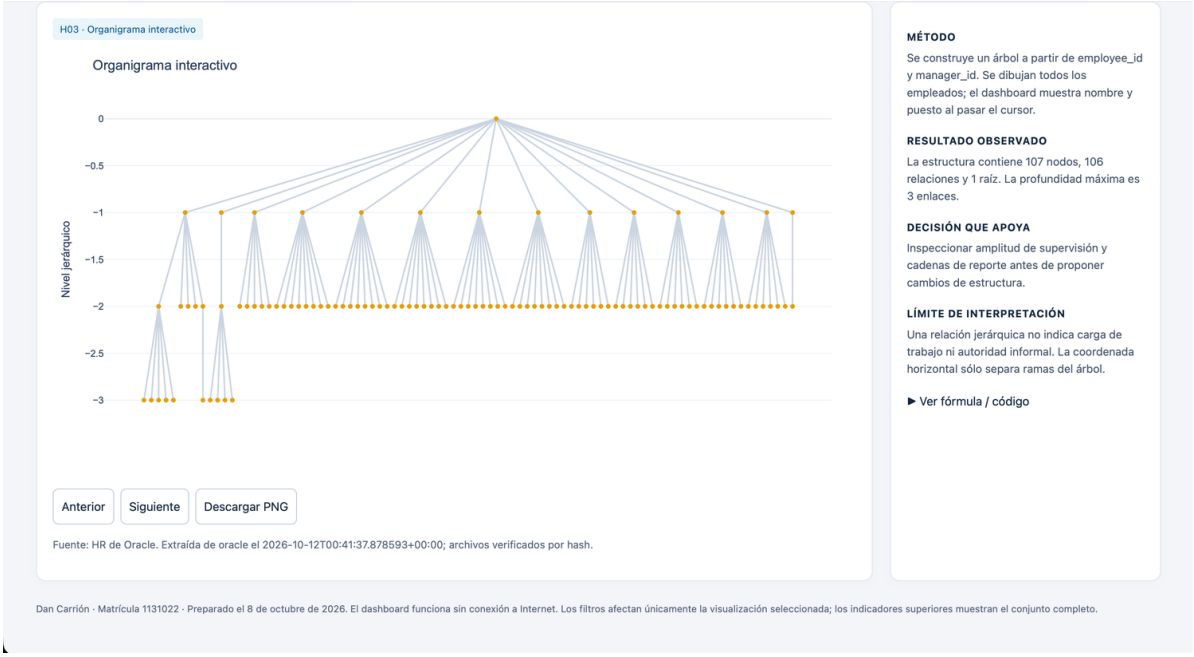



***Figura 64.*** *Dashboard: H03 · Organigrama interactivo.*

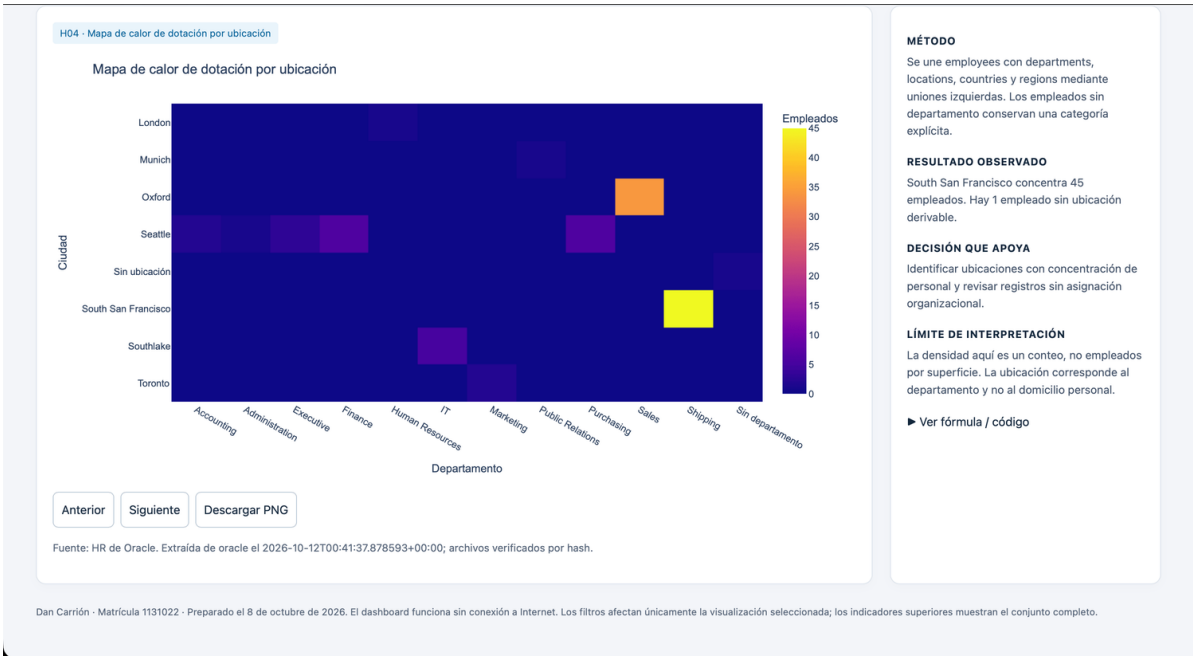



***Figura 65.*** *Dashboard: H04 · Mapa de calor de dotación por ubicación.*

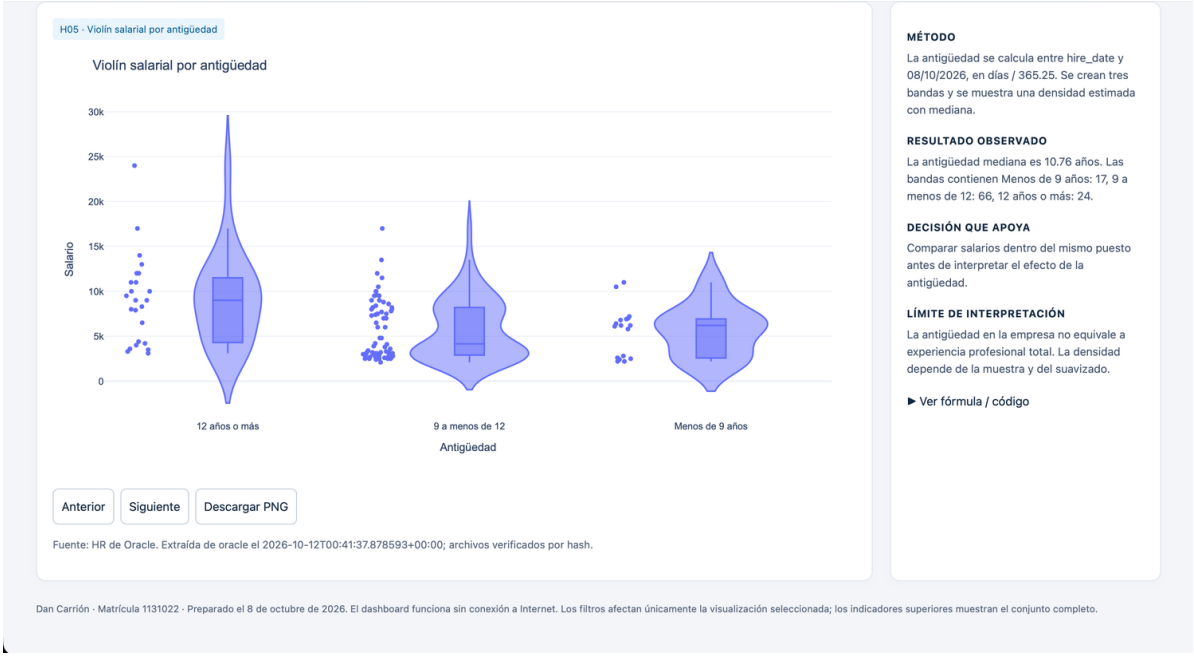



***Figura 66.*** *Dashboard: H05 · Violín salarial por antigüedad.*

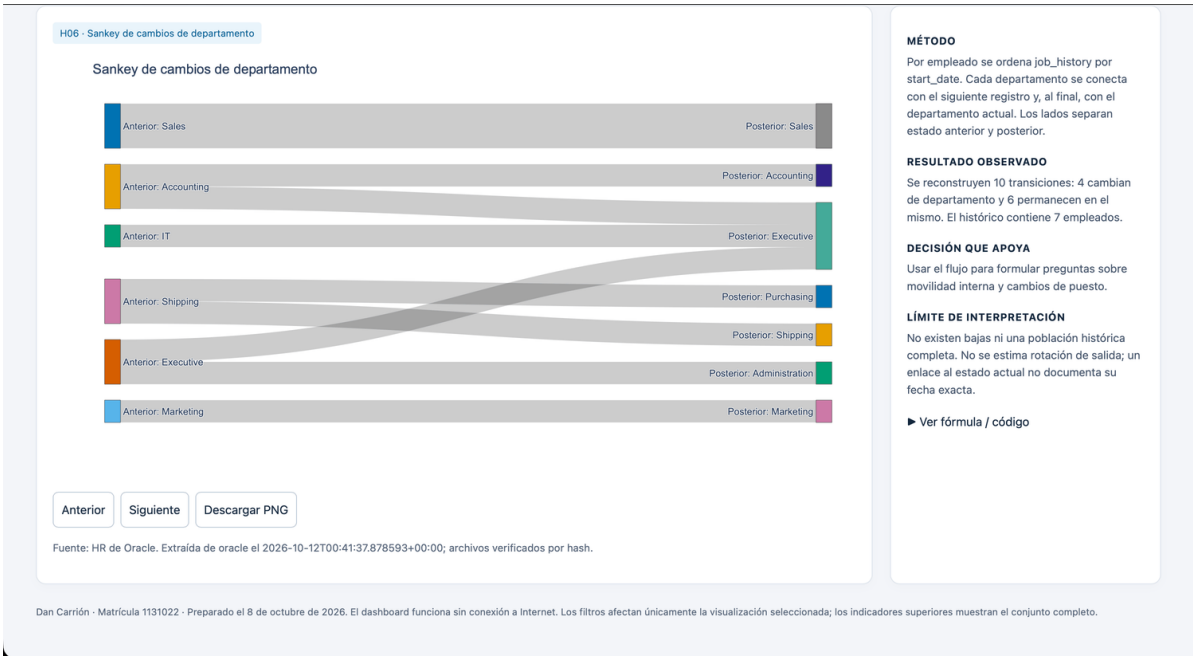



***Figura 67.*** *Dashboard: H06 · Sankey de cambios de departamento.*

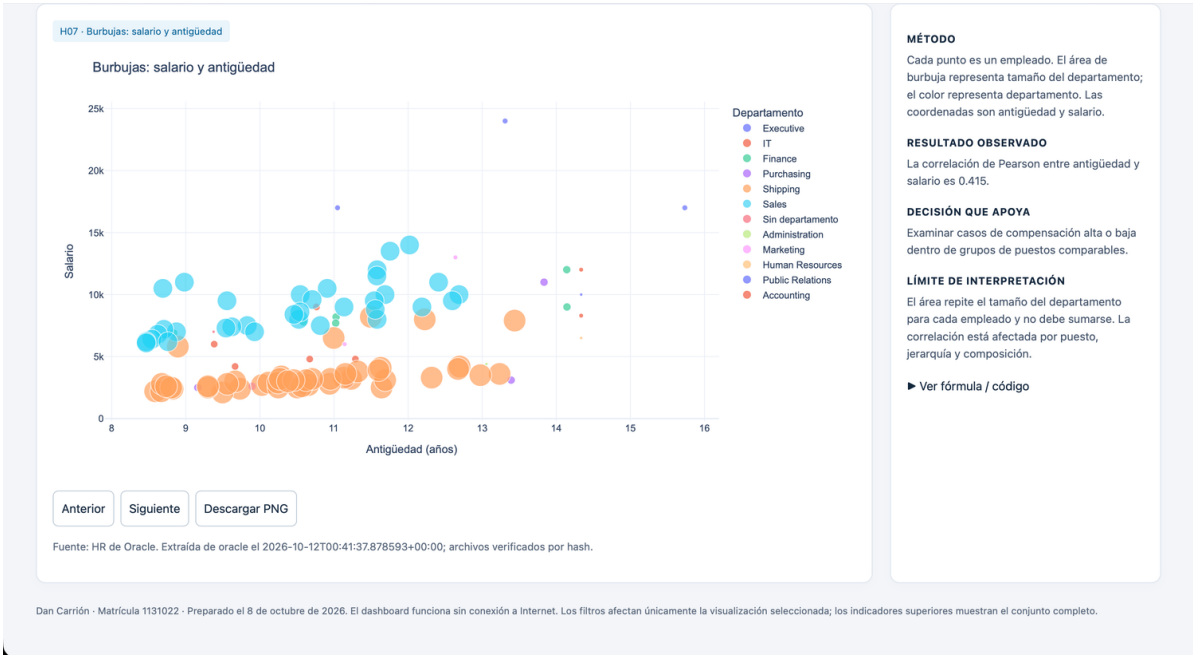



***Figura 68.*** *Dashboard: H07 · Burbujas: salario y antigüedad.*

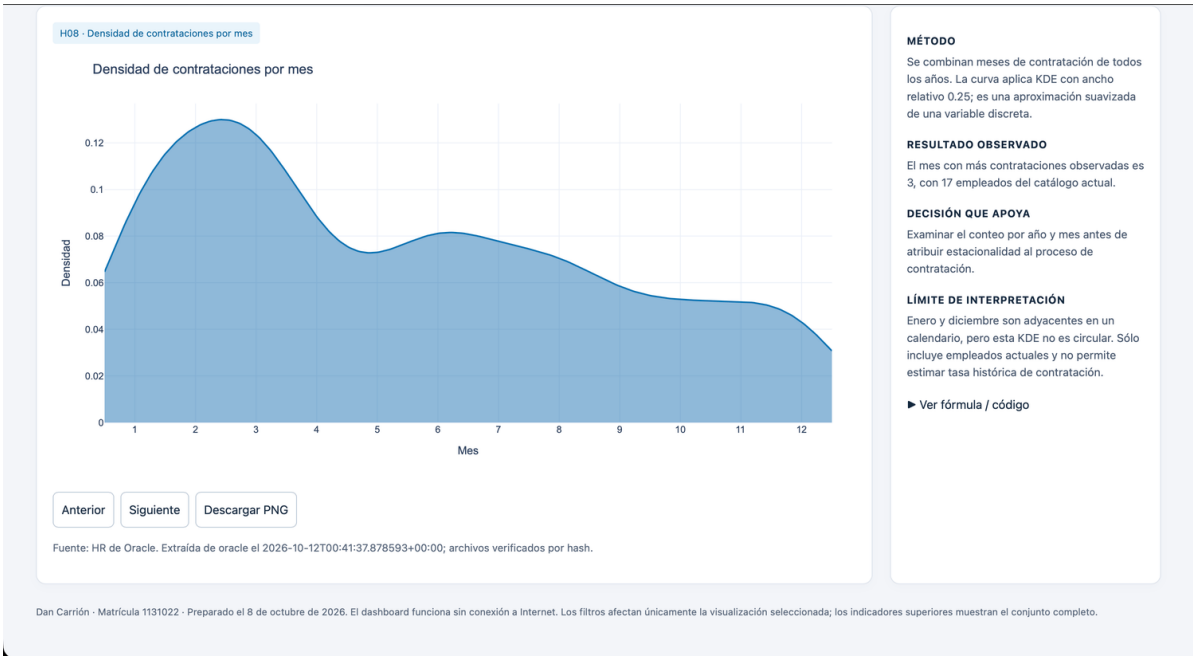



***Figura 69.*** *Dashboard: H08 · Densidad de contrataciones por mes.*

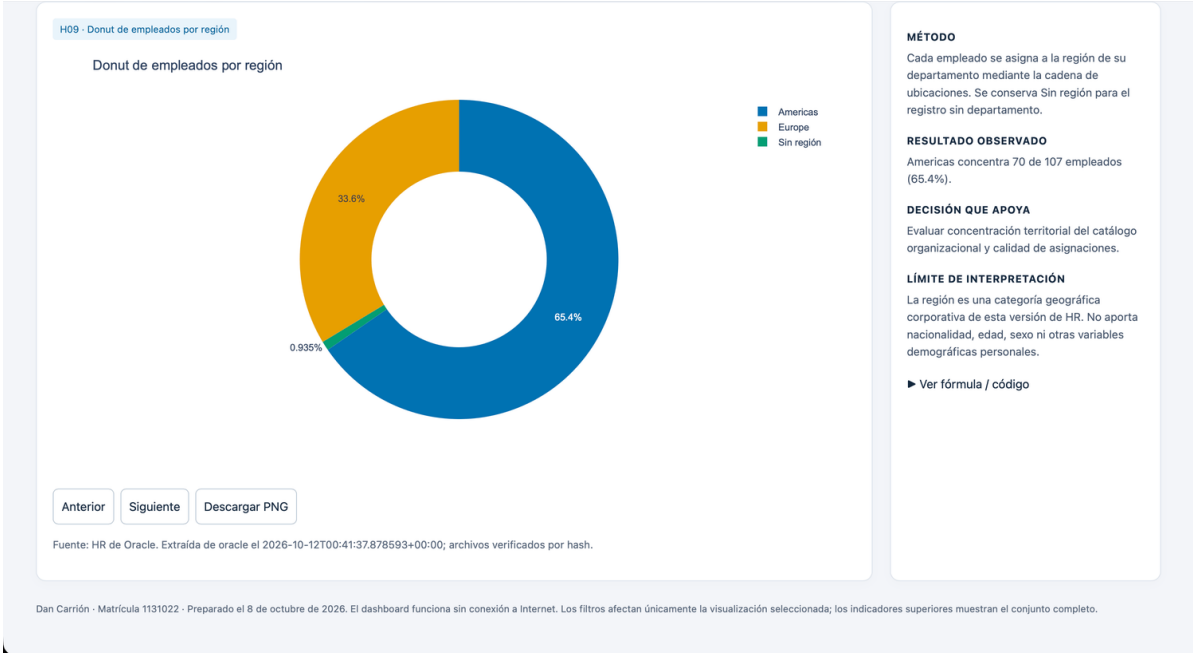



***Figura 70.*** *Dashboard: H09 · Donut de empleados por región.*




***Figura 71.*** *Dashboard: H10 · Waterfall de bandas salariales: escenario ilustrativo.*

## C.3 MongoDB / sensores

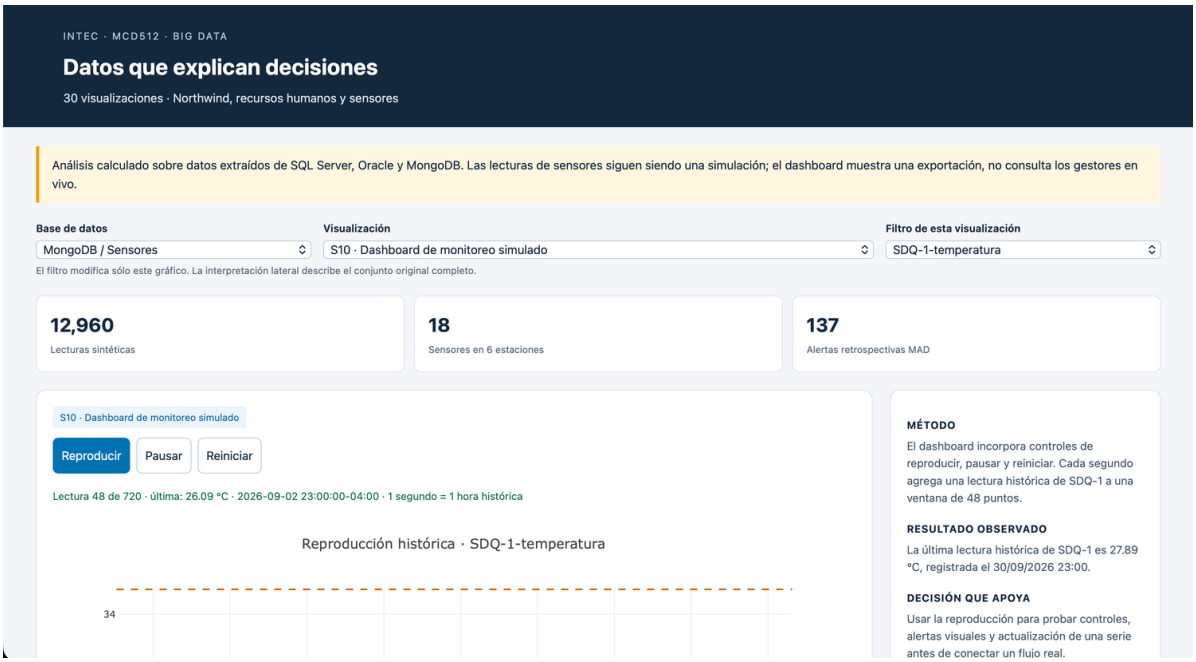



***Figura 72.*** *Dashboard: vista general de MongoDB / sensores con indicadores clave.*




***Figura 73.*** *Dashboard: S01 · Serie temporal de temperatura por ciudad.*

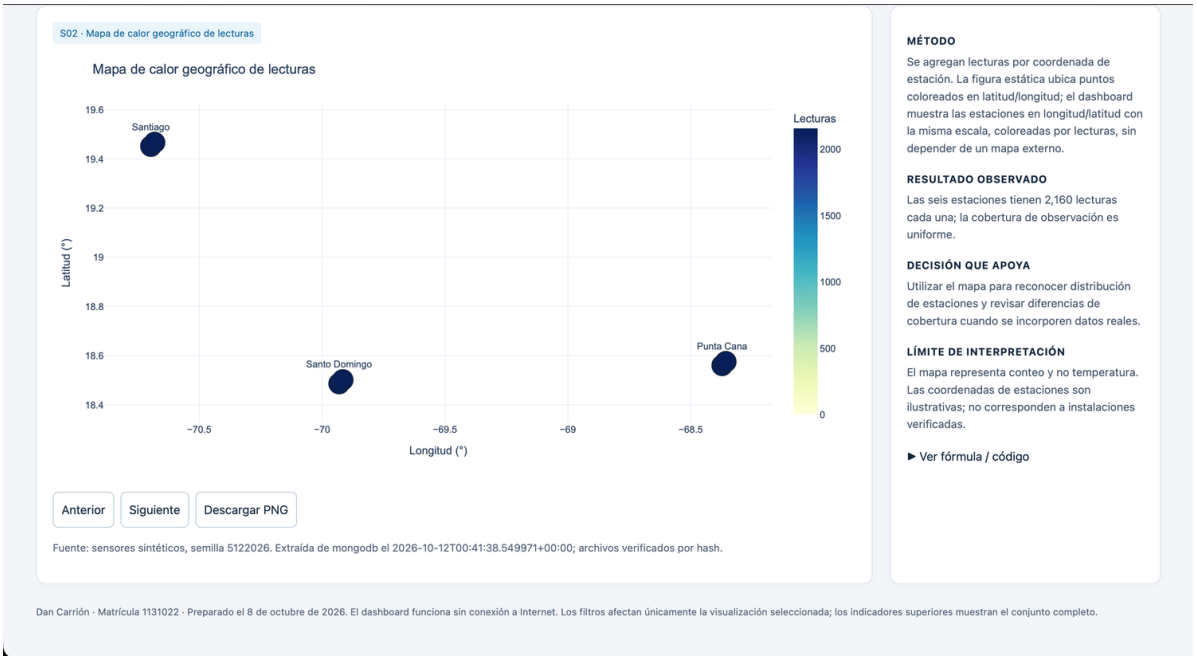



***Figura 74.*** *Dashboard: S02 · Mapa de calor geográfico de lecturas.*

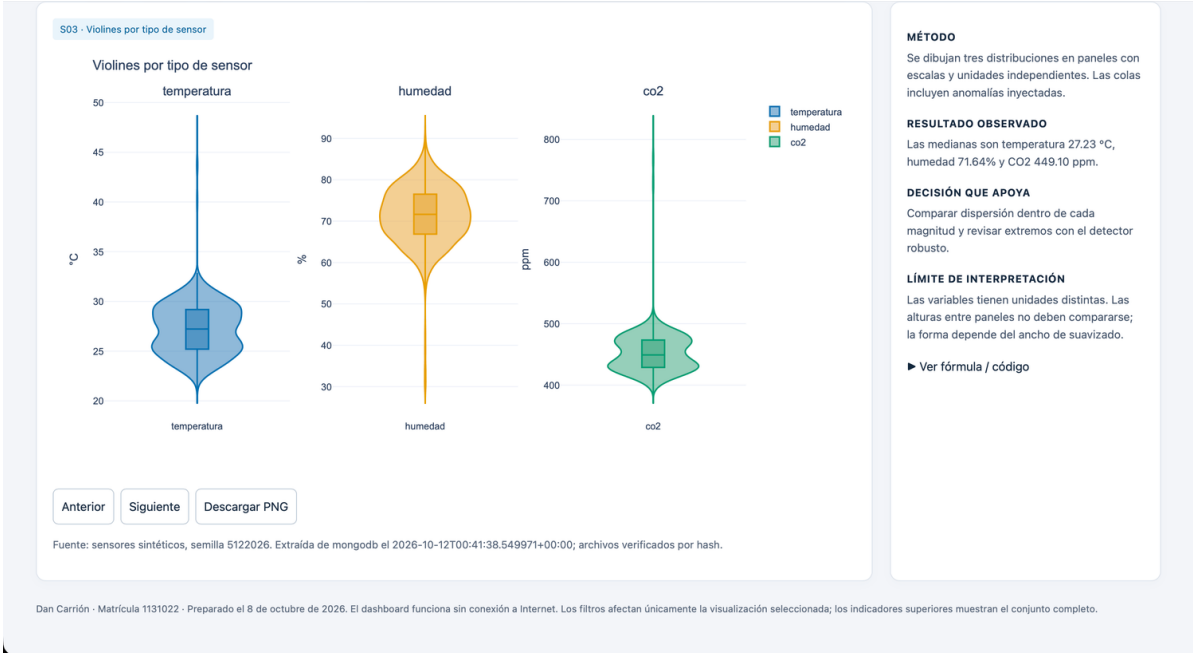



***Figura 75.*** *Dashboard: S03 · Violines por tipo de sensor.*

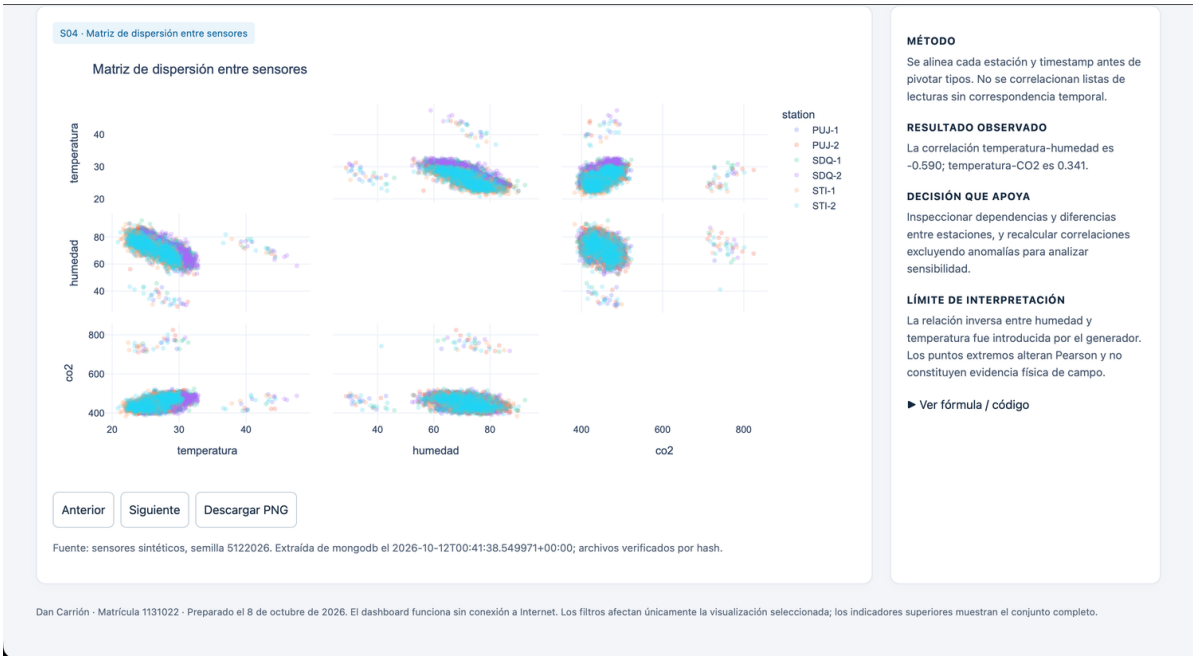



***Figura 76.*** *Dashboard: S04 · Matriz de dispersión entre sensores.*




***Figura 77.*** *Dashboard: S05 · Áreas apiladas por ubicación.*

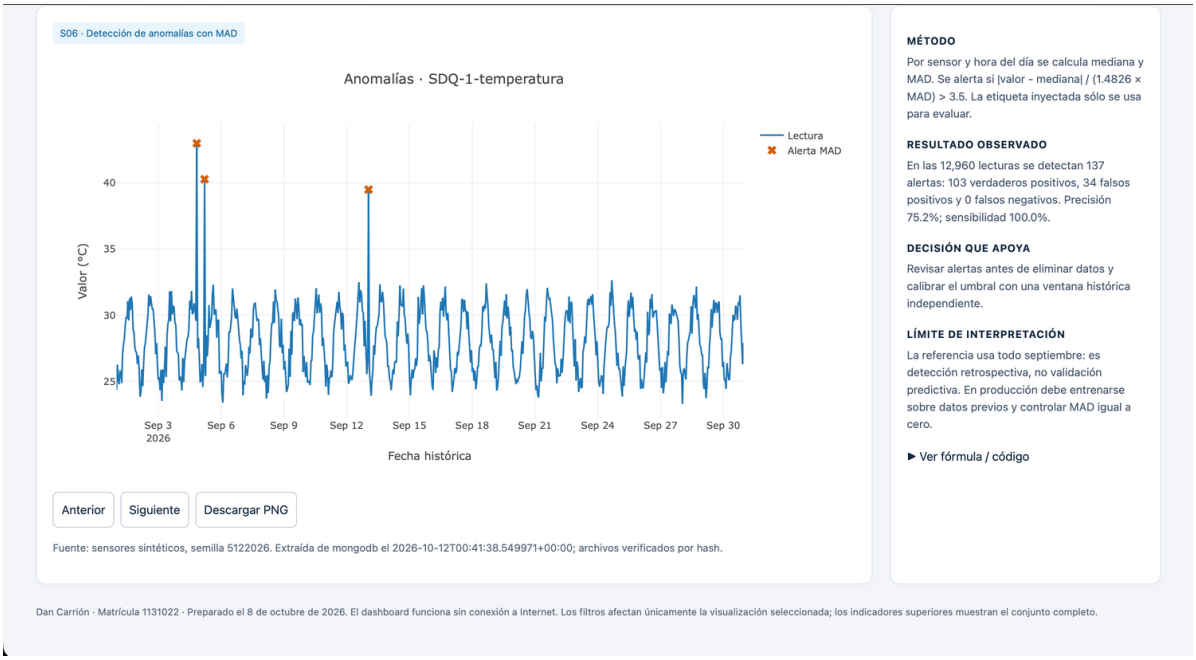



***Figura 78.*** *Dashboard: S06 · Detección de anomalías con MAD.*

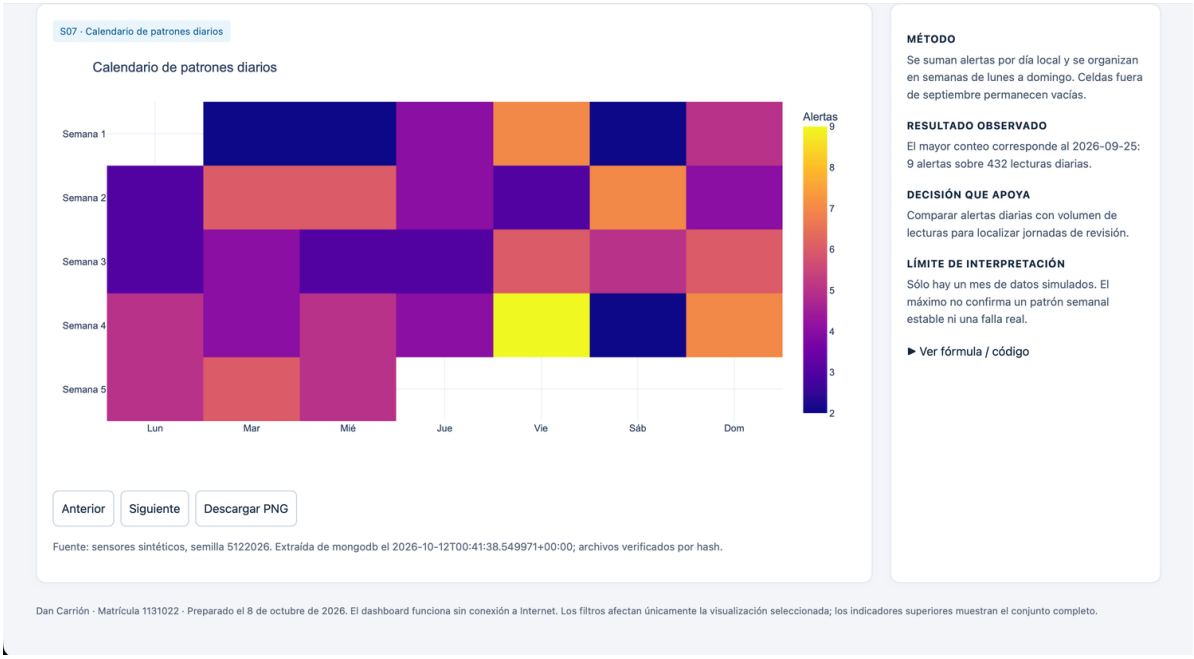



***Figura 79.*** *Dashboard: S07 · Calendario de patrones diarios.*

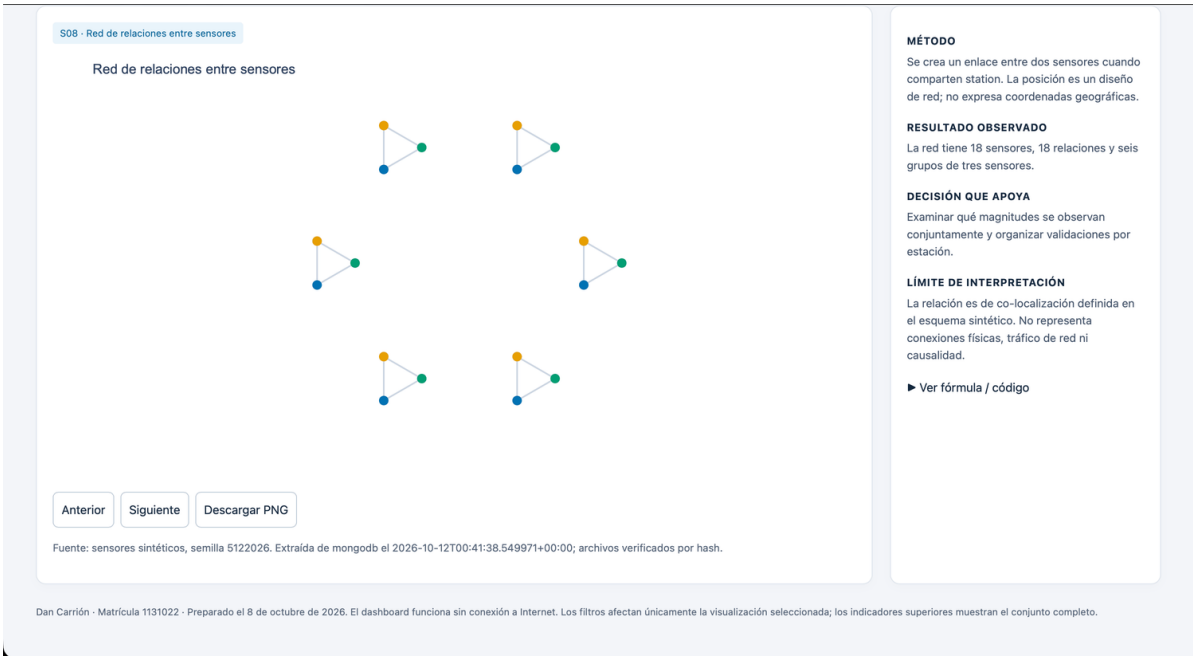



***Figura 80.*** *Dashboard: S08 · Red de relaciones entre sensores.*




***Figura 81.*** *Dashboard: S09 · Dispersión 3D: valor, ubicación y tiempo.*

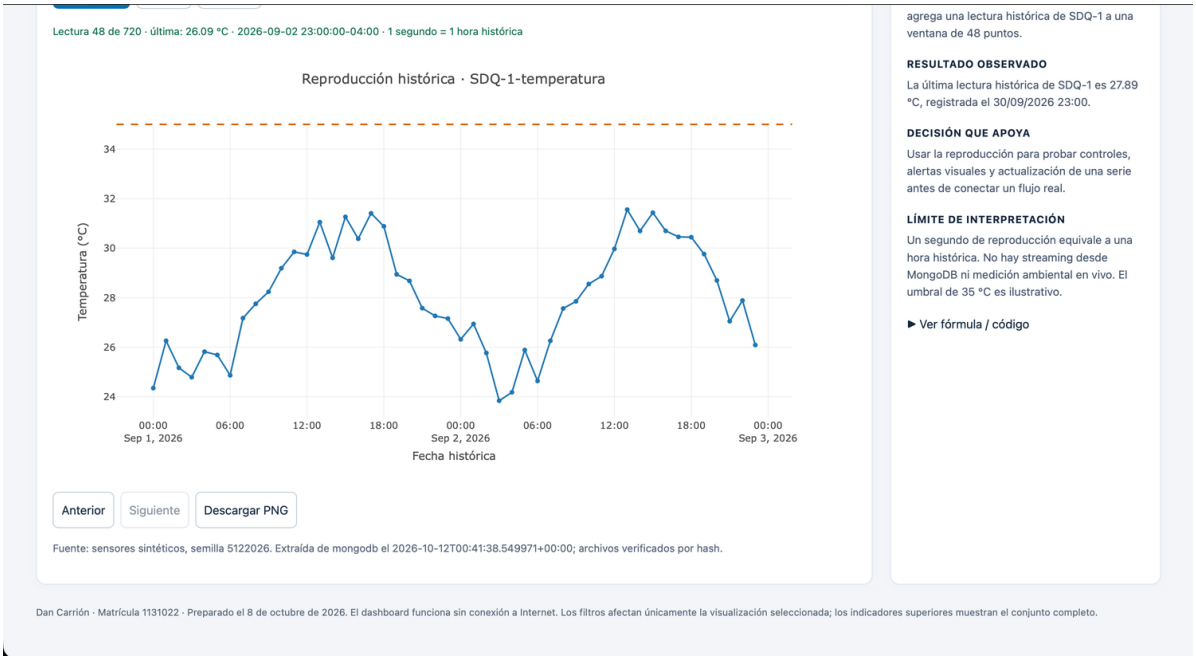



***Figura 82.*** *Dashboard: S10 · Dashboard de monitoreo simulado.*